

Software Analysis

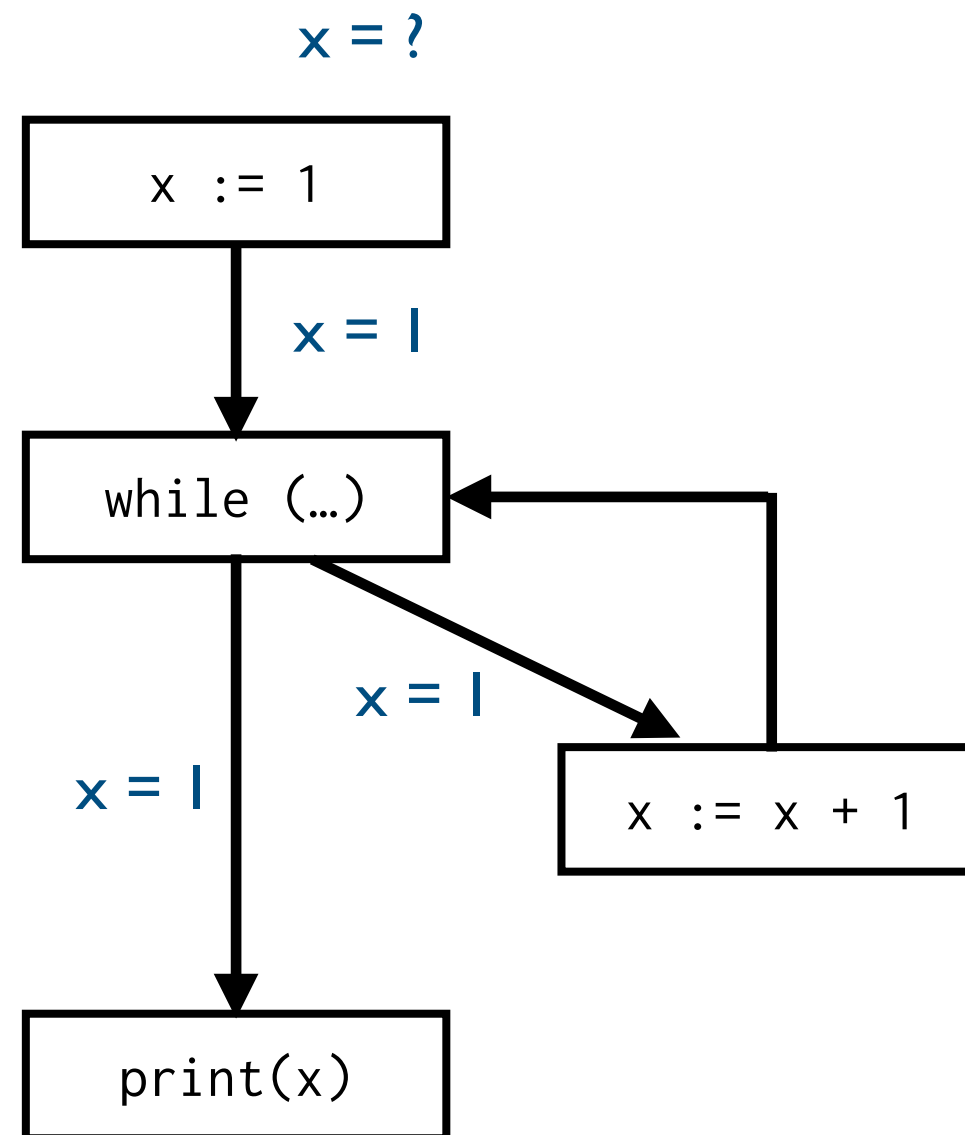
Dataflow Analysis (Part 2)
Abstract Interpretation

Gordon Fraser
Lehrstuhl für Software Engineering II

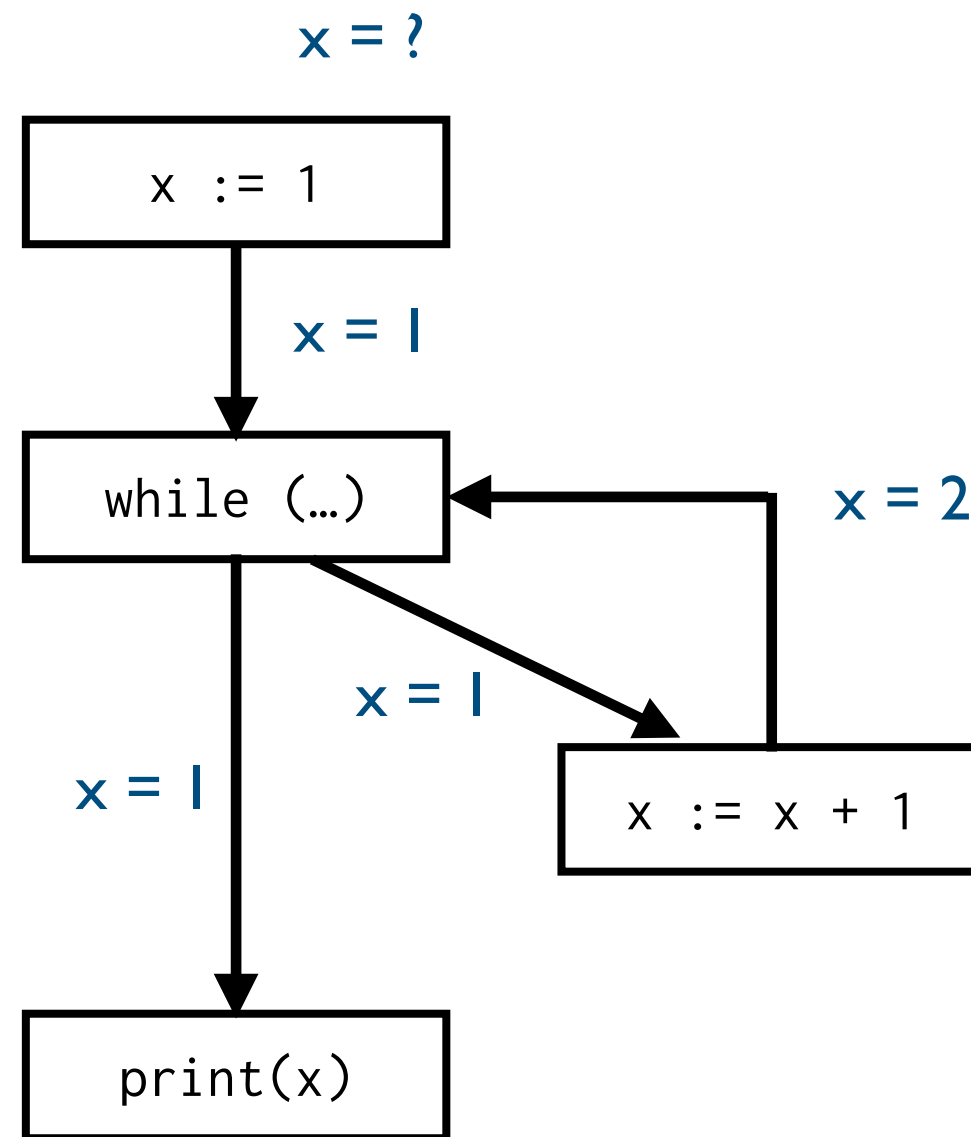
Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis
5. Lattice-Theoretic Framework
6. Example: Zero Analysis

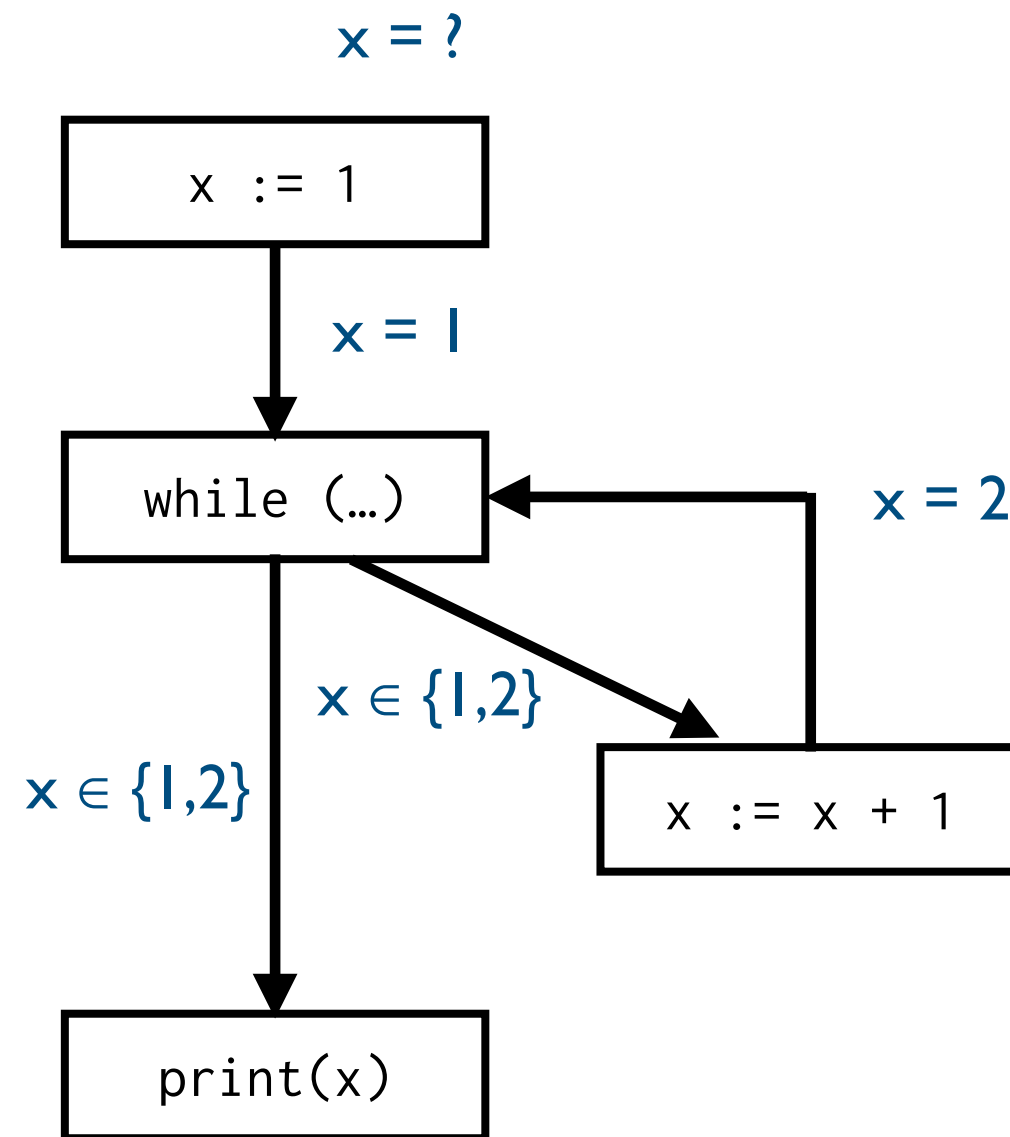
Example: Loops



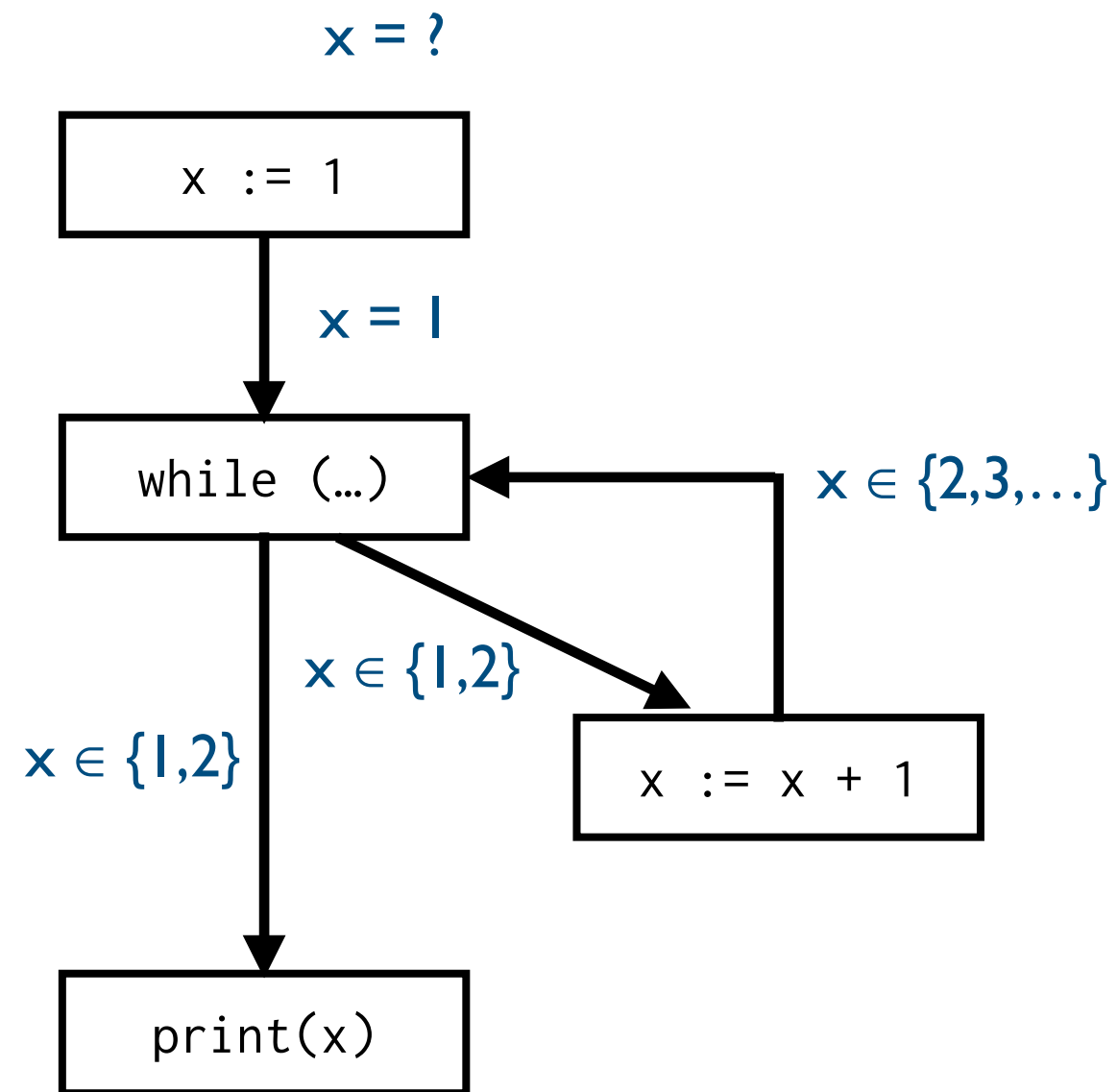
Example: Loops



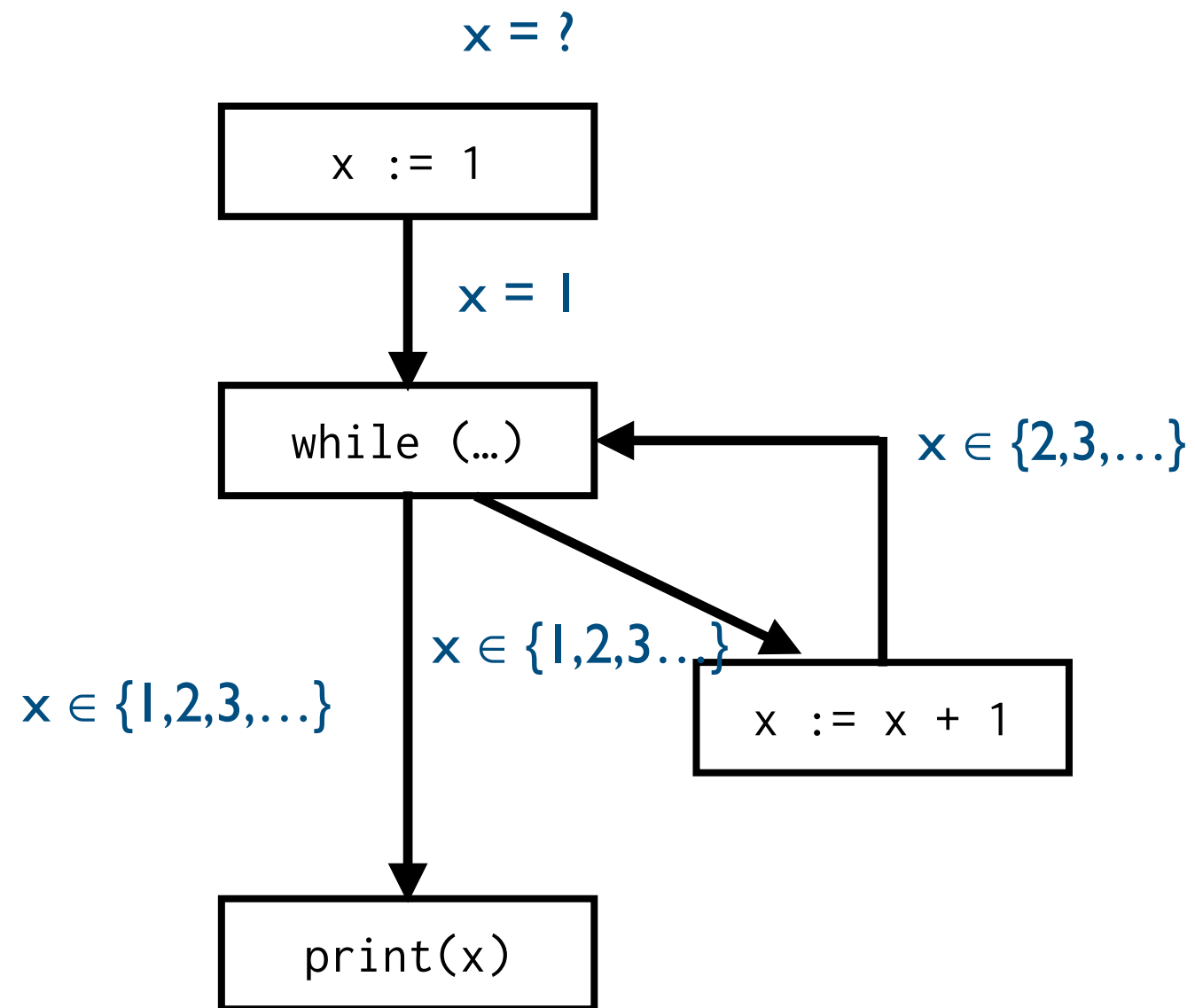
Example: Loops



Example: Loops



Example: Loops



Lattice-Theoretic Framework

- Provide a single formal model that describes all data flow analyses
- Formalize the notions of *correct*, *conservative*, and *optimistic*
- Prove correctness for iterative data flow analysis
- Place bounds on time complexity of data flow analysis

Lattice-Theoretic Framework

- Define domain of data flow values computed by dataflow analysis, and **organize** the domain of elements as a **lattice**
- Define **flow functions** and a **merge function** over this domain using **lattice operations**
- Exploit lattice theory in achieving goals (e.g., lattice height)

Partial Orders

A partial order is a pair $(P, \leq)$ such that

- $\leq \subseteq P \times P$ binary relation
- $\leq$ is reflexive: $x \leq x$
- $\leq$ is anti-symmetric: $x \leq y$ and $y \leq x \Rightarrow x = y$
- $\leq$ is transitive: $x \leq y$ and $y \leq z \Rightarrow x \leq z$

Partially ordered set: poset

Lattice

- A lattice is a **partially ordered set** in which every two elements have a unique **supremum** ($\sqcup$) and a unique **infimum** ($\sqcap$)
- Supremum is also called a **least upper bound** or **join**
- Infimum is also called a **greatest lower bound** or **meet**

$(P, \leq)$

$$x \leq x \sqcup y \text{ and } y \leq x \sqcup y$$

$$\text{if } x \leq z \text{ and } y \leq z, \text{ then } x \sqcup y \leq z$$

$$x \sqcap y \leq x \text{ and } x \sqcap y \leq y$$

$$\text{if } z \leq x \text{ and } z \leq y, \text{ then } z \leq x \sqcap y$$

Upper Bound

If $S \subseteq P$ then

$x \in P$ is an upper bound of S if $[\forall y \in S \mid y \leq x]$

$x \in P$ is the least upper bound of S if

x is an upper bound of S , and

$x \leq y$ for all upper bounds y of S

$\sqcup$ - join, least upper bound, lub, supremum, sup

$\sqcup S$ is the least upper bound of S

$x \sqcup y$ is the least upper bound of $\{x, y\}$

$x \leq x \sqcup y$ and $y \leq x \sqcup y$

if $x \leq z$ and $y \leq z$ then $x \sqcup y \leq z$

Lower Bound

If $S \subseteq P$ then

$x \in P$ is a lower bound of S if for $\forall y \in S. x \leq y$

$x \in P$ is the greatest lower bound of S if

x is a lower bound of S , and

$y \leq x$ for all lower bounds y of S

$\sqcap$ - meet, greatest lower bound, glb, infimum, inf

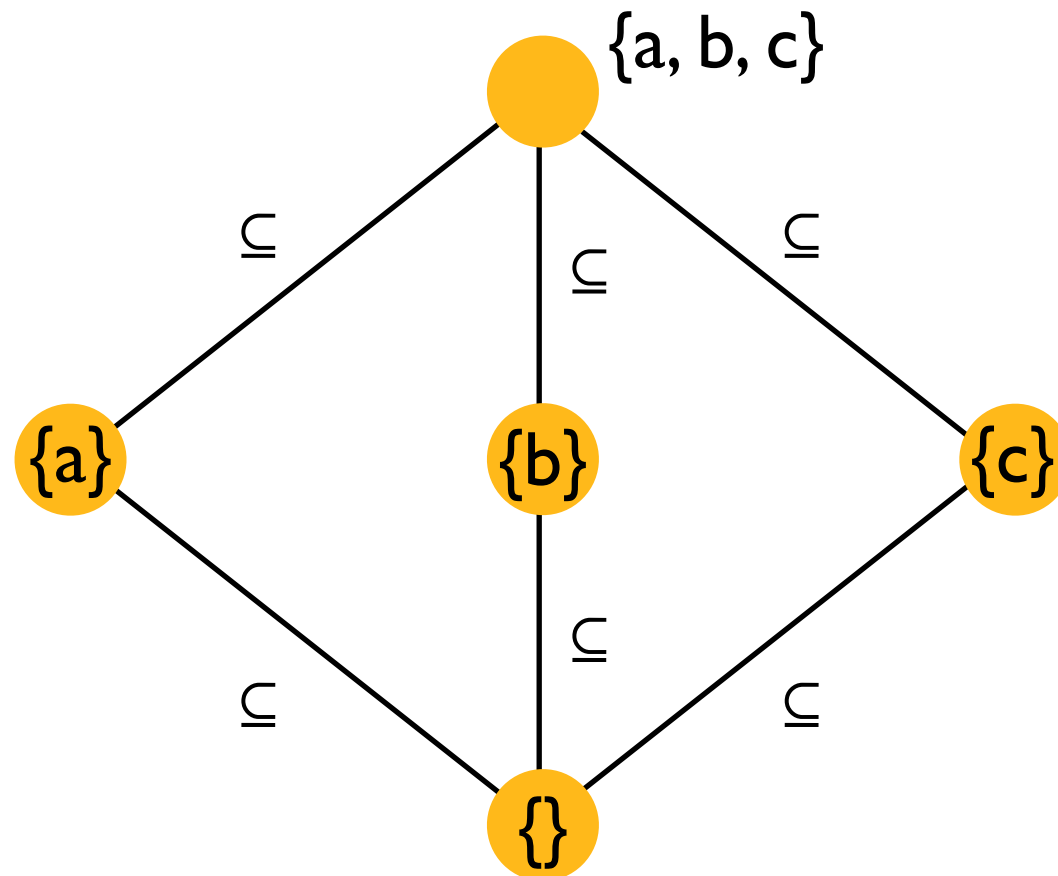
$\sqcap S$ is the greatest lower bound of S

$x \sqcap y$ is the greatest lower bound of $\{x, y\}$

$x \sqcap y \leq x$ and $x \sqcap y \leq y$

if $z \leq x$ and $z \leq y$ then $z \leq x \sqcap y$

Hasse Diagram

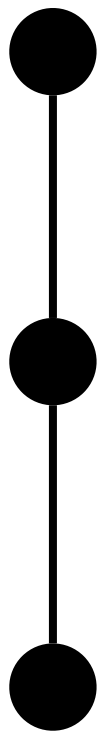


- If $p < q$ in the poset, then the point corresponding to p appears lower in the drawing than the point corresponding to q .
- The two points p and q will be joined by line segment if p is related to q .

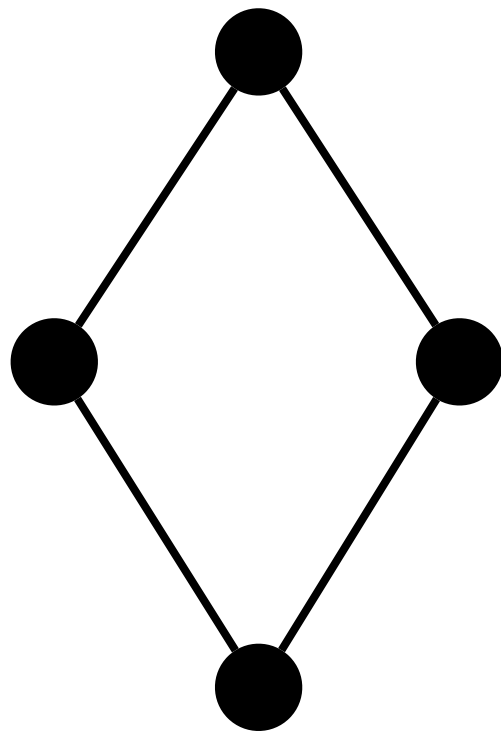
Lattice or not Lattice?



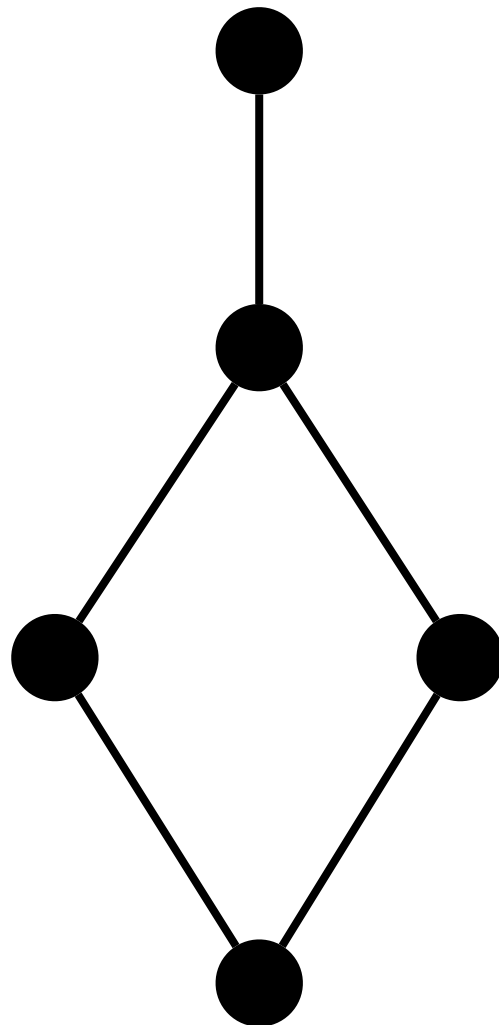
Yes



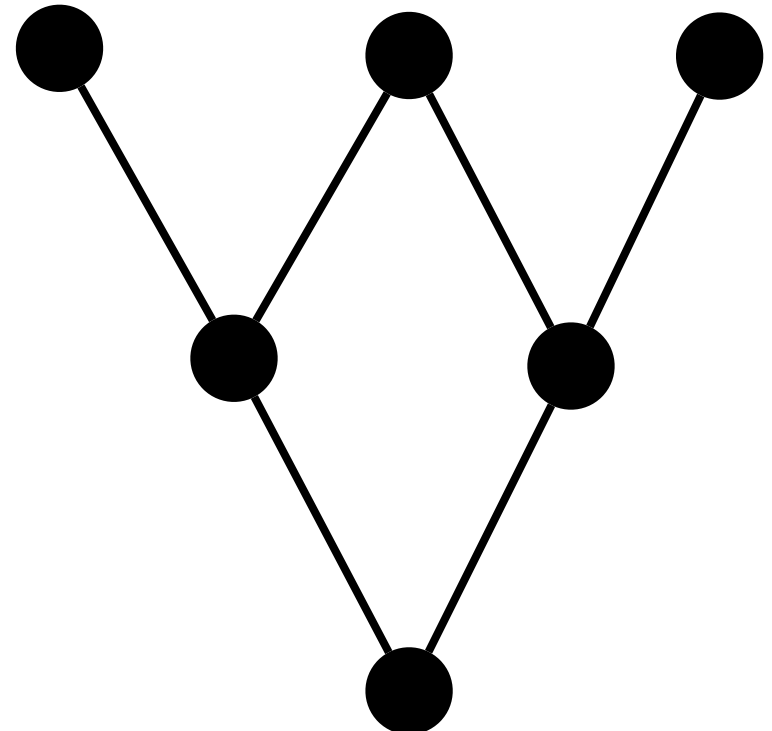
Yes



Yes



Yes

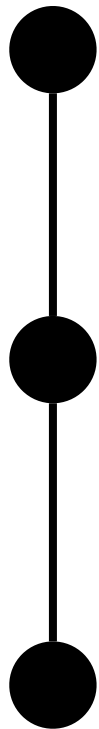


No

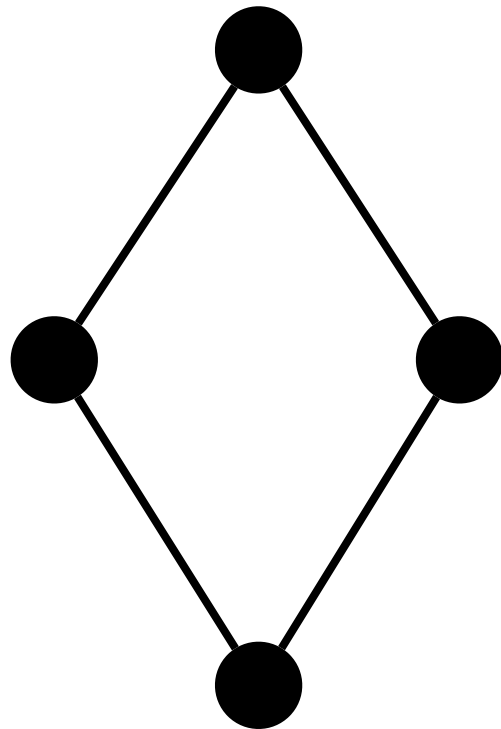
Lattice or not Lattice?



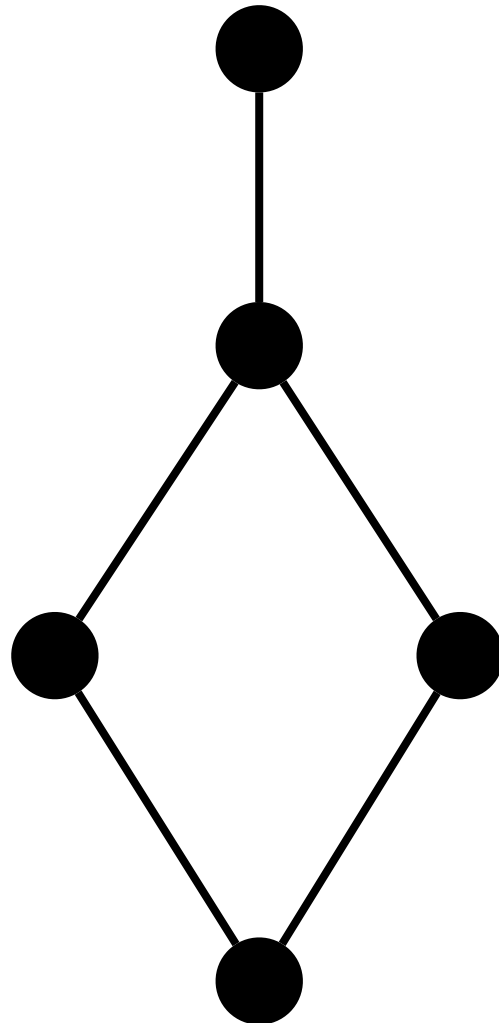
Yes



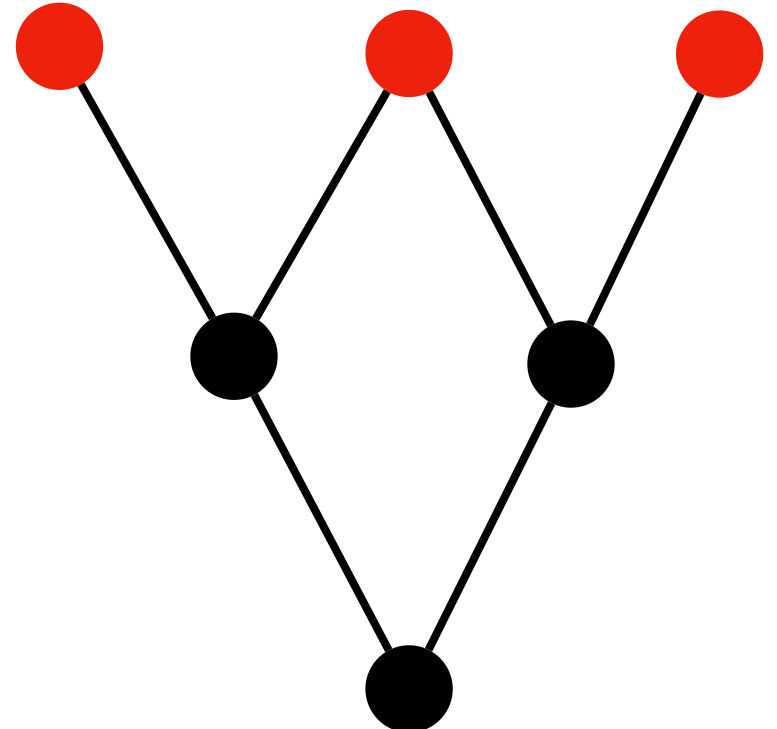
Yes



Yes

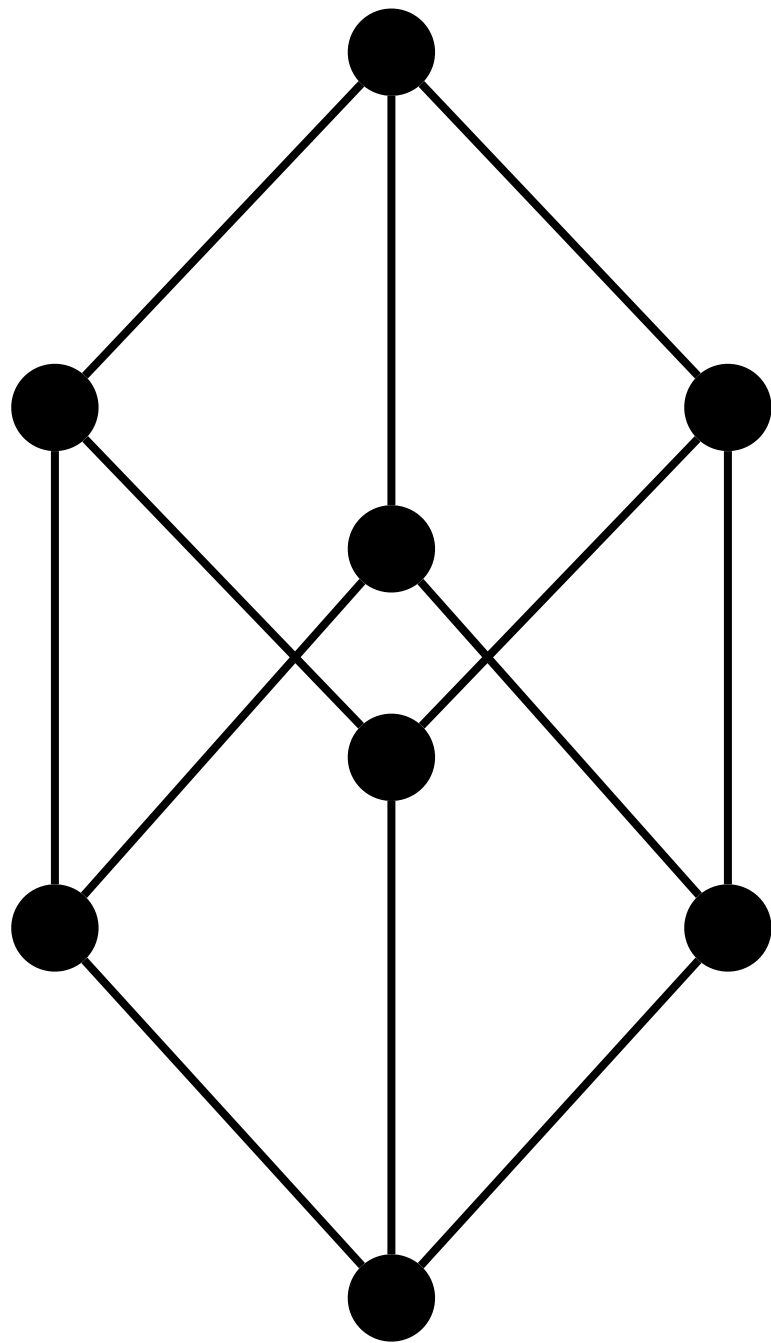


Yes

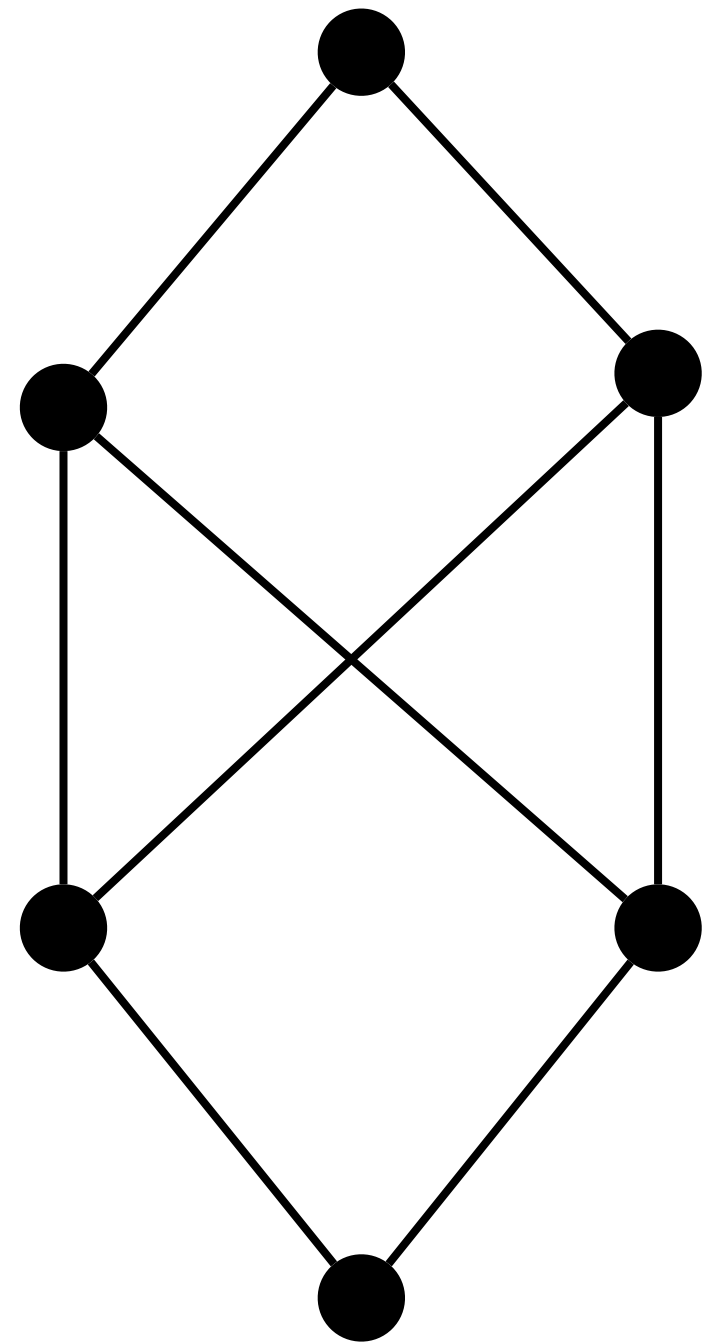


No

Lattice or not Lattice?

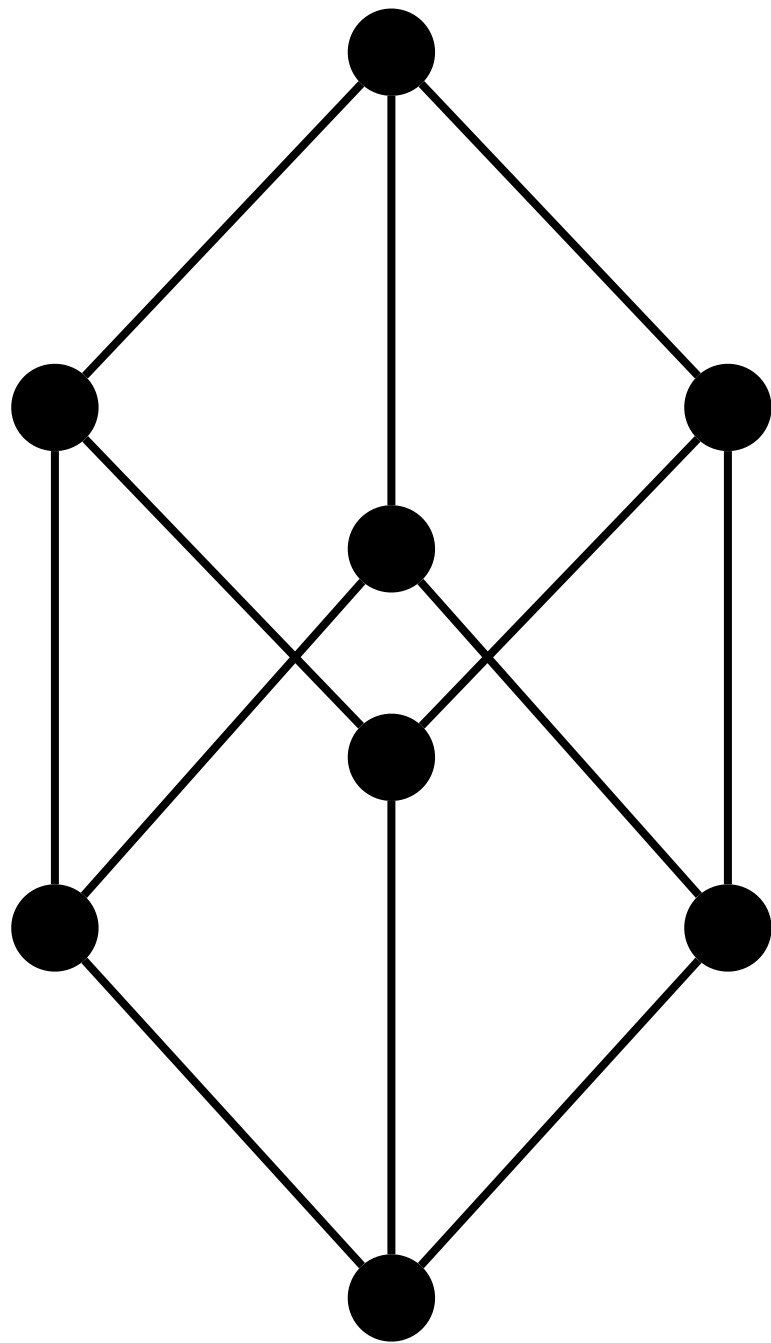


Yes

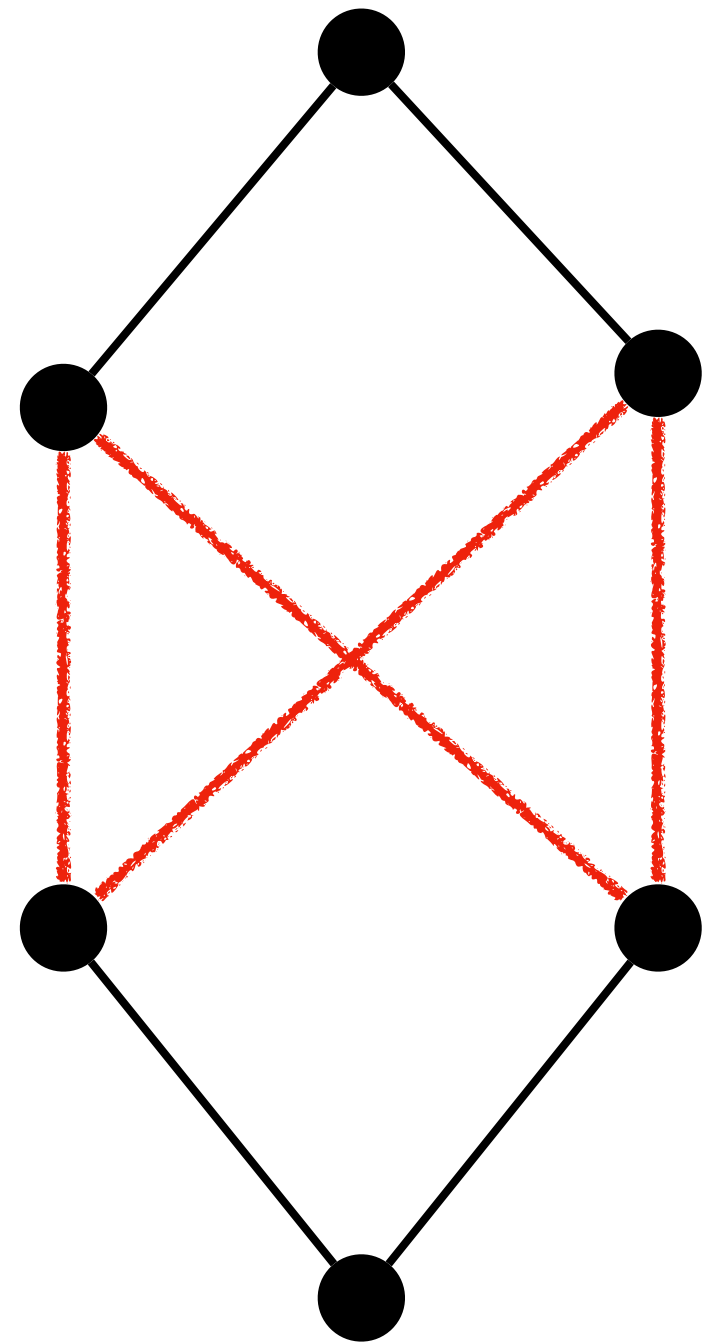


No

Lattice or not Lattice?



Yes



No

More on Lattices

- **Top ($\top$)**

- The unique “greatest” element of P (if it exists)
- $\forall x \in P - \{\top\}, x \leq \top \quad x \sqcap \top = x \quad x \sqcup \top = \top$

- **Bottom ($\perp$)**

- A unique “least” element of P (if it exists)
- $\forall x \in P - \{\perp\}, \perp \leq x \quad x \sqcap \perp = \perp \quad x \sqcup \perp = x$

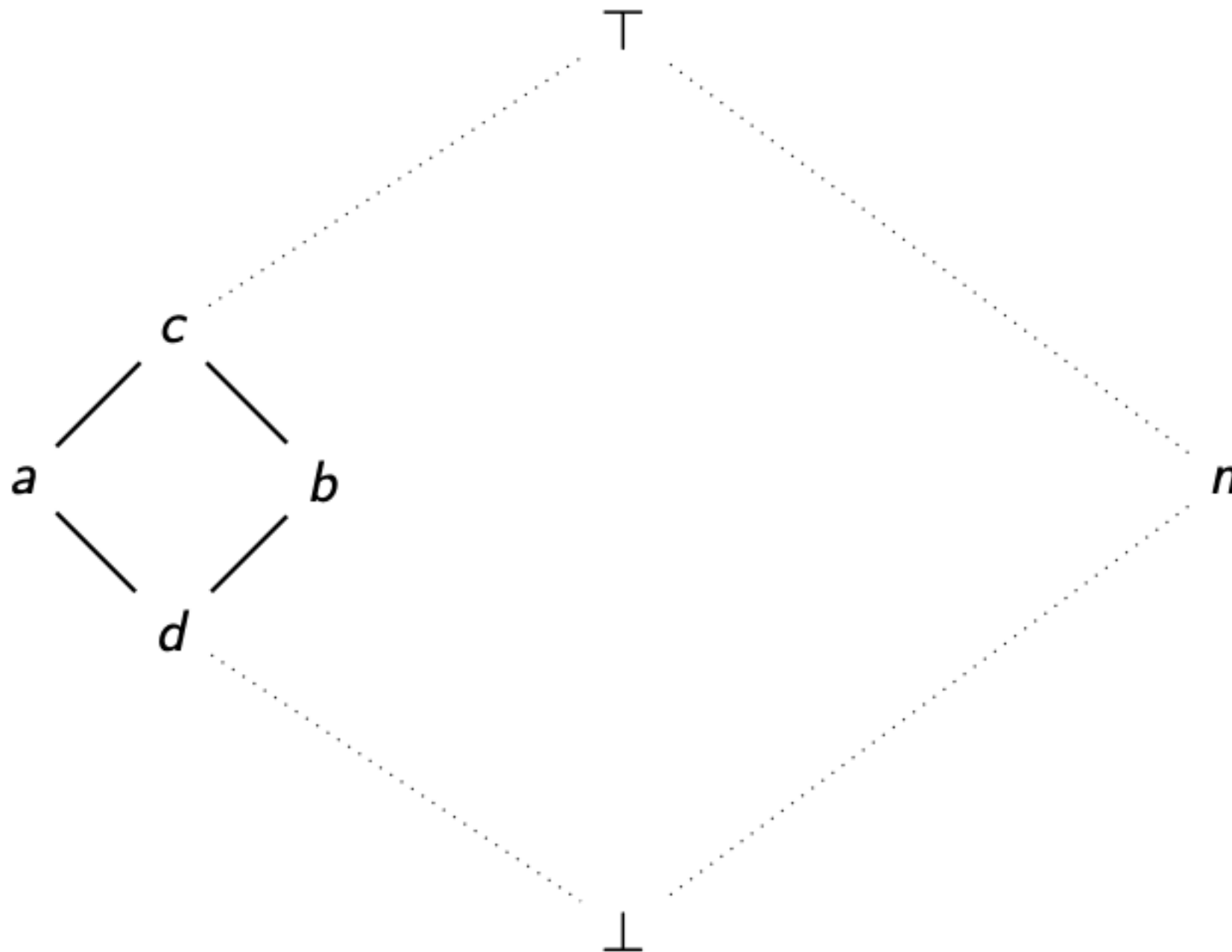
- **Height of lattice (L)**

- The longest path through the partial order from greatest to least element (top to bottom)

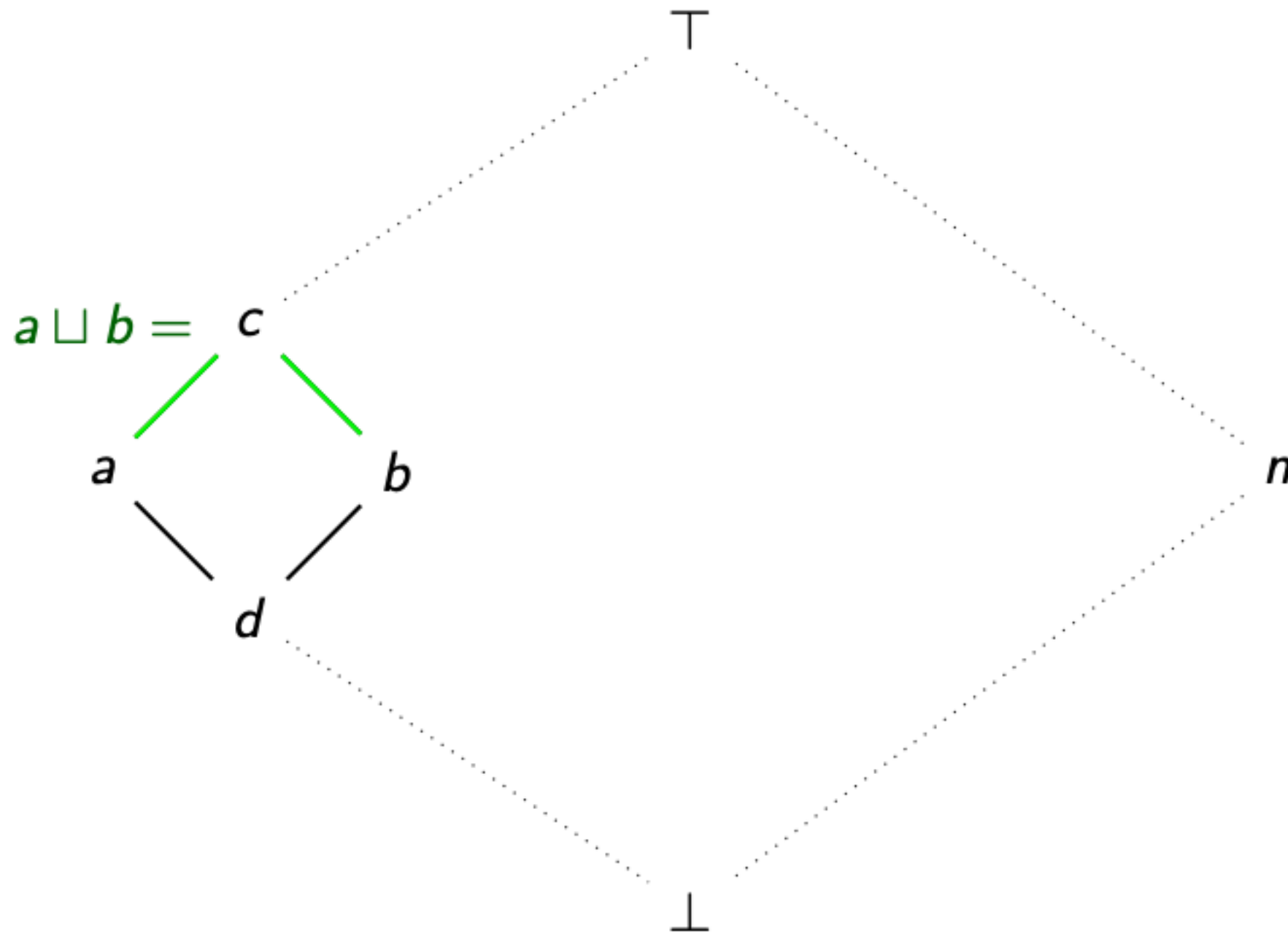
Lattices, Lattices, Lattices

- A semi-lattice, has only the join (or meet) operator defined
- A lattice is **complete** if meet and join are defined for all subsets (i.e., not just for all pairs)
 - If $\sqcap S$ and $\sqcup S$ exist for all $S \in P$ then P is a complete lattice
- All finite lattices are complete

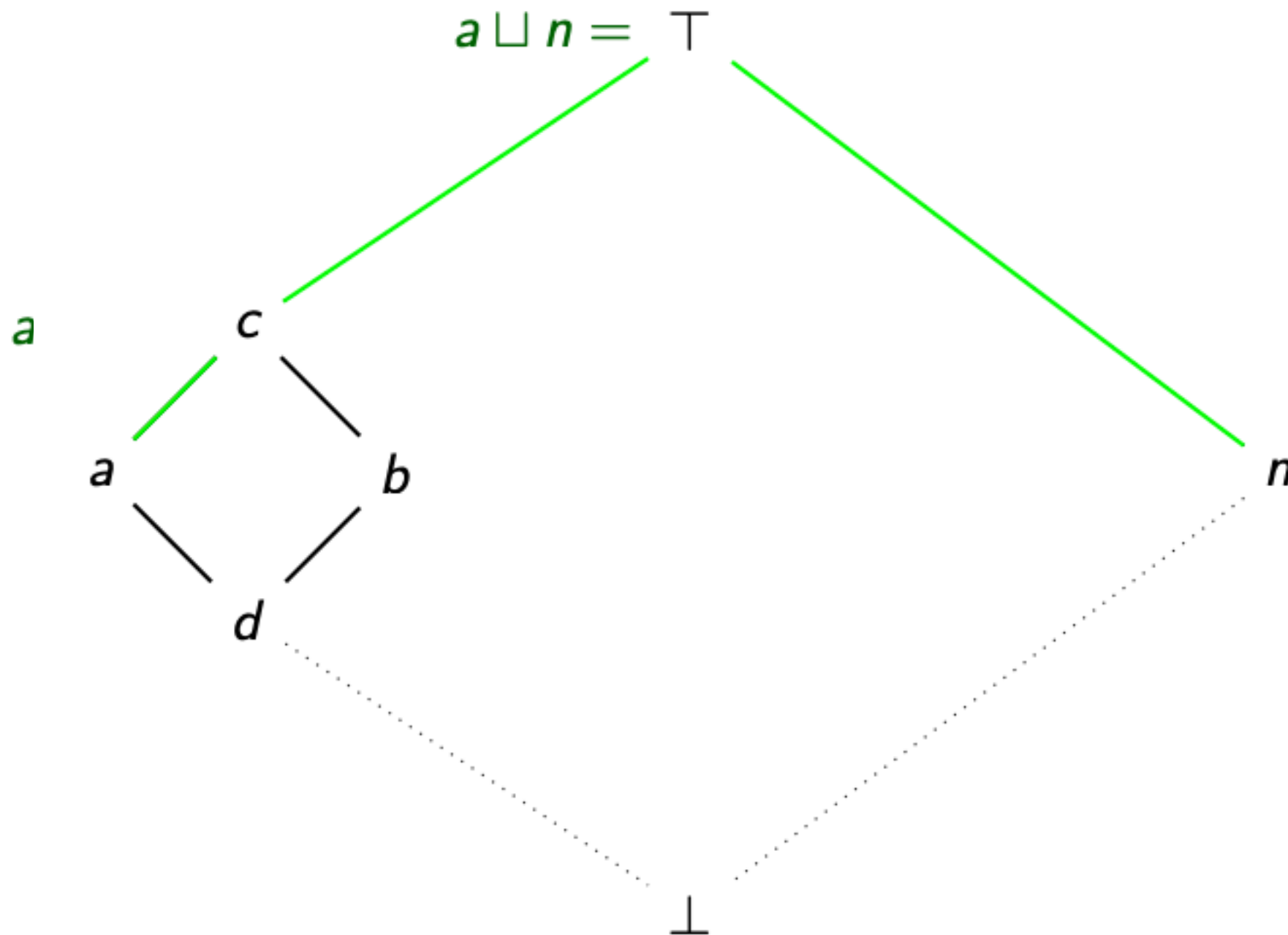
Complete Lattices



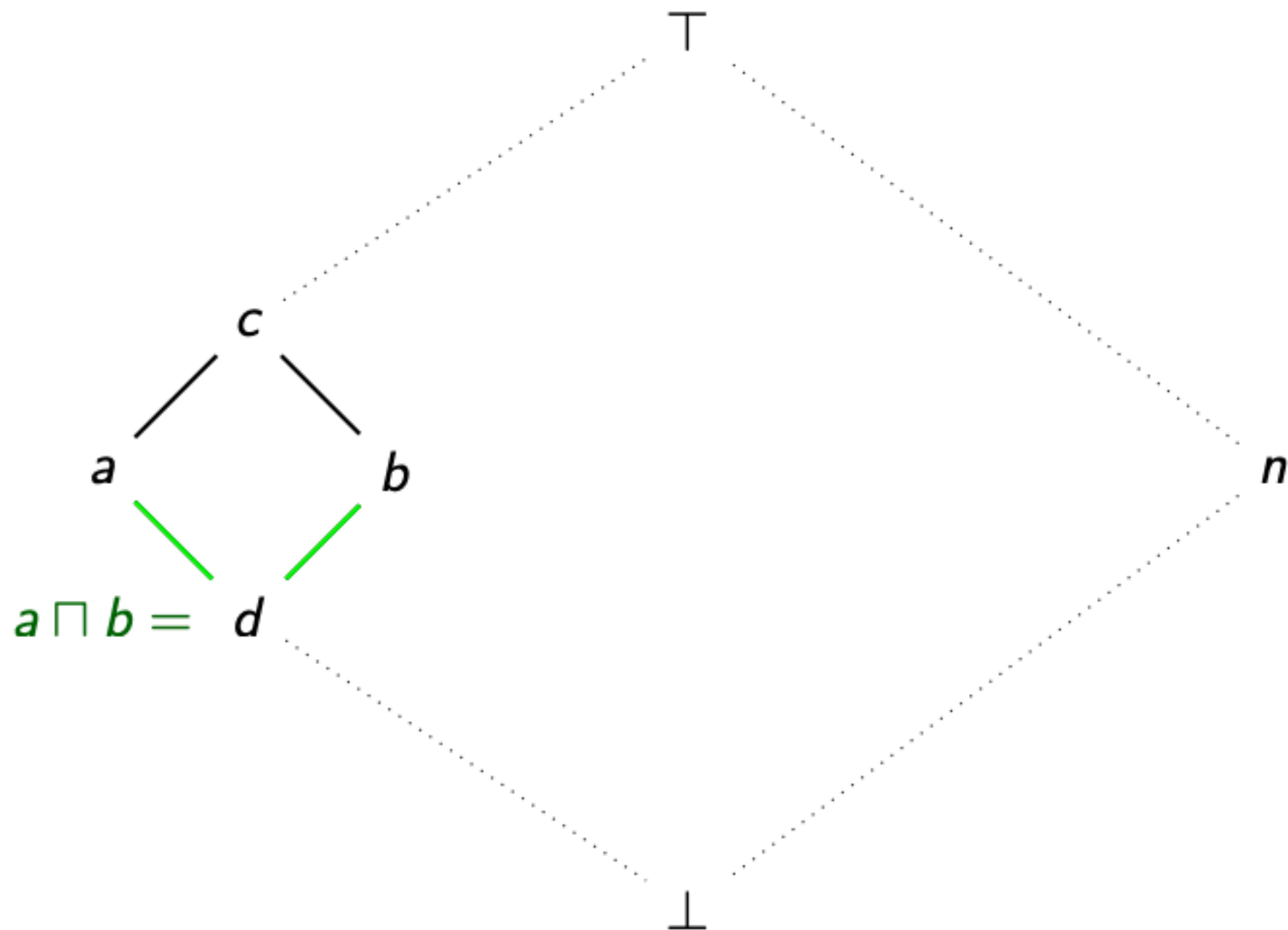
Complete Lattices



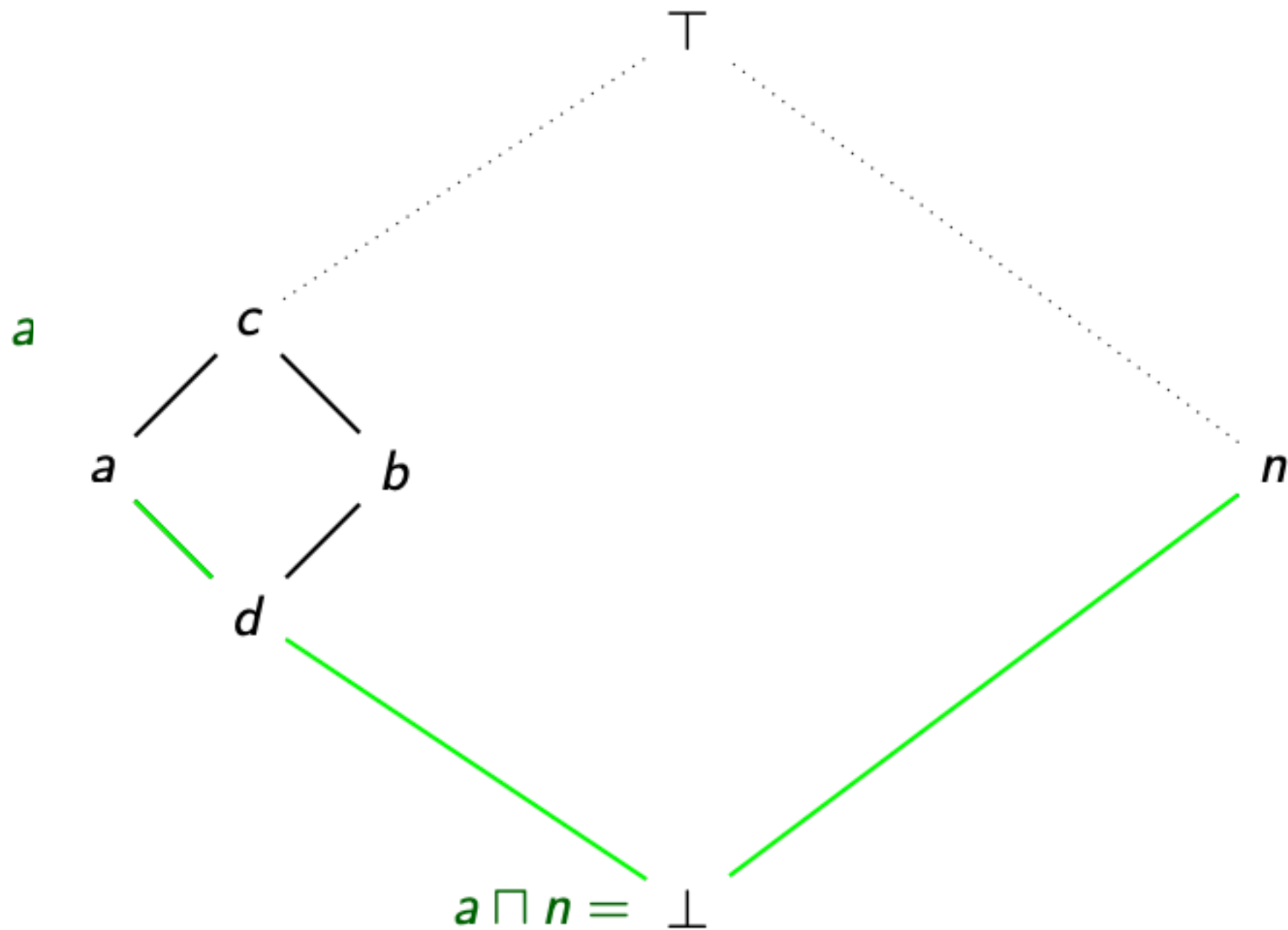
Complete Lattices



Complete Lattices



Complete Lattices

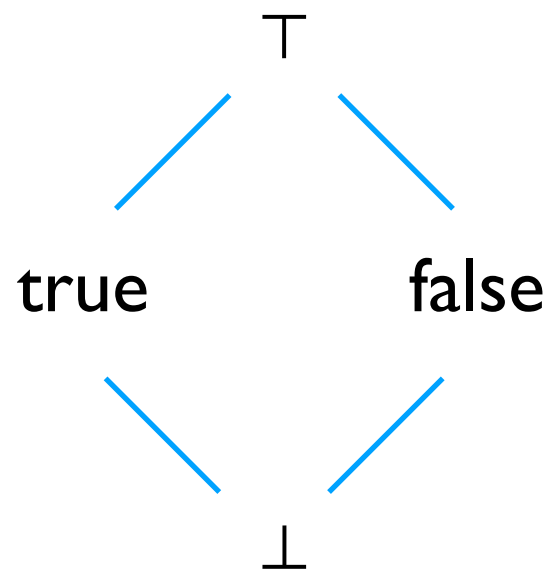


What about *Functions* and Lattices?

- For a lattice L , one can define arbitrary functions:
 $f: L \rightarrow L$
- If for all $x, y \in L$ $x \leq y$ implies $f(x) \leq f(y)$ the function is **monotonic**
- If for all $x, y \in L$ $f(x \sqcap y) = f(x) \sqcap f(y)$ the function is **distributive**
- A **fixpoint** of f is a $z \in L$ such that $f(z) = z$

Typical Lattices in Dataflow Analysis

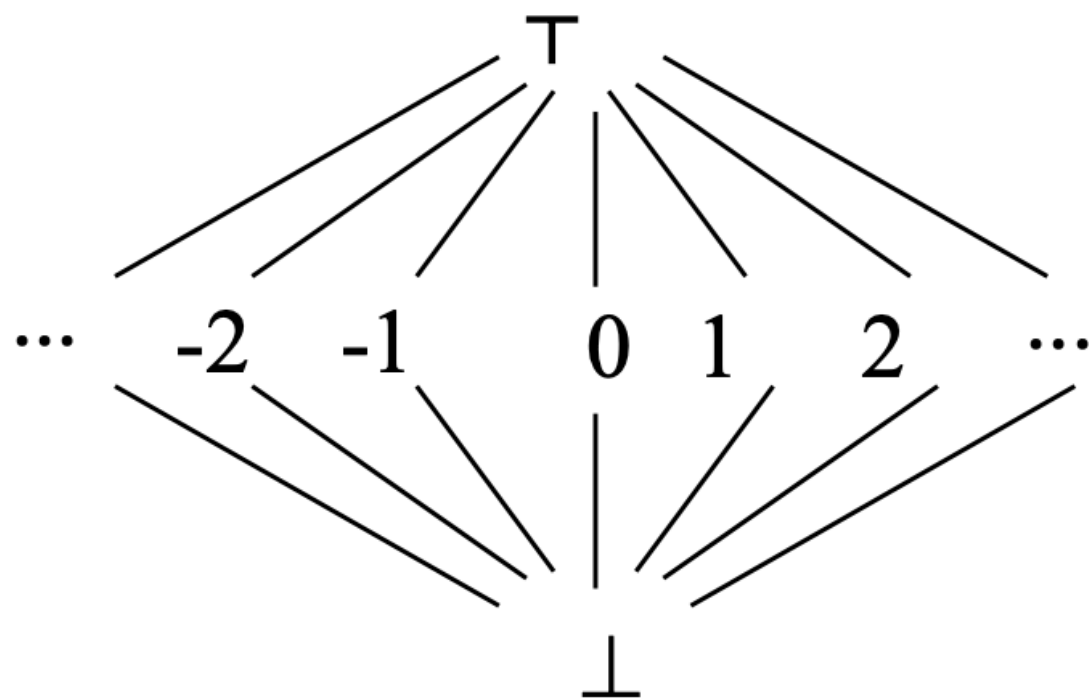
- **Two-point lattice: top and bottom**
 - Represents a particular boolean property



- Interpretation for data flow e.g.:
- T = true-or-false
- $\perp$ = unknown
- $a \sqcup b$: either a or b
- $a \sqcap b$: both a and b

Typical Lattices in Dataflow Analysis

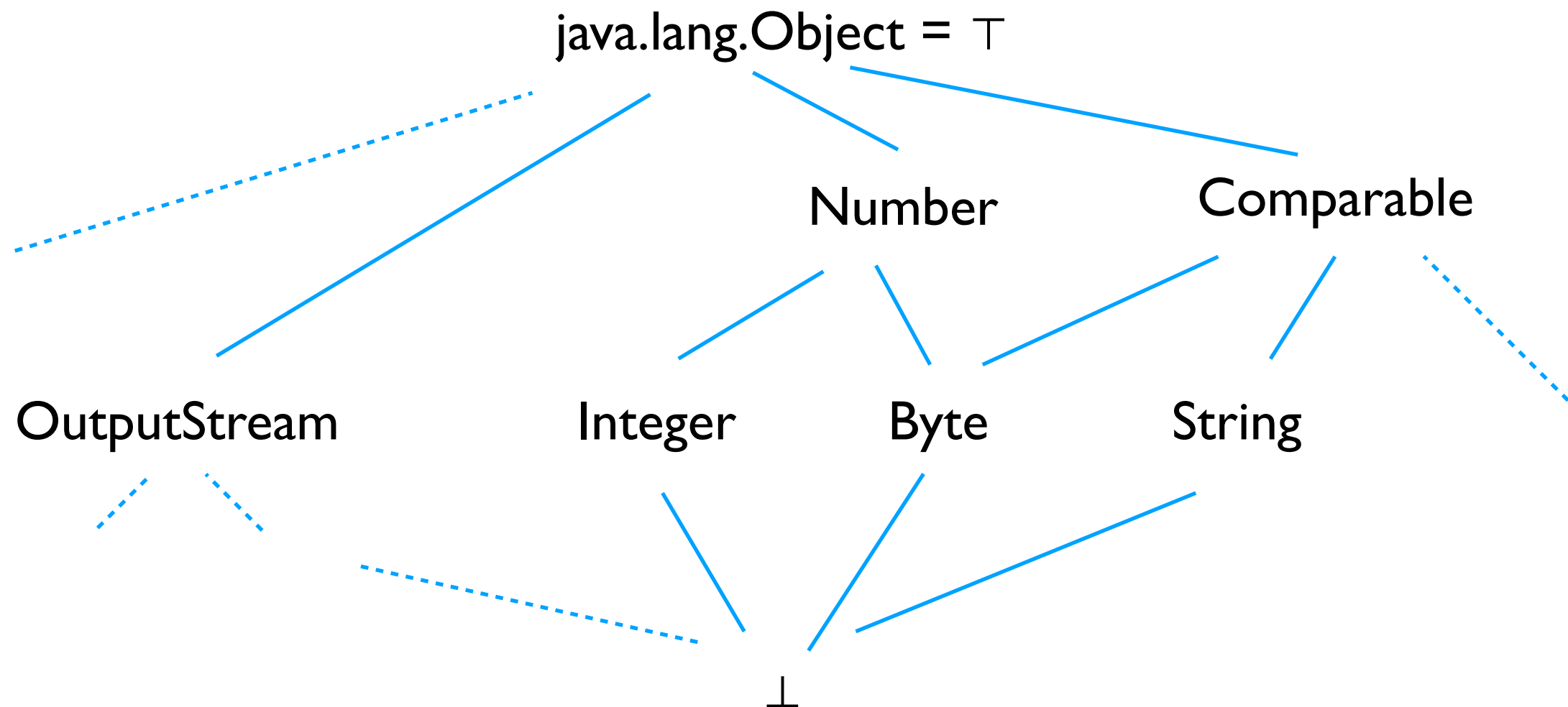
- Set of unordered values plus top and bottom (a.k.a., a flat lattice)
 - Example: Reaching constants domain for a particular variable
 - Height = 2 (but *width* may be infinite)



- $\top = \mathbb{Z}$
- $\perp = \emptyset$
- $a \sqcup b: a$ iff $a = b$, $\top$ otherwise
- $a \sqcap b: a$ iff $a = b$, $\perp$ otherwise

Typical Lattices in Dataflow Analysis

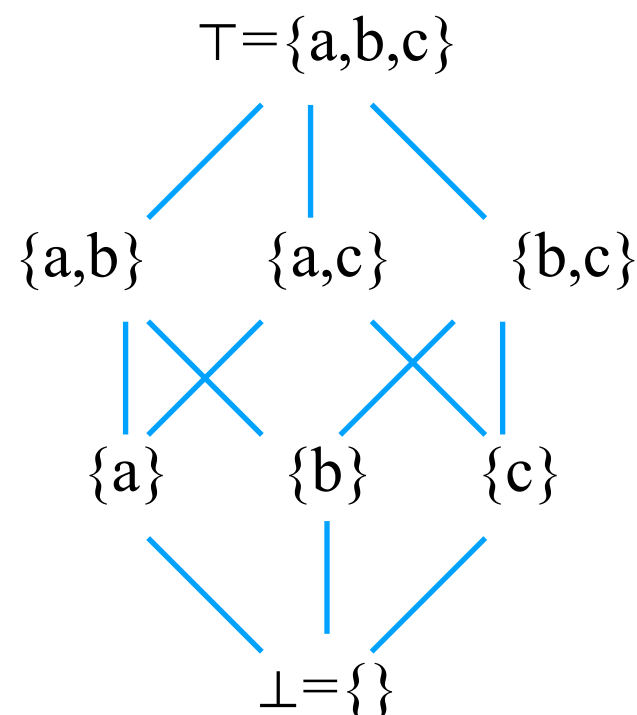
- Type hierarchy lattice



- constructs most precise supertype
- constructs intersection types:
`java.lang.Comparable` $\sqcap$ `java.io.Serializable`

Typical Lattices in Dataflow Analysis

- **Powerset lattice:** set of all subsets of a set U
 - Meet/join is union ($\cup$) or intersection ($\cap$)
 - Partial ordering ($\leq$) is $\supseteq$ or $\subseteq$
 - Bottom ($\perp$) and Top ($\top$) are U and $\emptyset$, or vice versa
 - Height = $|U|$ (infinite if U is infinite)



Typical Lattices in Dataflow Analysis

- **Product Lattices:**

- Assume (complete) lattices:

- $L_1 = \langle P_1, \sqsubseteq_1, \sqcap_1, \sqcup_1, \top_1, \perp_1 \rangle$

- $L_2 = \langle P_2, \sqsubseteq_2, \sqcap_2, \sqcup_2, \top_2, \perp_2 \rangle$

- Let $L_1 \times L_2 = \langle P_1 \times P_2, \sqsubseteq, \sqcap, \sqcup, \top, \perp \rangle$ where:

- $\langle a, b \rangle \sqsubseteq \langle a', b' \rangle$ iff $a \sqsubseteq_1 a'$ and $b \sqsubseteq_2 b'$

- $\langle a, b \rangle \sqcap \langle a', b' \rangle = \langle a \sqcap_1 a', b \sqcap_2 b' \rangle$

- $\langle a, b \rangle \sqcup \langle a', b' \rangle = \langle a \sqcup_1 a', b \sqcup_2 b' \rangle$

- $\top = \langle \top_1, \top_2 \rangle$

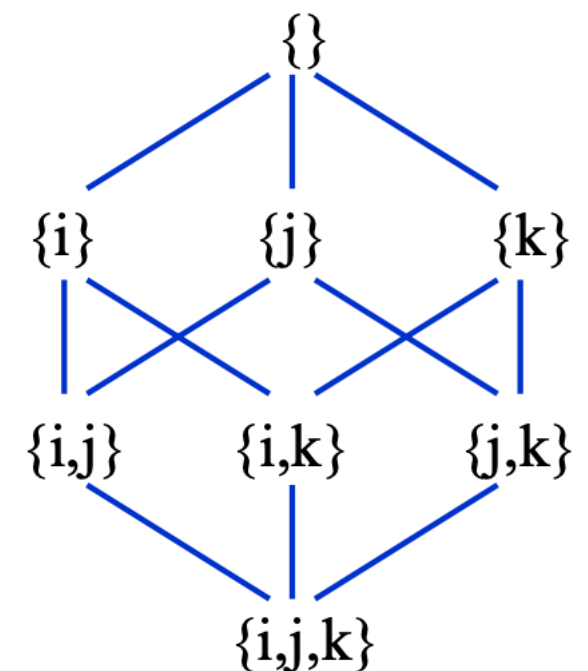
- $\perp = \langle \perp_1, \perp_2 \rangle$

Data-Flow Analysis via Lattices

- Elements of the lattice (P) represent flow values (in[] and out[] sets)
 - e.g., Sets of live variables for liveness
- $\perp$ represents “best-case” information (initial flow value)
 - e.g., Empty set
- $\top$ represents “worst-case” information
 - e.g., Universal set
- $\sqcup$ (join) merges flow values
 - e.g., Set union
- If $x \leq y$, then y is a conservative approximation of x
 - e.g., Superset

Data-Flow Analysis via Lattices: Alternative View

- Elements of the lattice (P) represent flow facts at each program point (in[] and out[] sets)
 - e.g., Sets of live variables for liveness
- $\top$ represents “best-case” information (initial flow value)
 - e.g., Empty set
- $\perp$ represents “worst-case” information
 - e.g., Universal set
- $\sqcap$ (meet) merges flow values
 - e.g., Set union
- If $x \leq y$, then x is a conservative approximation of y
 - e.g., Superset



Monotone Framework

Monotone Framework

Abstract Domain

$\text{join}_b(x_1, \dots, x_n)$

“Unknown” start value

“Could be anything” end value

Lattice

$L = \langle P, \sqsubseteq, \sqcap, \sqcup \rangle$

$x_1 \sqcup \dots \sqcup x_n$

$\perp$

$\top$

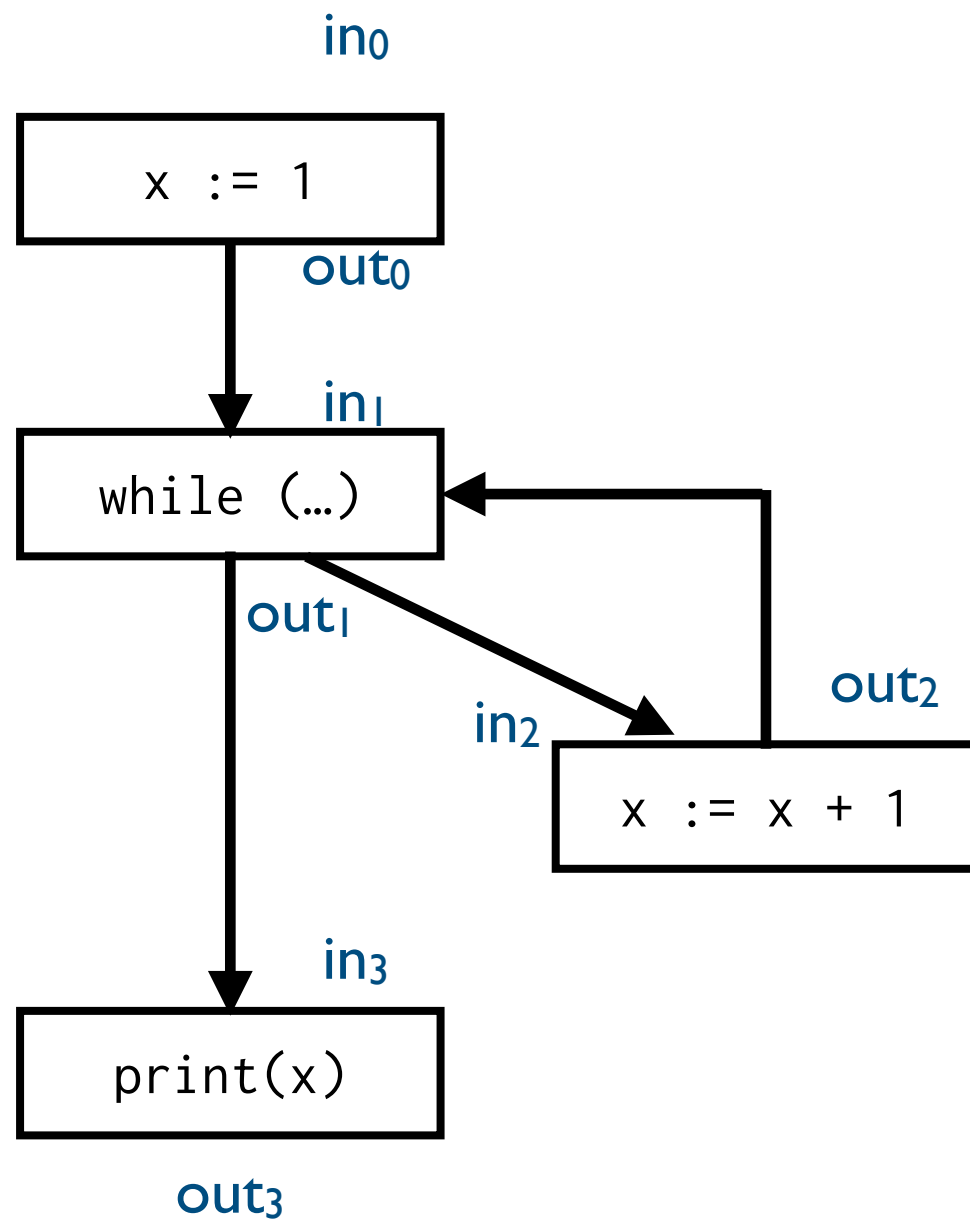
Monotone Frameworks (Killdall '77):

Lattice L of finite height

Monotone trans_b

$\Rightarrow$ Data flow analysis with Soundness and Termination

Monotone Framework



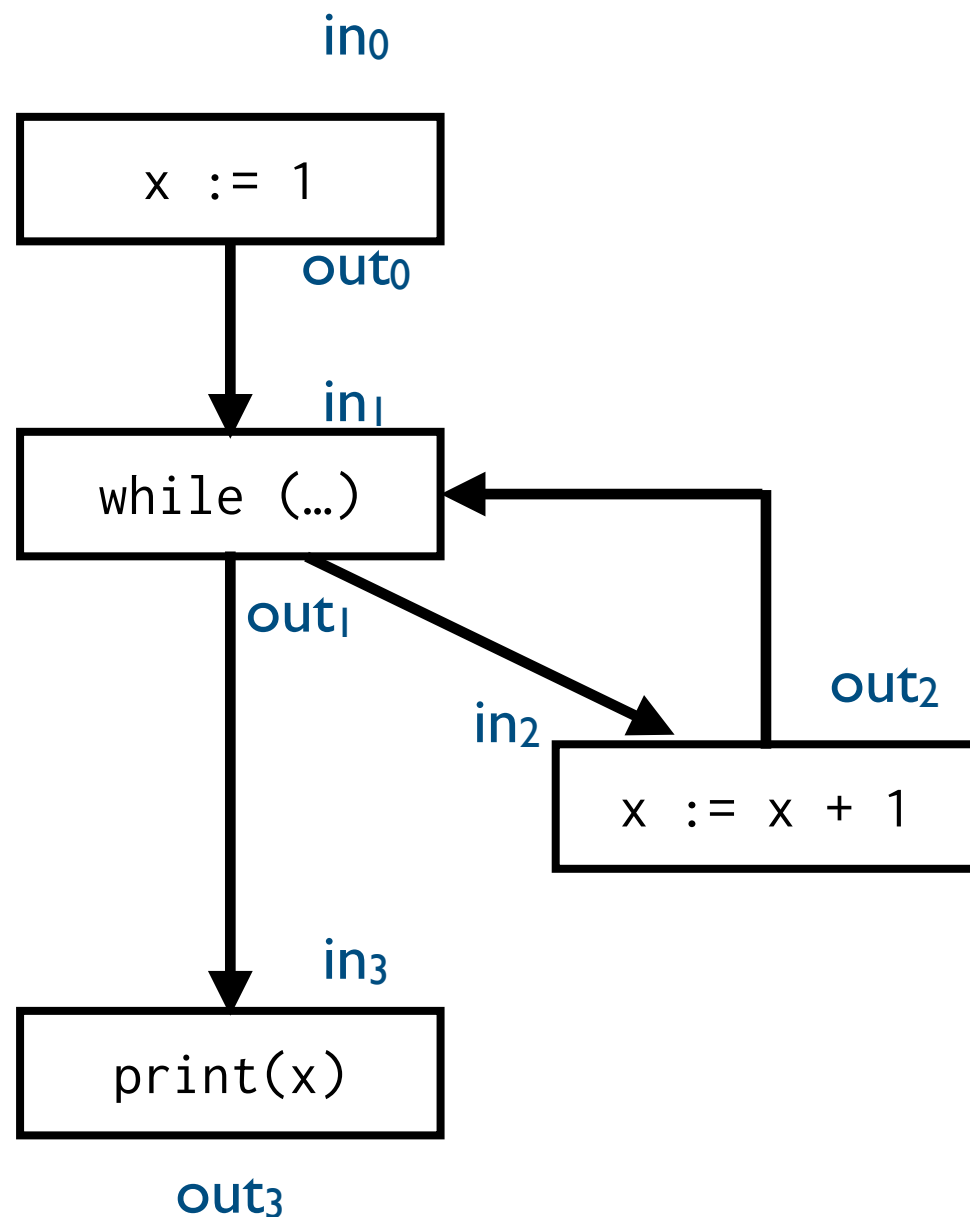
$$\text{out}_0 = \text{trans}_0(\perp)$$

$$\text{out}_1 = \text{trans}_1(\text{out}_0 \sqcup \text{out}_2)$$

$$\text{out}_2 = \text{trans}_2(\text{out}_1)$$

$$\text{out}_3 = \text{trans}_3(\text{out}_2)$$

Monotone Framework



$$\text{out}_0 = \text{trans}_0(\perp)$$

$$\text{out}_1 = \text{trans}_1(\text{out}_0 \sqcup \text{out}_2)$$

$$\text{out}_2 = \text{trans}_2(\text{out}_1)$$

$$\text{out}_3 = \text{trans}_3(\text{out}_2)$$

Lattices $\text{out}_0 : L_0, \dots, \text{out}_3 : L_3$

Can build:

$$L_{0..3} = L_0 \times L_1 \times L_2 \times L_3$$

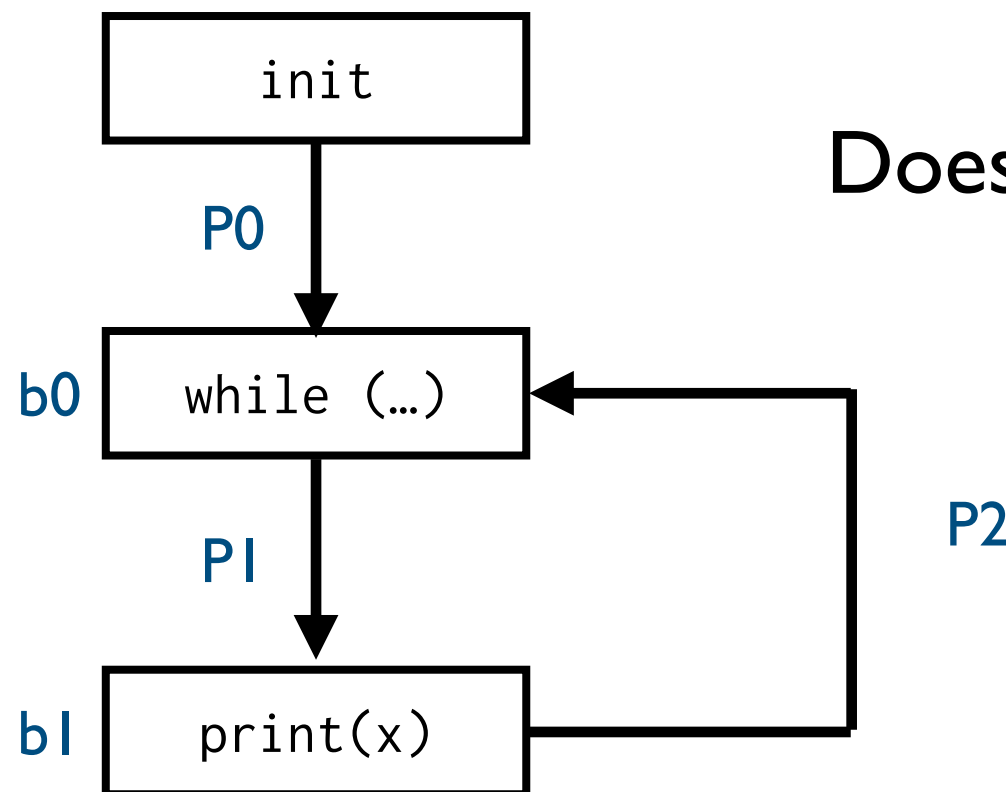
$$\perp_{0..3} = \langle \perp_0 \times \perp_1 \times \perp_2 \times \perp_3 \rangle$$

$$\text{trans}_{0..3} = \langle \text{trans}_0(\perp), \text{trans}_1(\text{out}_0 \sqcup \text{out}_2), \text{trans}_2(\text{out}_1), \text{trans}_3(\text{out}_2) \rangle$$

Monotone Framework

- Program P:
 - Program Lattice L_P
 - $\perp_P$
 - trans_P : Compute one step of naive analysis
- Repeat trans_P :
 - $\text{fp}_\perp = \text{trans}_P^n(\perp_P)$
- $\text{fp}_\perp$ is fixpoint of trans_P
 - $\text{fp}_\perp = \text{trans}_P(\text{fp}_\perp)$

Termination



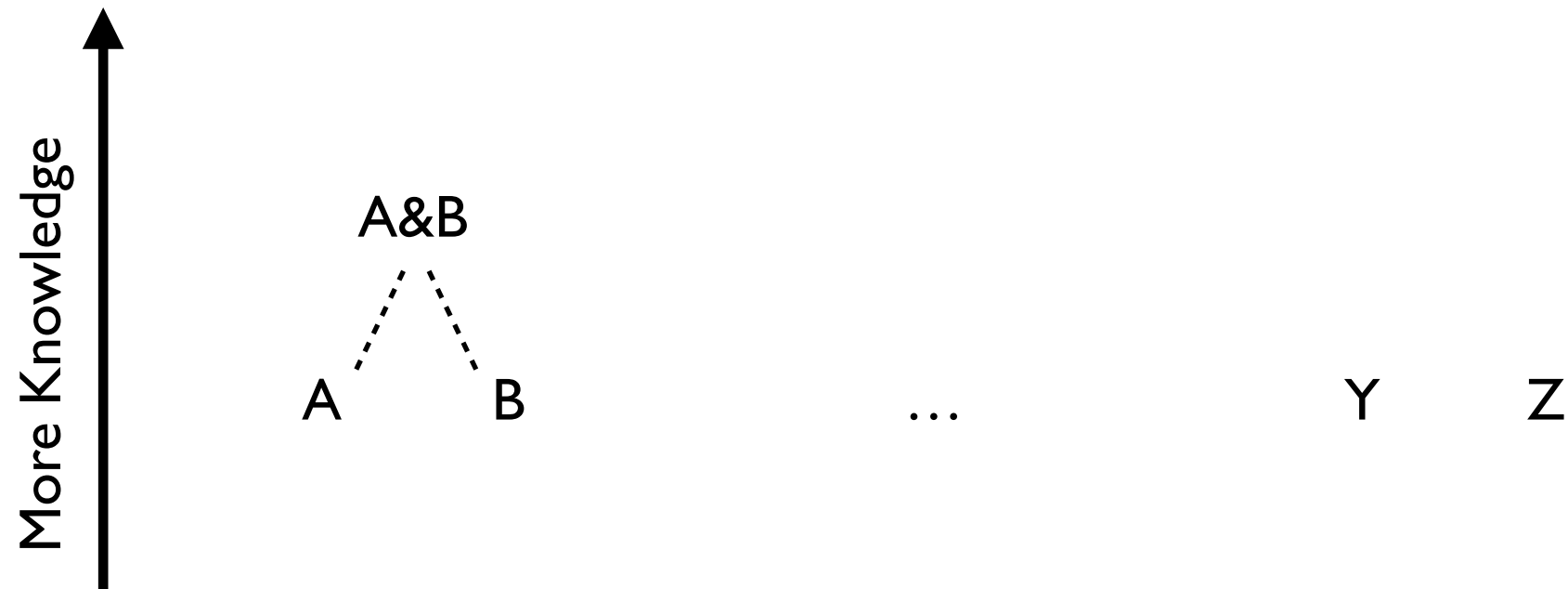
Does the following ever stop changing:

$$\text{in}_{b0} = \text{join}_{b0}(P0, P2)$$

- Intuition: we keep accumulating knowledge
- Growth limit: bound amount of knowledge we collect
- Make sure join_b , trans_b never throw knowledge away
- Eventually, either nothing changes or we hit growth limit

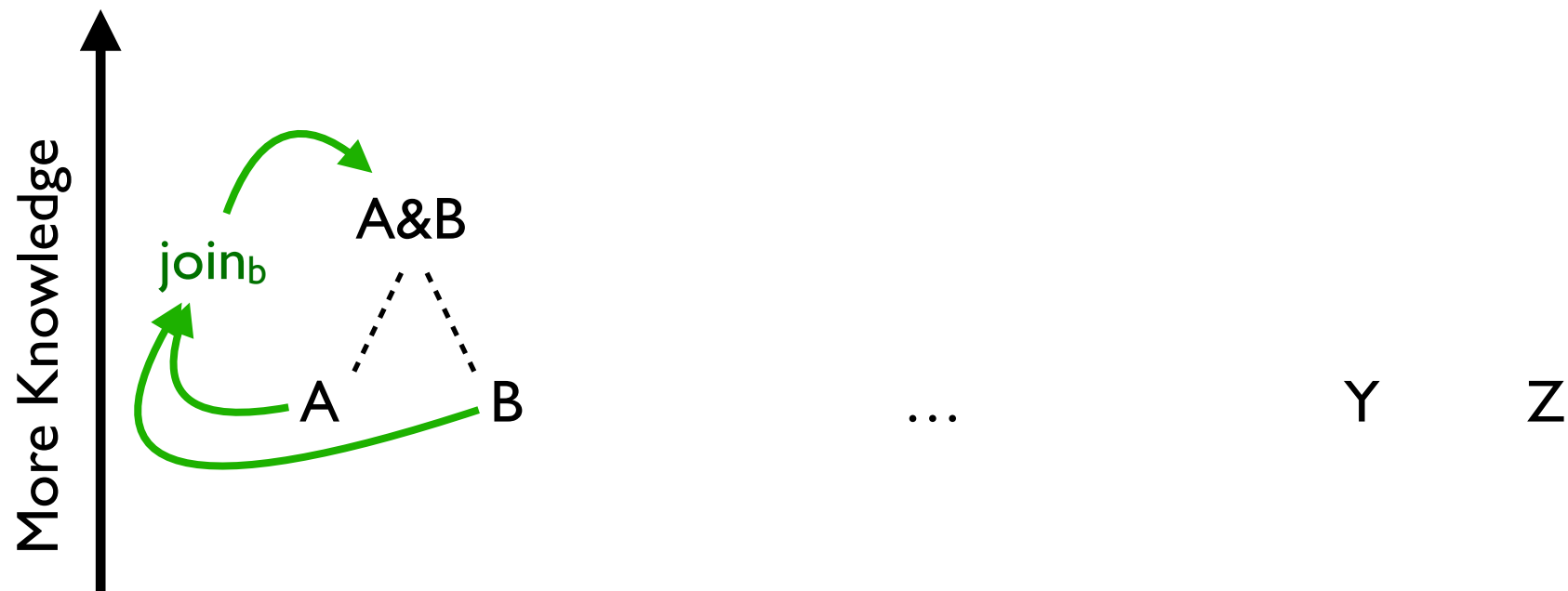
Termination

Structure of L:



Termination

Structure of L:



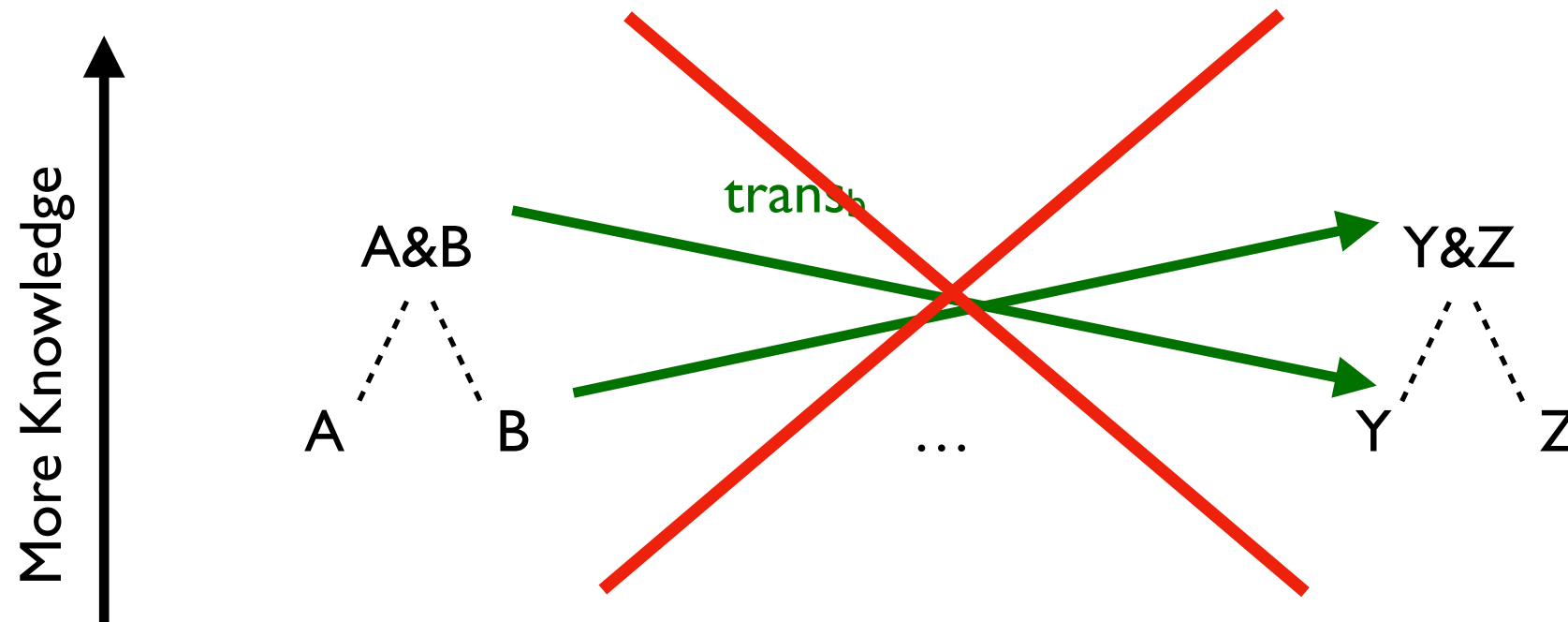
join_b must not lose knowledge:

$$\text{join}_b(A, B) \sqsupseteq A$$

$$\text{join}_b(A, B) \sqsupseteq B$$

Termination

Structure of L:



join_b must not lose knowledge:

$$\text{join}_b(A, B) \sqsupseteq A$$

$$\text{join}_b(A, B) \sqsupseteq B$$

trans_b must be monotonic over amount of knowledge:

$$x \sqsubseteq y \rightarrow \text{trans}_b(x) \sqsubseteq \text{trans}_b(y)$$

Introduce bound: $\top$ means 'too much information'

Termination

out only grows

$\text{trans}_b(\sqcup \text{out}_b(s'))$ must be monotonic

A function f on a partial order is monotonic if

$$x \leq y \Rightarrow f(x) \leq f(y)$$

out starts out as $\perp$

→ So $\perp$ must be $\leq \text{trans}_b(\sqcup \text{out}_b(s'))$ after first step

Assume **out_b** grows for all predecessors s' of s

→ Then $\sqcup \text{out}_b(s')$ grows

trans_b has to be monotonic

→ $\text{Gen}(s) \cup (\text{in}_b(s) - \text{Kill}(s))$

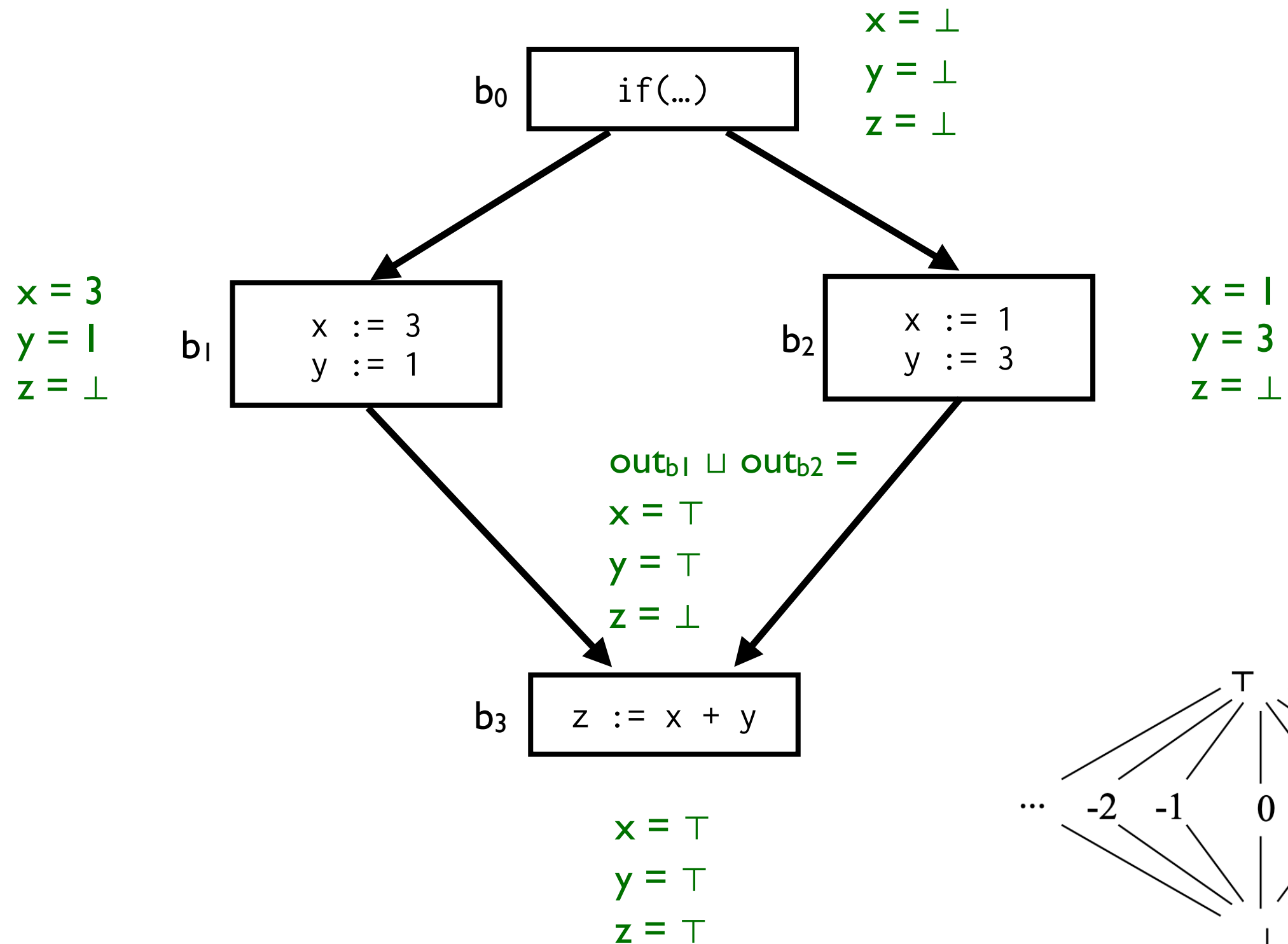
Minimal Fixpoint Algorithm

- So far: naïve algorithm for computing fixpoint
 - Produces a fixpoint
 - Keeps iterating all trans_b / join_b functions, even if nothing changed
- Optimise processing with worklist
 - Set-like datastructure:
 - add element (if not already present)
 - contains test: is element present?
 - pop element: remove and return
 - Tracks what's left to be done $\Rightarrow$ “MFP” (Minimal Fixed Point) Algorithm

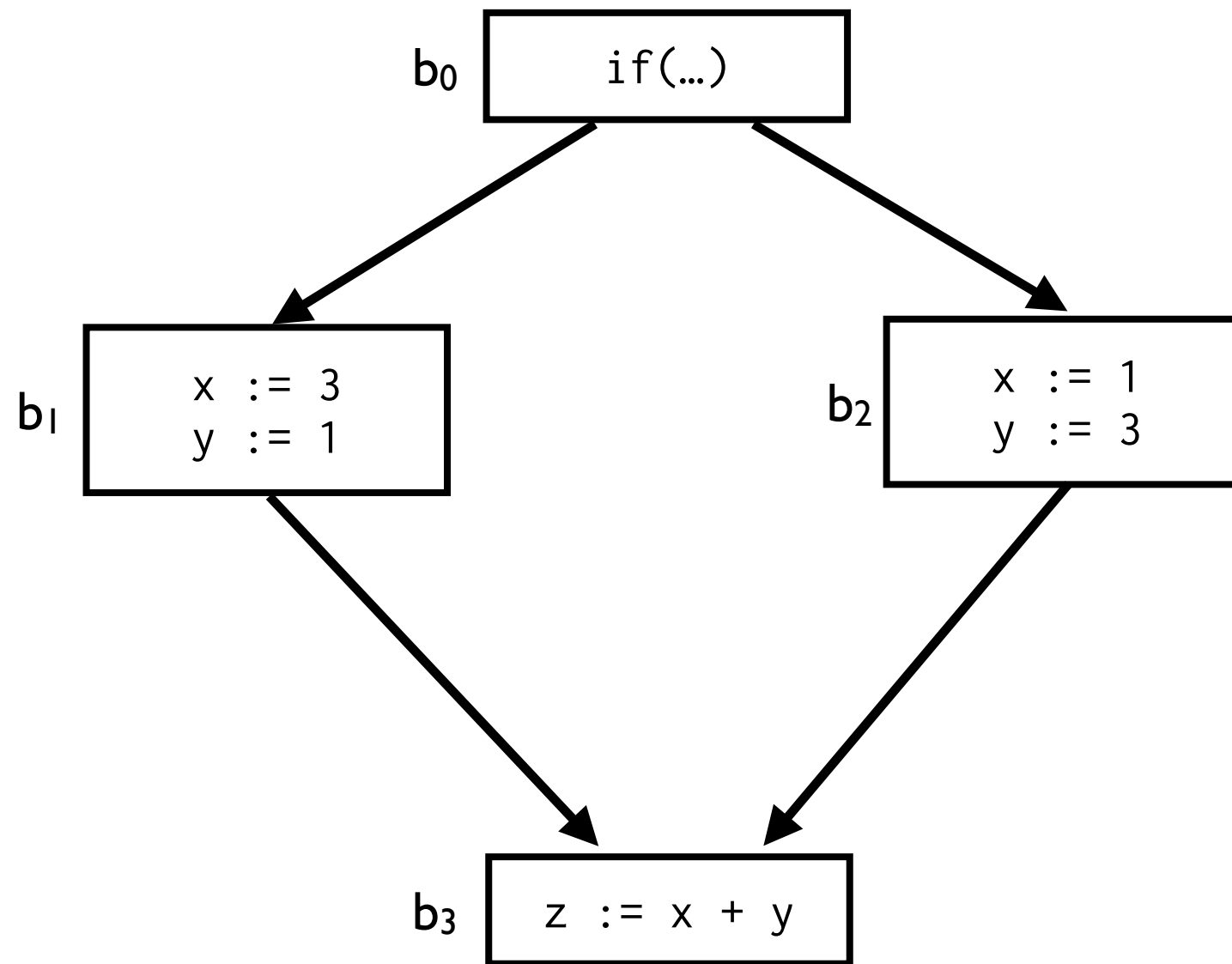
Minimal Fixpoint Algorithm (MFP)

```
Procedure MFP( $\perp$ ,  $\sqcup$ ,  $\Xi$ , CFG, trans, is-backward)
begin
  if is-backward then reverse edges(CFG);
  worklist := edges(CFG);
  foreach  $n \in \text{nodes}(\text{CFG})$  do
    analysis[n] =  $\perp$ ;
  done
  while not empty(worklist) do
     $\langle n, n' \rangle := \text{pop}(\text{worklist})$ ;
    if  $\text{trans}_n(\text{analysis}[n]) \not\sqsubseteq \text{analysis}[n']$  then begin
       $\text{analysis}[n'] := \text{analysis}[n'] \sqcup \text{trans}_n(\text{analysis}[n])$ 
      foreach  $n'' \in \text{successor-nodes}(\text{CFG}, n')$  do
        push(worklist,  $\langle n', n'' \rangle$ );
      done
    end
  done
  return analysis;
end
```

Minimal Fixpoint Algorithm (MFP)



Execution Paths



$\text{path}_{b_0} = \{ [] \}$
 $\text{path}_{b_1} = \{ [b_0] \}$
 $\text{path}_{b_2} = \{ [b_0] \}$
 $\text{path}_{b_3} = \{ [b_0, b_1], [b_0, b_2] \}$

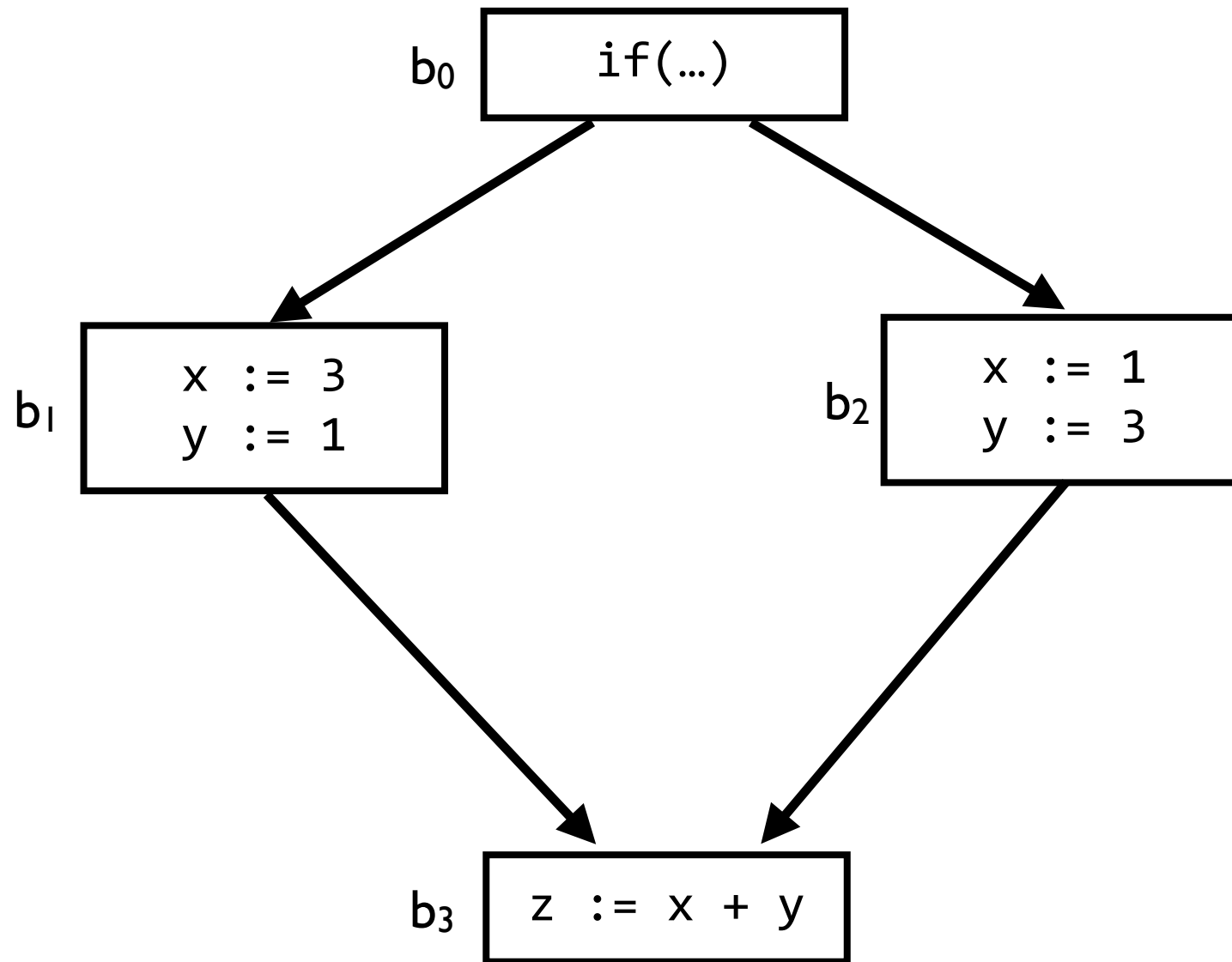
Meet Over All Paths (MOP)

Compute the MOP solution:

- Iterate over all paths $[p_0, \dots, p_k]$ in path_{b_i}
- Compute precise result for each path
- Join $\sqcup$ with all other precise results

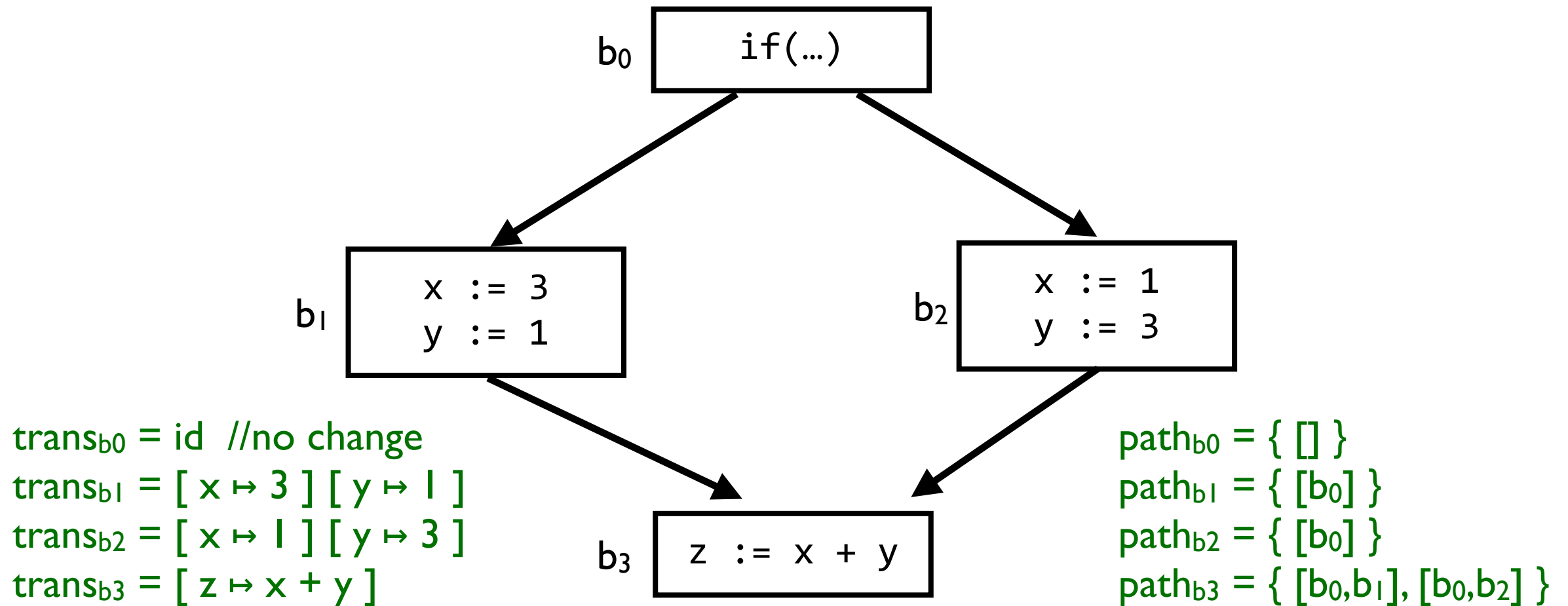
$$\text{out}_{b_i} = \bigsqcup_{[p_0, \dots, p_k] \in \text{path}_{b_i}} \text{trans}_{p_0} \circ \dots \circ \text{trans}_{p_k} \circ \text{trans}_{b_i} (\perp)$$

Execution Paths



$\text{path}_{b_0} = \{ [] \}$
 $\text{path}_{b_1} = \{ [b_0] \}$
 $\text{path}_{b_2} = \{ [b_0] \}$
 $\text{path}_{b_3} = \{ [b_0, b_1], [b_0, b_2] \}$

Meet Over All Paths (MOP)



$$\begin{aligned}
 \text{out}_{b_3} &= [z \mapsto x + y][x \mapsto 3][y \mapsto 1](\perp) \sqcup [z \mapsto x + y][x \mapsto 1][y \mapsto 3](\perp) \\
 &= \{ z \mapsto 3 + 1, x \mapsto 3, y \mapsto 1 \} \sqcup \{ z \mapsto 1 + 3, x \mapsto 1, y \mapsto 3 \} \\
 &= \{ z \mapsto 4, x \mapsto 3, y \mapsto 1 \} \sqcup \{ z \mapsto 4, x \mapsto 1, y \mapsto 3 \} \\
 &= \{ z \mapsto 4, x \mapsto \top, y \mapsto \top \}
 \end{aligned}$$

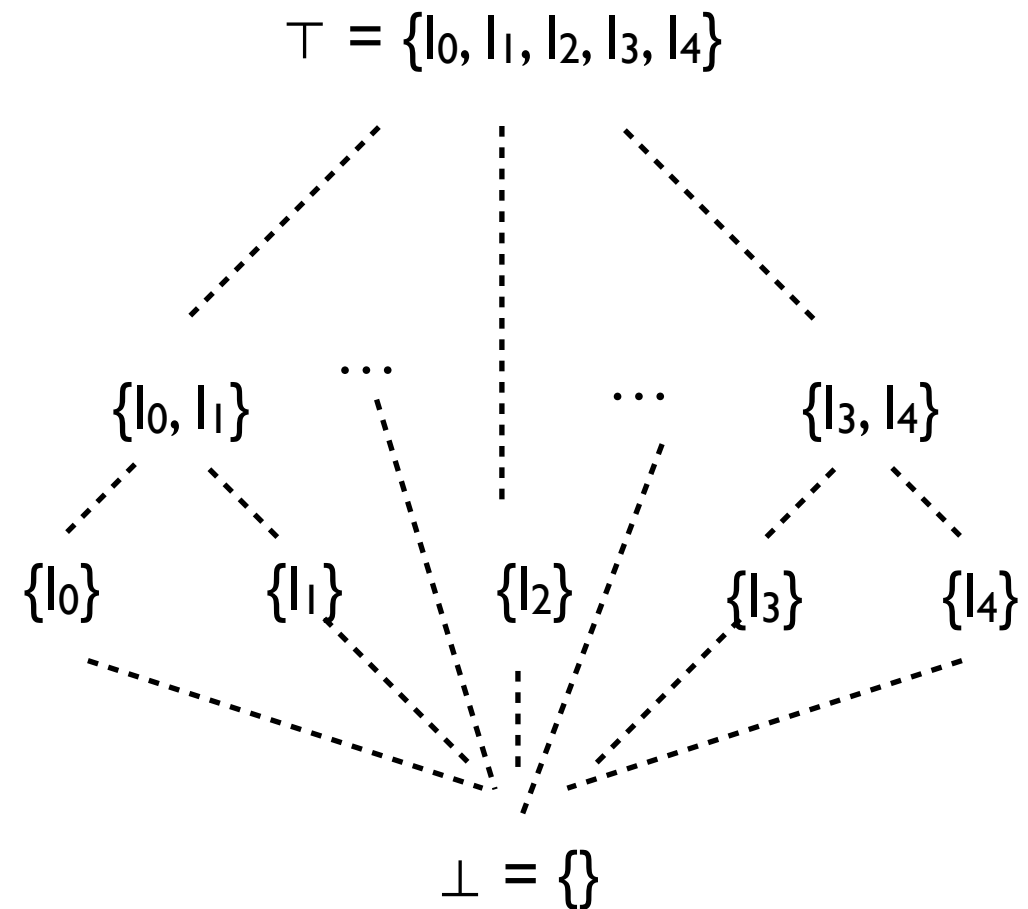
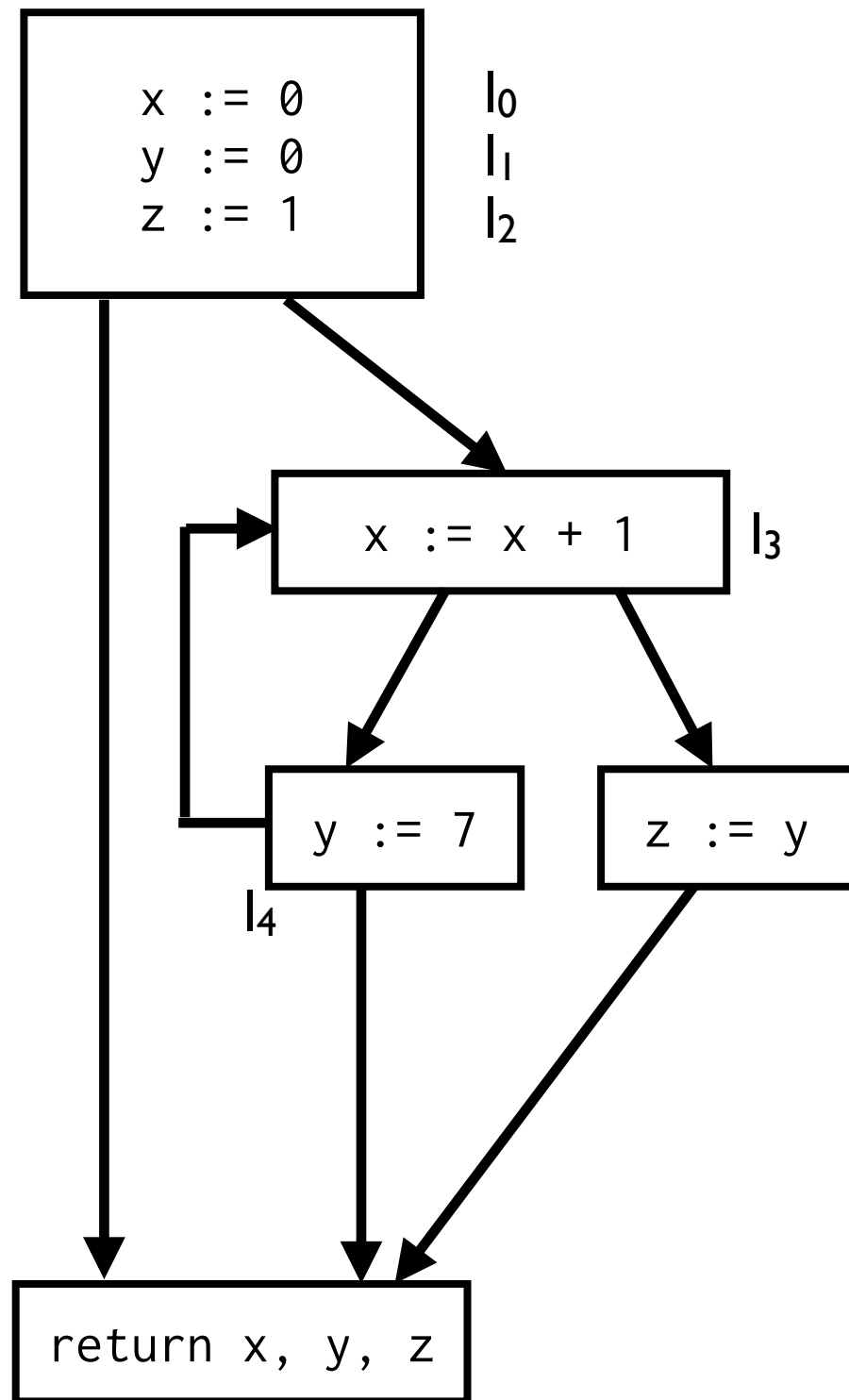
Accuracy of Data Flow Analysis

- Ideally, we would like to compute the **best solution**: the **meet over all paths (MOP)** solution.
 - MOP combines the results from all the possible execution paths, but is undecidable
- We can "settle" down to a more "empirically found" solution, the **Minimum Fixpoint Algorithm (MFP)** solution
 - MFP is still sound, but not necessarily as precise as MOP
- If the data flow problem is **distributive**, then MFP and MOP solutions are the same

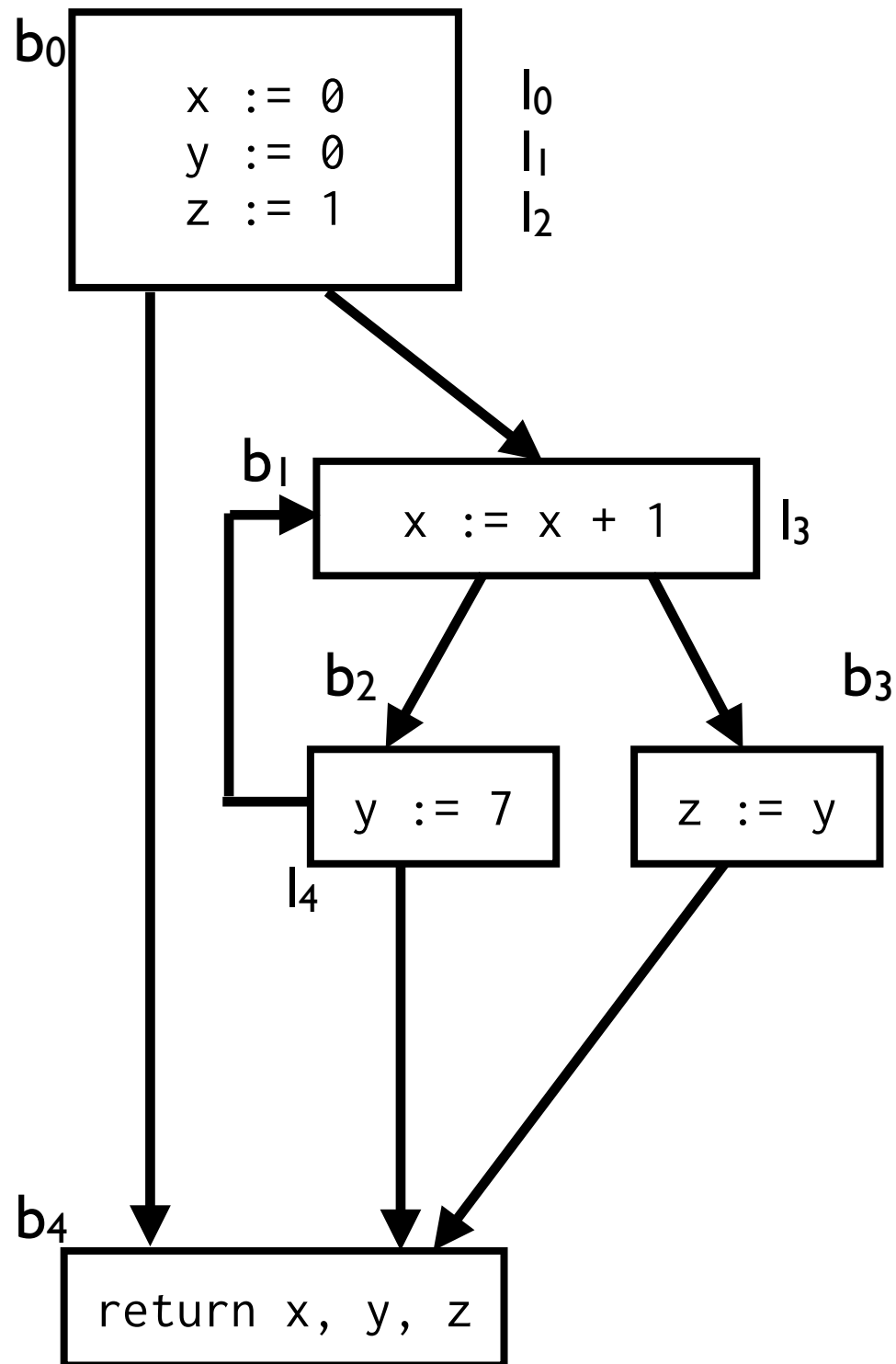
MOP vs MFP

- Both solutions are sound
- MOP is more precise
- MOP is undecidable
- MFP is easier to implement

MOP vs MFP



MOP vs MFP



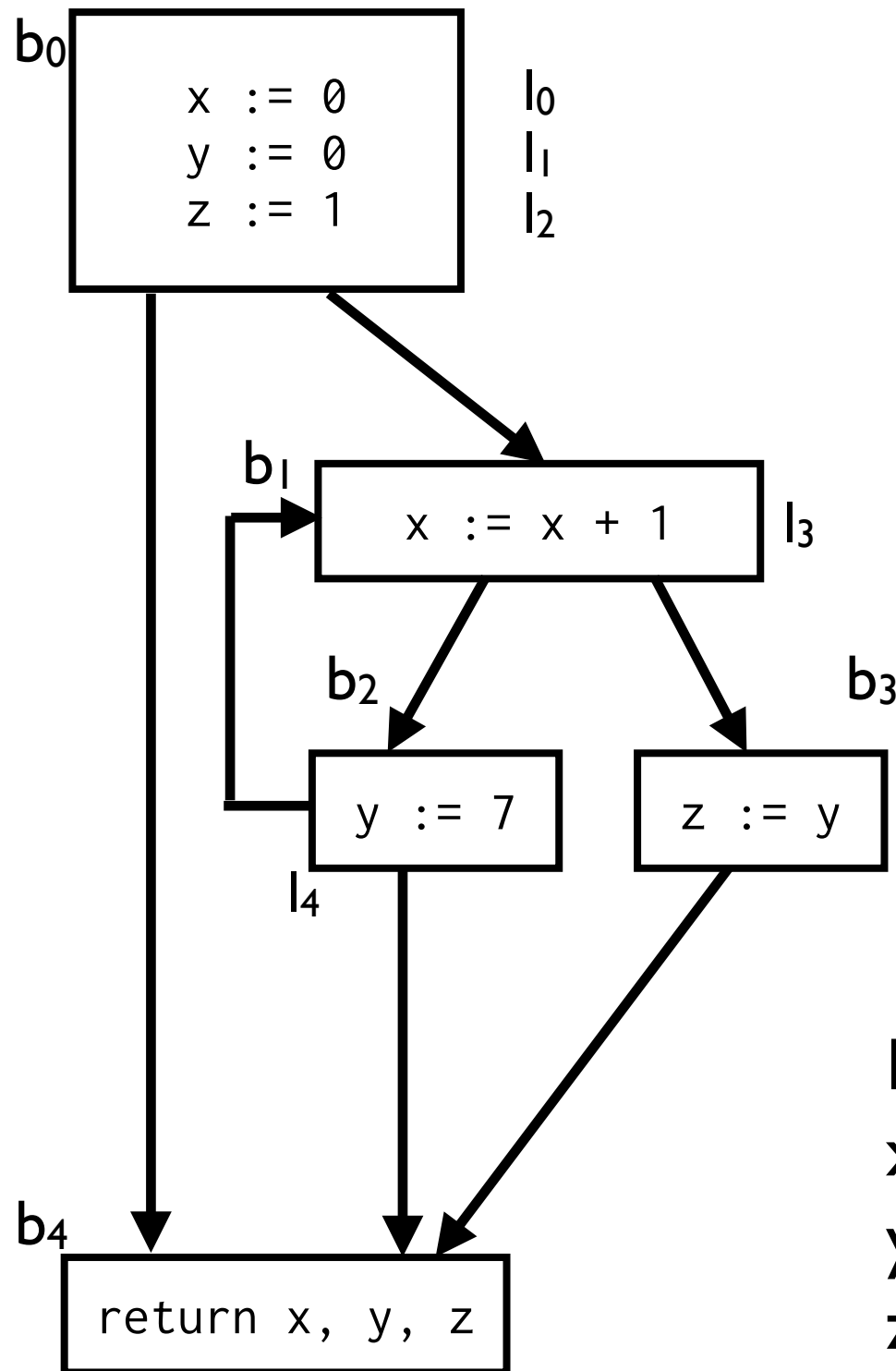
$$\text{trans}_{b0} = [x \mapsto \{l_0\}] [y \mapsto \{l_1\}] [z \mapsto \{l_2\}]$$

$$\text{trans}_{b1} = [x \mapsto \{l_3\}]$$

$$\text{trans}_{b2} = [y \mapsto \{l_4\}]$$

$$\text{trans}_{b3} = [z \mapsto y]$$

MOP vs MFP



$$\text{trans}_{b0} = [x \mapsto \{l_0\}] [y \mapsto \{l_1\}] [z \mapsto \{l_2\}]$$

$$\text{trans}_{b1} = [x \mapsto \{l_3\}]$$

$$\text{trans}_{b2} = [y \mapsto \{l_4\}]$$

$$\text{trans}_{b3} = [z \mapsto y]$$

MOP solution for b4:

$$\begin{aligned} x &\mapsto \{l_0, l_3\} \\ y &\mapsto \{l_1, l_4\} \\ z &\mapsto \{l_1, l_2, l_4\} \end{aligned}$$

MFP solution for b4:

$$\begin{aligned} x &\mapsto \{l_0, l_3\} \\ y &\mapsto \{l_1, l_4\} \\ z &\mapsto \{l_1, l_2, l_4\} \end{aligned}$$

Distributive Frameworks

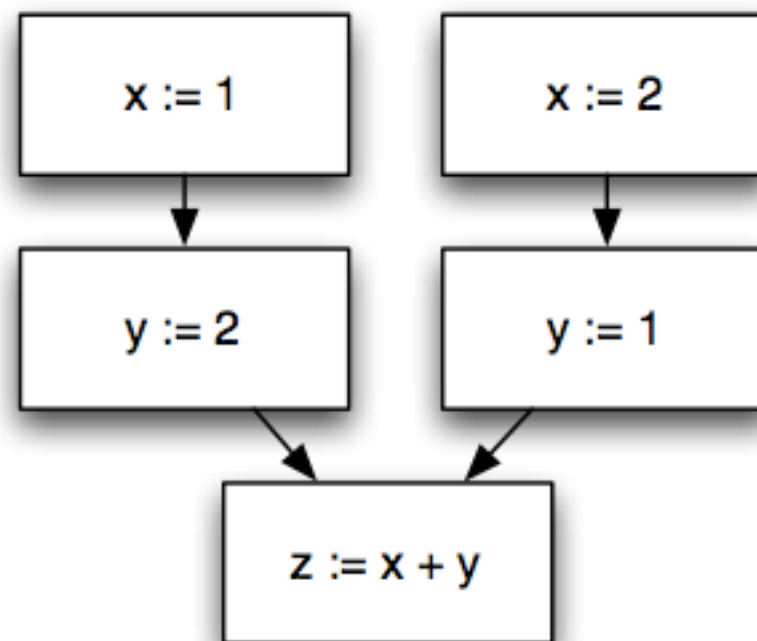
- A monotone framework
 - Guarantees that MFP approximates MOP
- trans_b **distributes** over $\sqcup$:
 - $\text{trans}_b(x \sqcup y) = \text{trans}_b(x) \sqcup \text{trans}_b(y)$
for all programs and all x, y, b
- Gen-Set/Kill-Set transfer functions are **distributive**
 - Guarantees that MFP is equal to MOP

What Problems are Distributive?

- Analyses of *how* the program computes
 - Live variables
 - Available expressions
 - Reaching definitions
 - Very busy expressions

Not all Problems are Distributive!

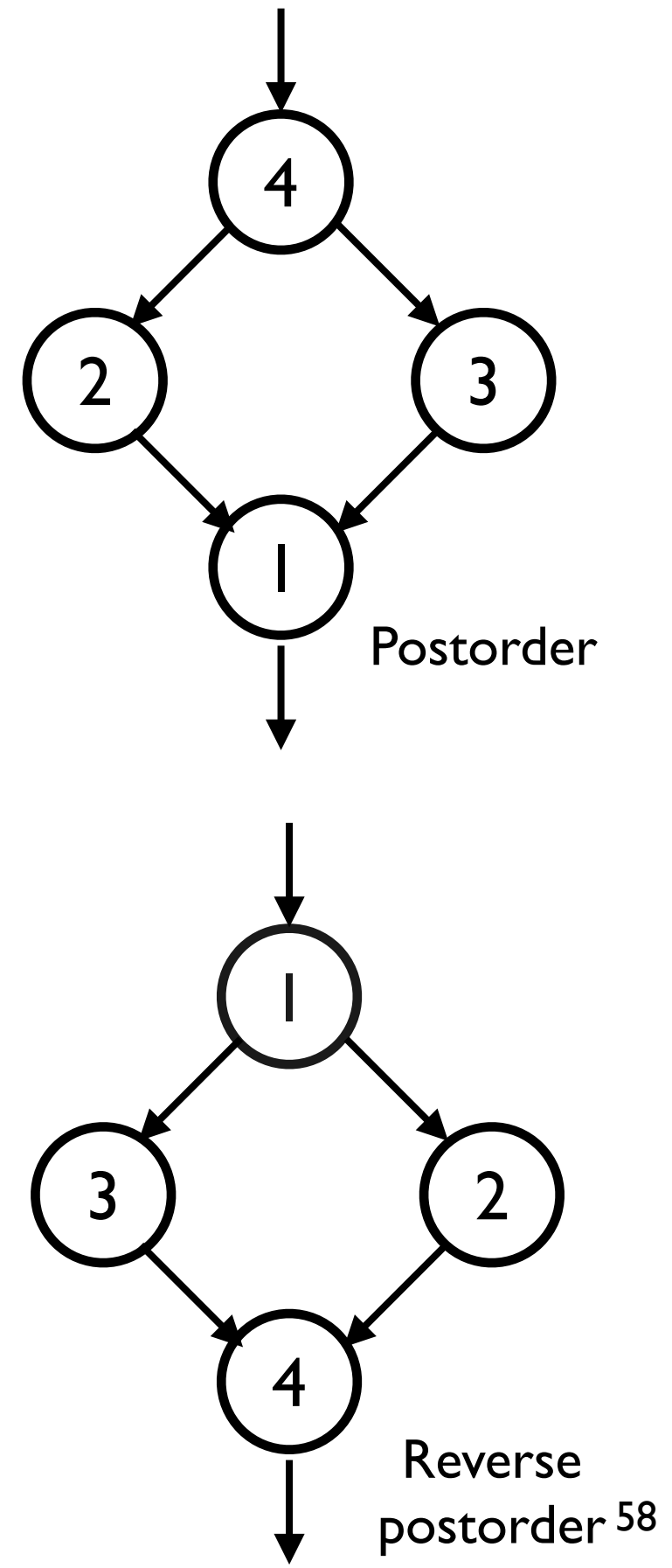
- In general, analysis of *what* the program computes is not distributive



- Example: Constant propagation

Traversal Order

- Facilitate fast convergence to the fixed point
- Postorder traversal
 - Visits as many of a node's successors as possible before visiting the node
 - Used in backward data-flow analysis
- Reverse postorder traversal
 - Visits as many of a node's predecessors as possible before visiting the node
 - Used in forward data-flow analysis

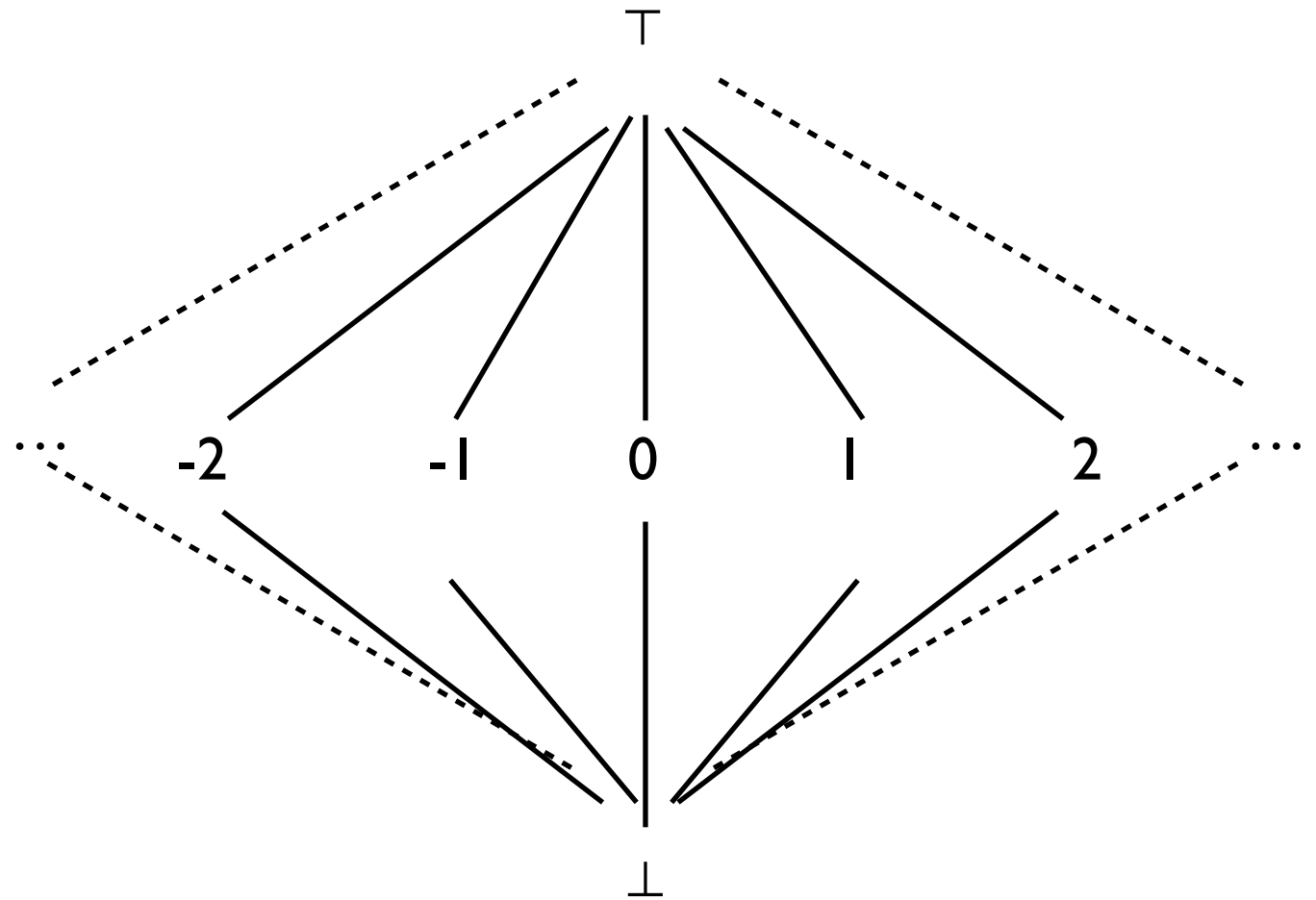


Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis
5. Lattice-Theoretic Framework
6. Abstract Interpretation

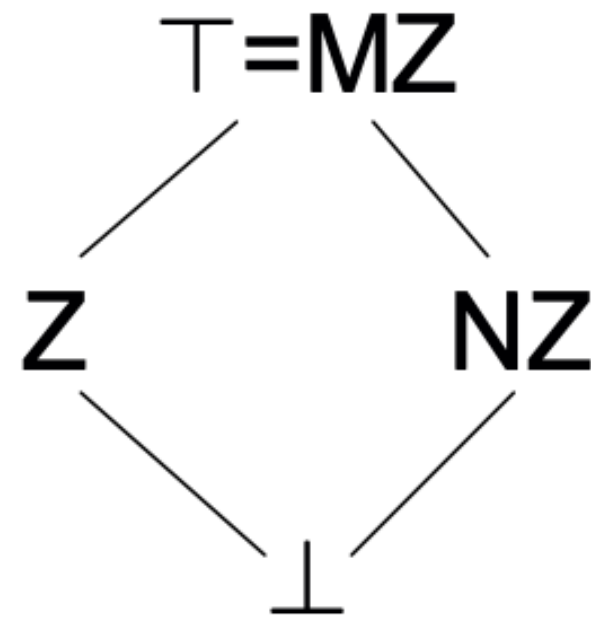
Example: Zero Analysis

```
x := 8;  
y := x;  
z := 0;  
while y > -1 do  
  x := x / y;  
  y := y - 2;  
  z := 5;
```



Example: Zero Analysis

```
x := 8;  
y := x;  
z := 0;  
while y > -1 do  
  x := x / y;  
  y := y - 2;  
  z := 5;
```

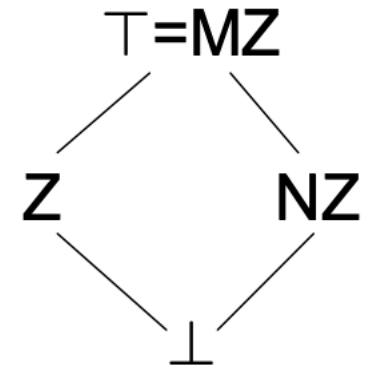


Z (definitive Zero)

NZ (definitive NonZero)

MZ (Maybe Zero)

Example: Zero Analysis



$$\perp \leq Z, \perp \leq NZ, NZ \leq MZ, Z \leq MZ, \perp \leq MZ$$

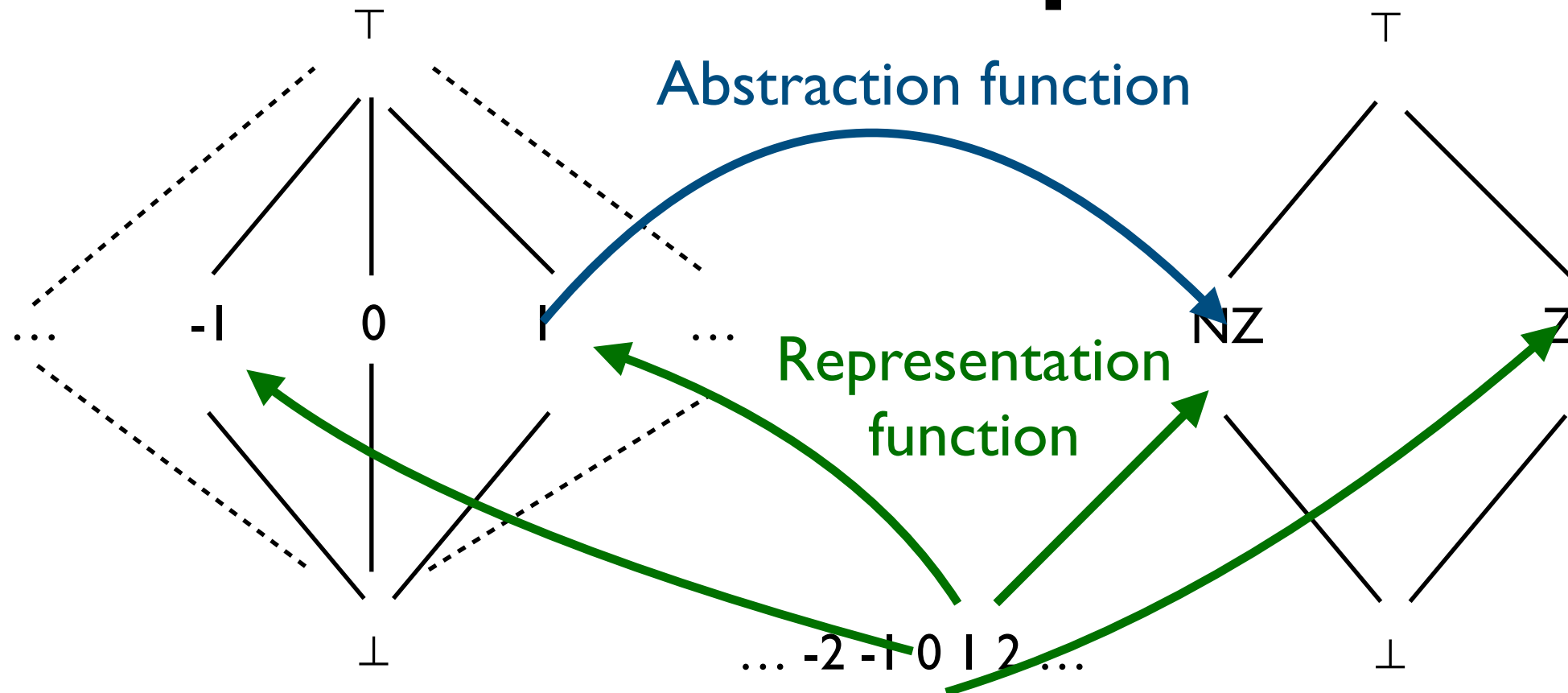
$\sqcup$ defined as **join** for $\leq$:

$x \sqcup y = z$ iff z is the **least upper bound** of x and y

Obeys laws: $\perp \sqcup X = X, \top \sqcup X = \top, X \sqcup X = X$

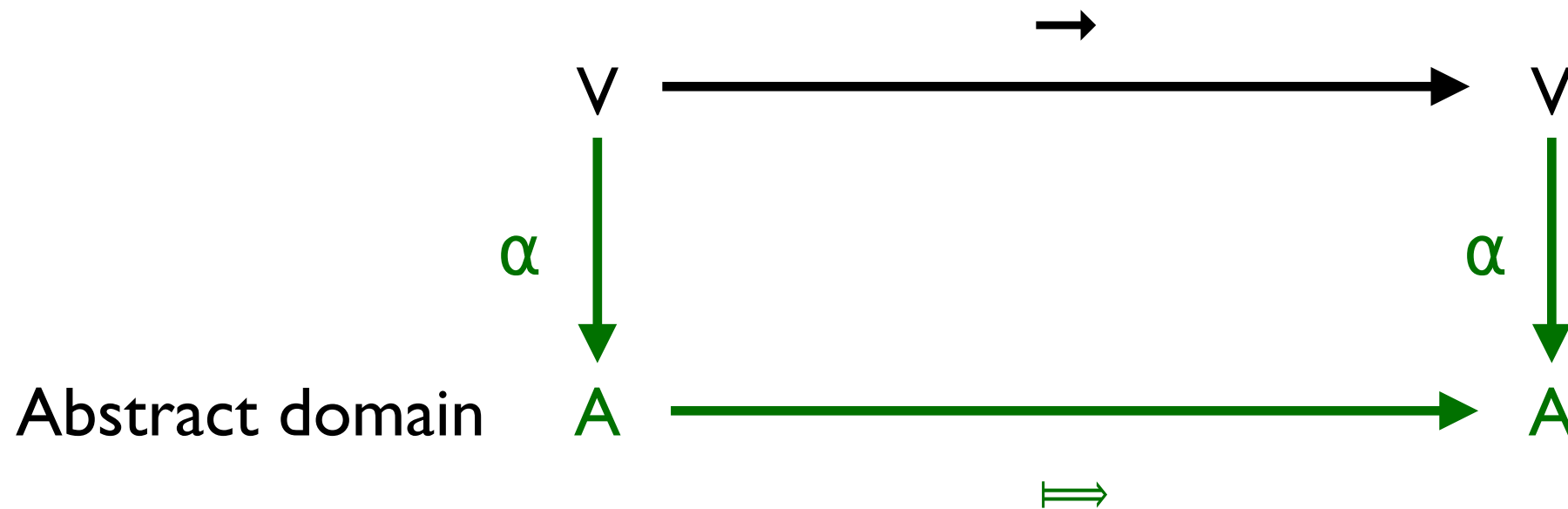
$Z \sqcup NZ = MZ$, and $\top = MZ$

Abstract Interpretation



Semantic abstraction: maps variables to NZ , Z , MZ

Abstract Interpretation



$(\rightarrow): V \rightarrow V$ is operational semantics

$\alpha: V \rightarrow A$ is abstraction function

$\Rightarrow: A \rightarrow A$ is abstraction of $(\rightarrow)$ wrt A

$\Rightarrow$ and $(\rightarrow)$ must agree wrt α

Example: Zero Analysis

Abstract integers

$$\alpha_{ZI}(0) = Z$$

$$\alpha_{ZI}(n) = NZ \text{ for all } n \neq 0$$

Abstract operators

$$\alpha_{ZI}(Z+NZ) = NZ$$

$$\alpha_{ZI}(Z+Z) = Z$$

$$\alpha_{ZI}(Z-Z) = Z$$

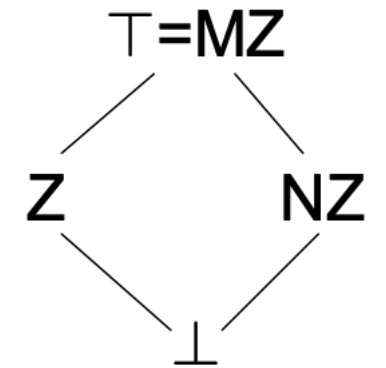
$$\alpha_{ZI}(NZ-NZ) = MZ$$

...

Example: Zero Analysis

Semantic abstraction:

σ maps every variable to non zero (NZ), zero(Z), or maybe zero (MZ)



Abstract evaluation:

+	⊥	Z	NZ	MZ
⊥	⊥	⊥	⊥	⊥
Z	⊥	Z	NZ	MZ
NZ	⊥	NZ	MZ	MZ
MZ	⊥	MZ	MZ	MZ

-	⊥	Z	NZ	MZ
⊥	⊥	⊥	⊥	⊥
Z	⊥	Z	NZ	MZ
NZ	⊥	NZ	MZ	MZ
MZ	⊥	MZ	MZ	MZ

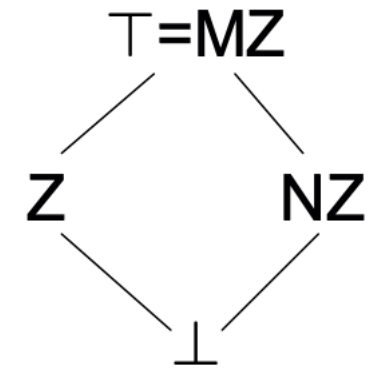
*	⊥	Z	NZ	MZ
⊥	⊥	⊥	⊥	⊥
Z	⊥	Z	Z	Z
NZ	⊥	Z	NZ	MZ
MZ	⊥	Z	MZ	MZ

...same for / etc.

Example: Zero Analysis

Semantic abstraction:

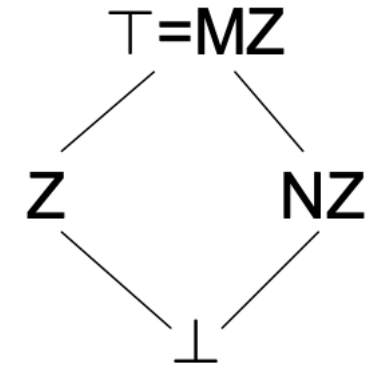
σ maps every variable to non zero (NZ), zero(Z), or maybe zero (MZ)



Abstract evaluation:

/	⊥	Z	NZ	MZ
⊥	⊥	⊥	⊥	⊥
Z	⊥	⊥	Z	⊥
NZ	⊥	⊥	NZ	MZ
MZ	⊥	⊥	MZ	MZ

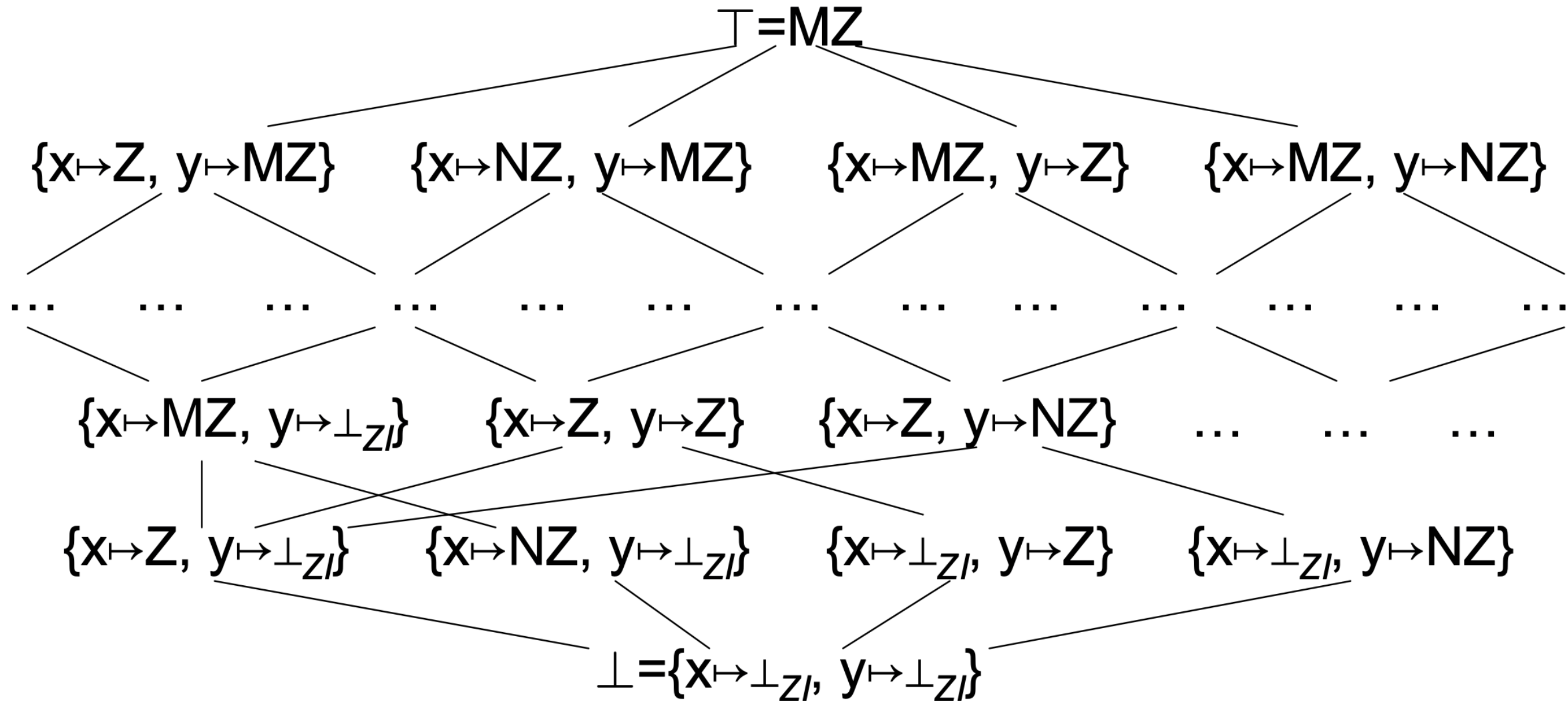
Example: Zero Analysis



Program lattice is a **tuple lattice**

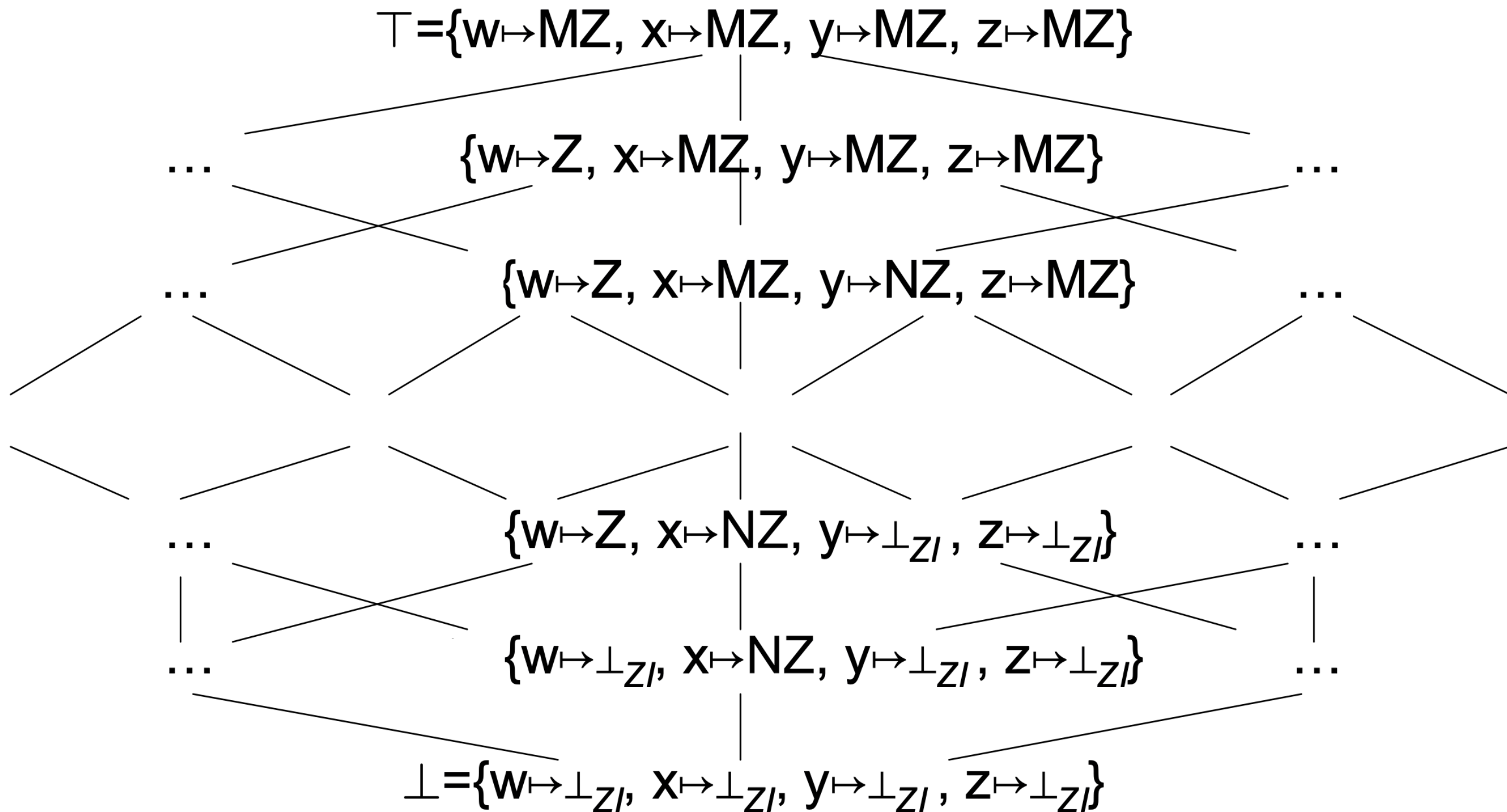
- L_Z is the set of all maps from Var to L_{Z1}
- $\sigma_1 \leq_Z \sigma_2$ iff $\forall x \in Var \mid \sigma_1(x) \leq_{Z1} \sigma_2(x)$
- $\sigma_1 \sqcup_Z \sigma_2 = \{ x \mapsto \sigma_1(x) \sqcup_{Z1} \sigma_2(x) \mid x \in Var \}$
- $\perp = \{ x \mapsto \perp_{Z1} \mid x \in Var \}$
- $T = \{ x \mapsto T_{Z1} \mid x \in Var \} = \{ x \mapsto MZ \mid x \in Var \}$

Example: Zero Analysis



For $\text{Var} = \{x, y\}$

Example: Zero Analysis



Example **Path** in the tuple lattice for $\text{Var} = \{x, y, w, z\}$

Example: Zero Analysis

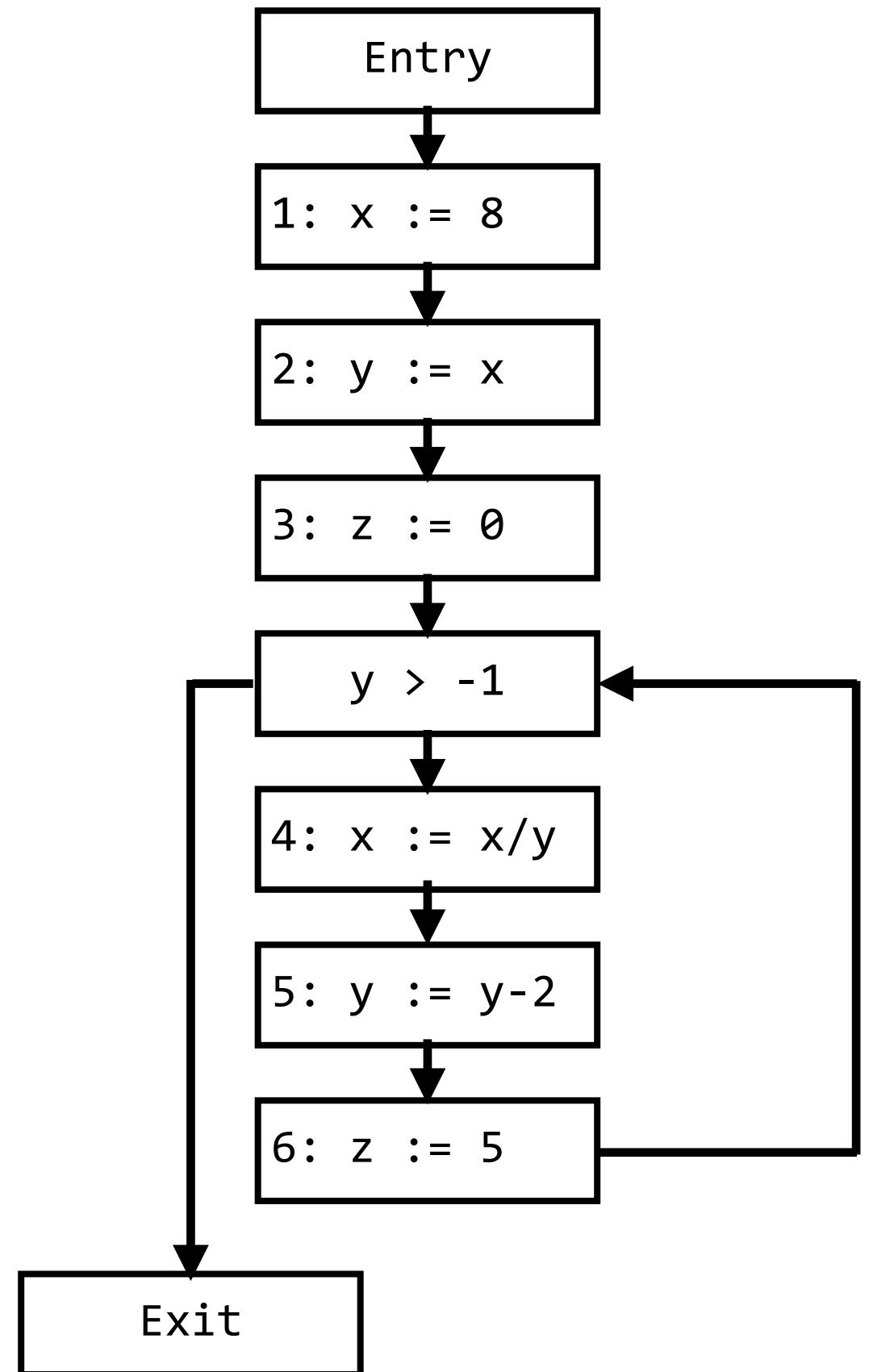
Transfer function: $F_x := e(\sigma) = \sigma[x \leftarrow \alpha_{ZI}(e)]$

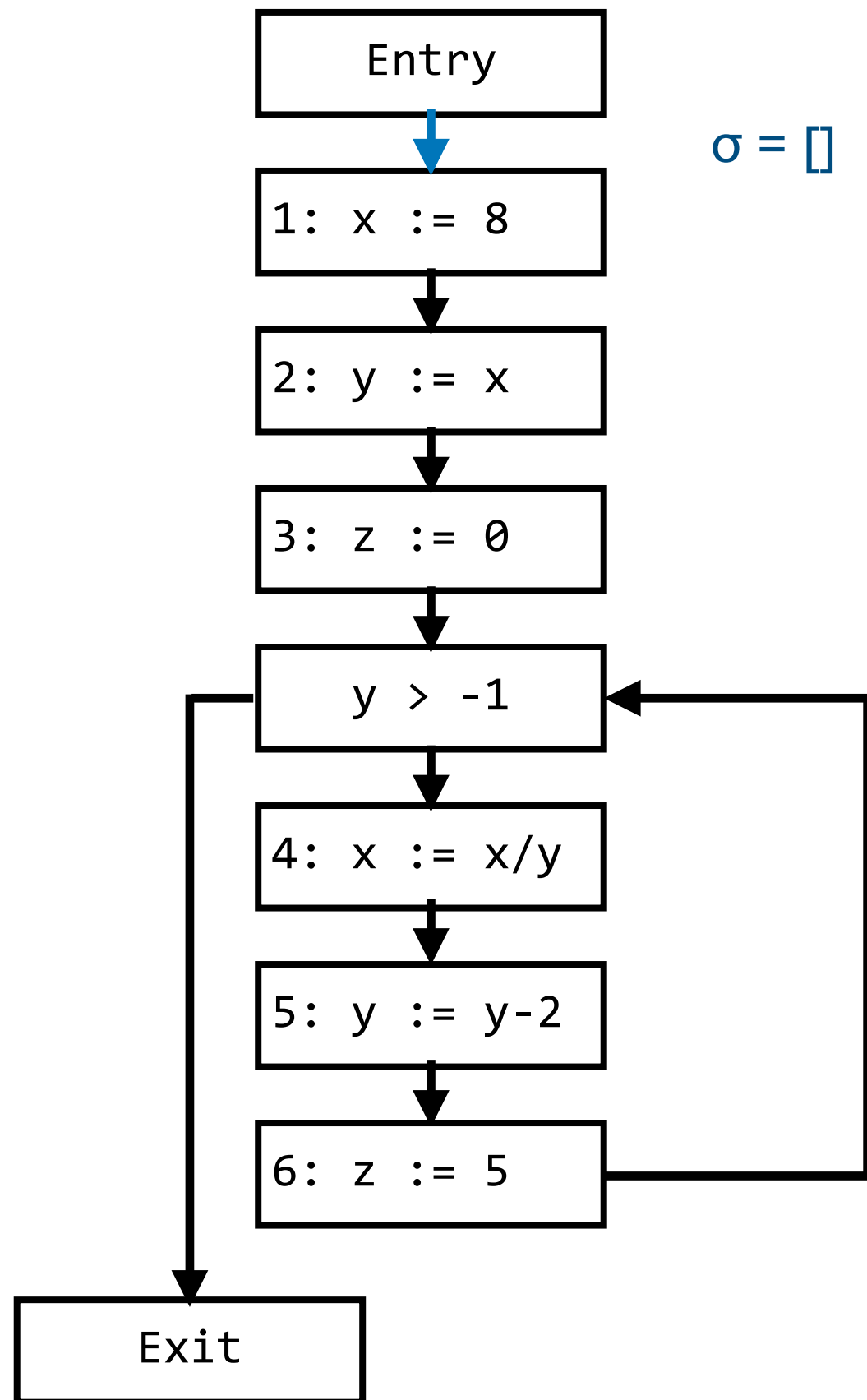
$\text{in}[n] := \bigcup \{ \text{out}[m] \mid m \in \text{pred}(n) \}$

$\text{out}[n] := \text{transfer}[n](\text{in}[n])$

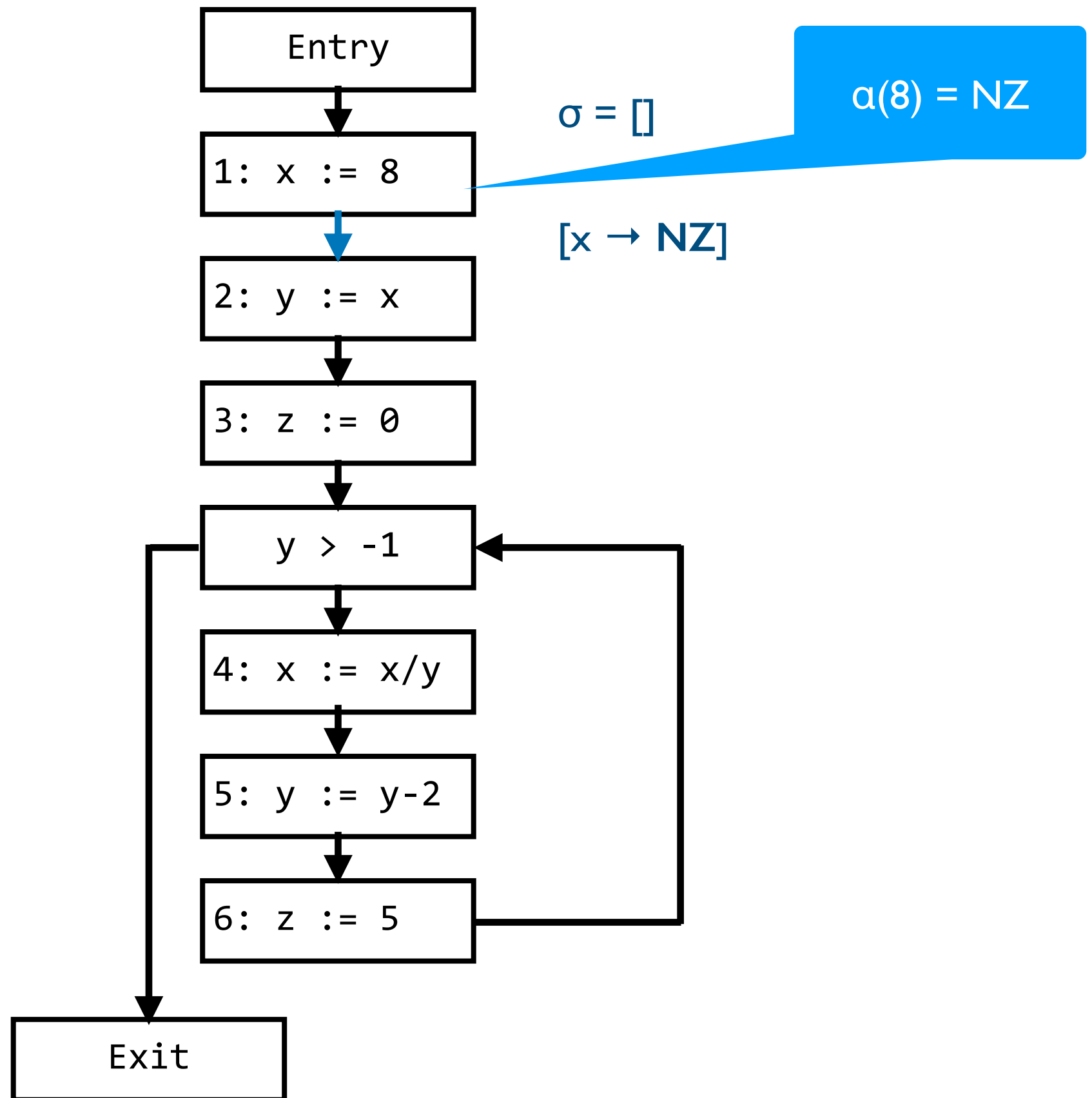
Example: Zero Analysis

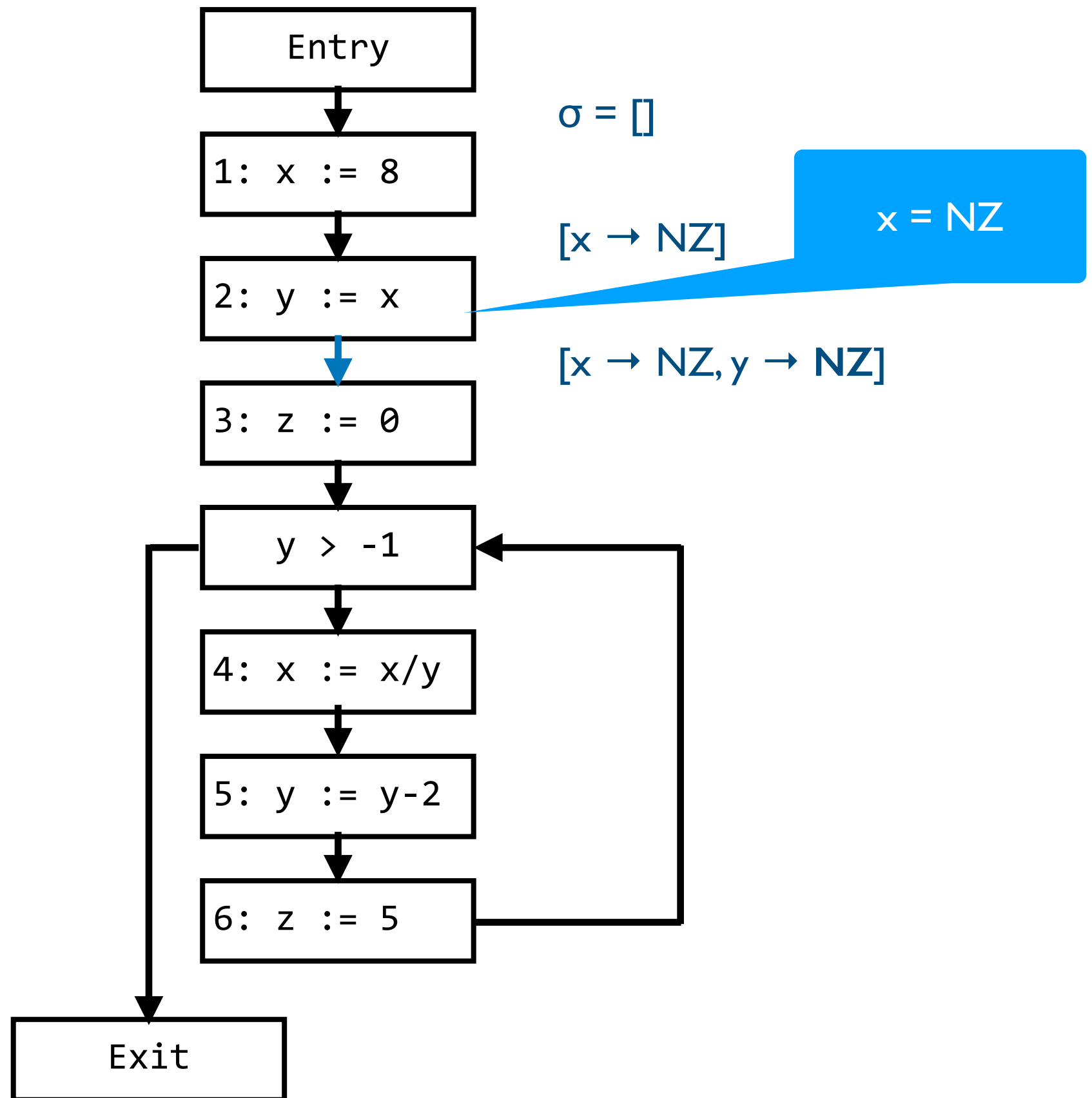
```
x := 8;  
y := x;  
z := 0;  
while y > -1 do  
  x := x / y;  
  y := y - 2;  
  z := 5;
```

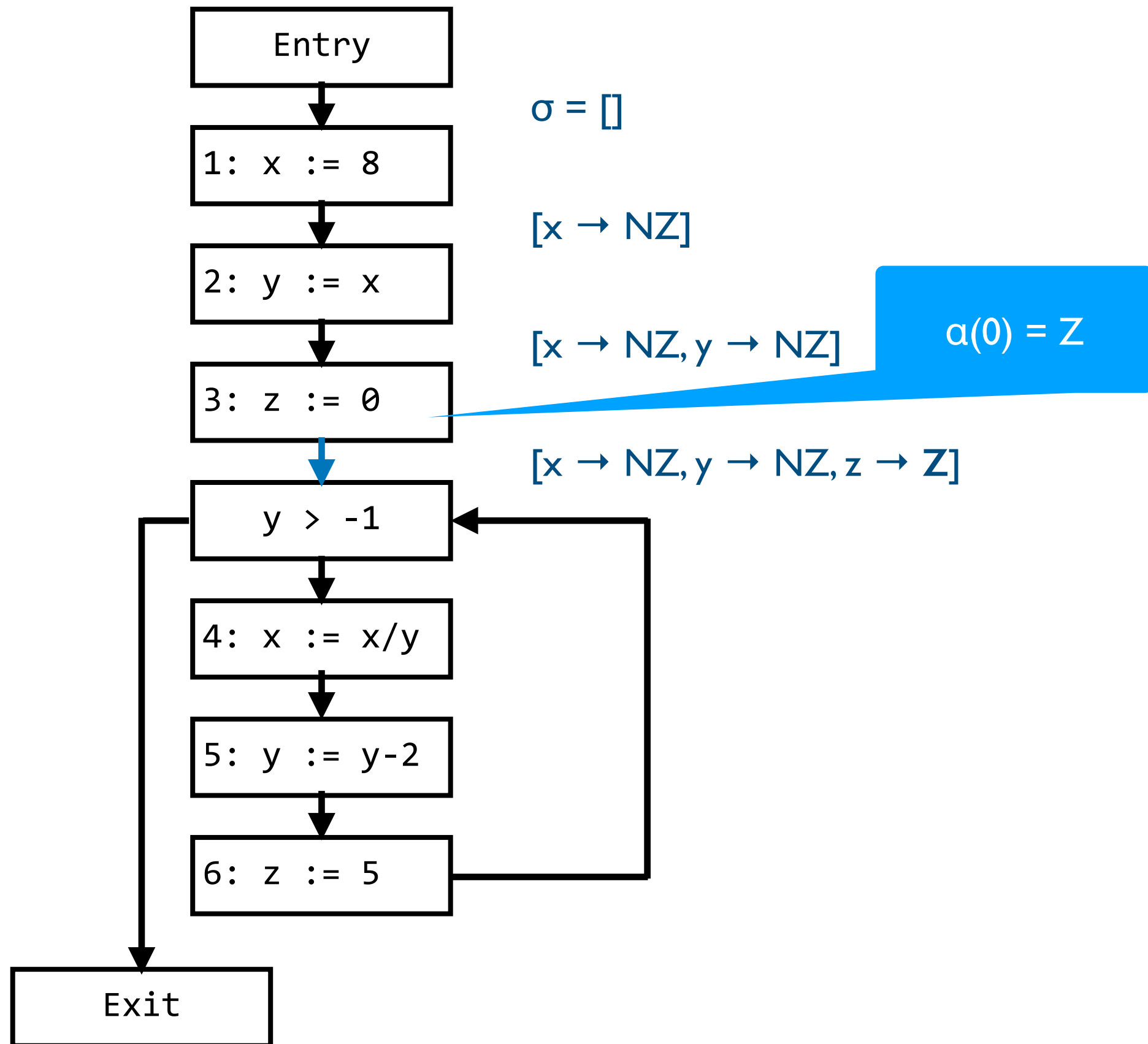


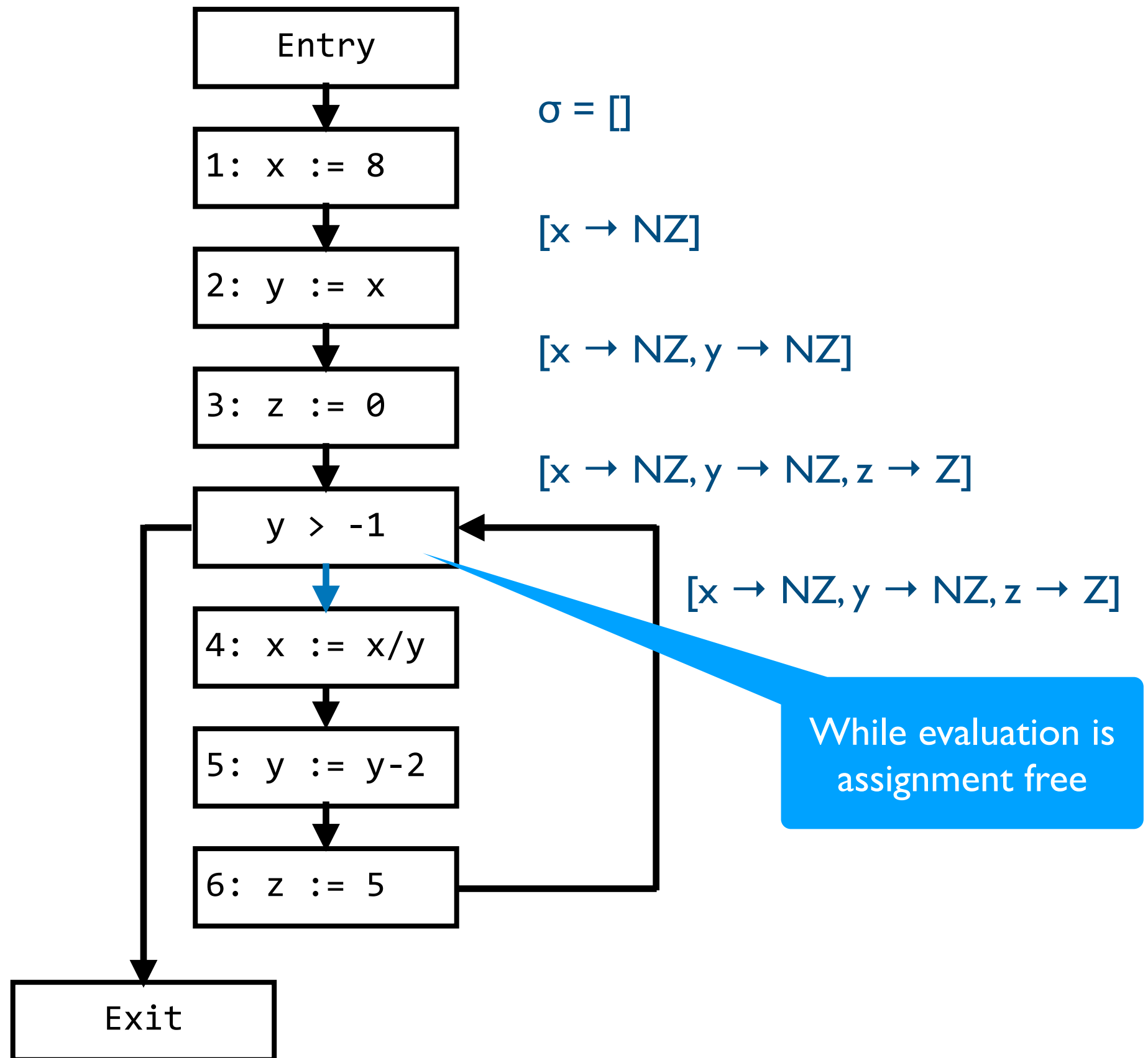


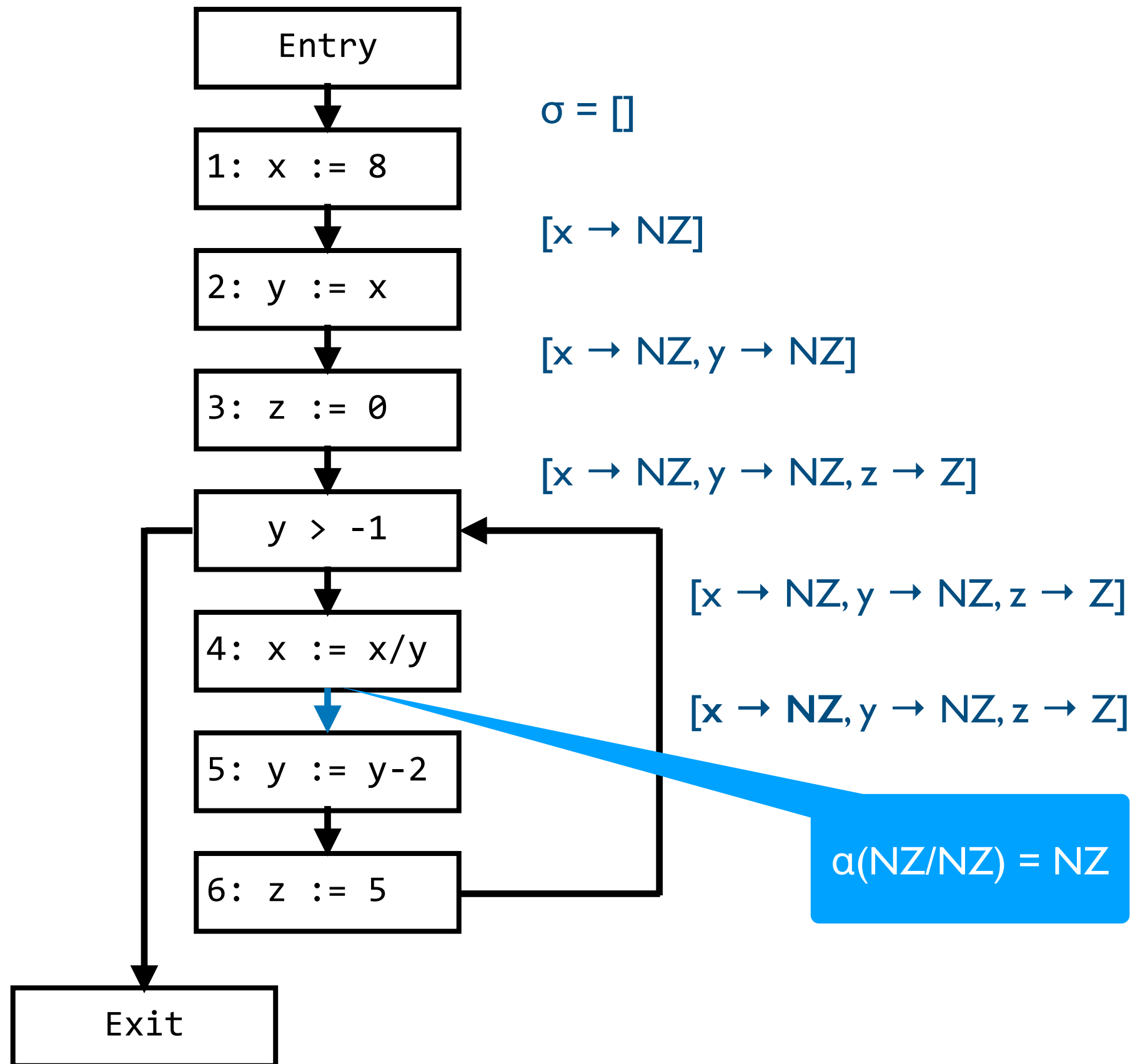
$\sigma = []$

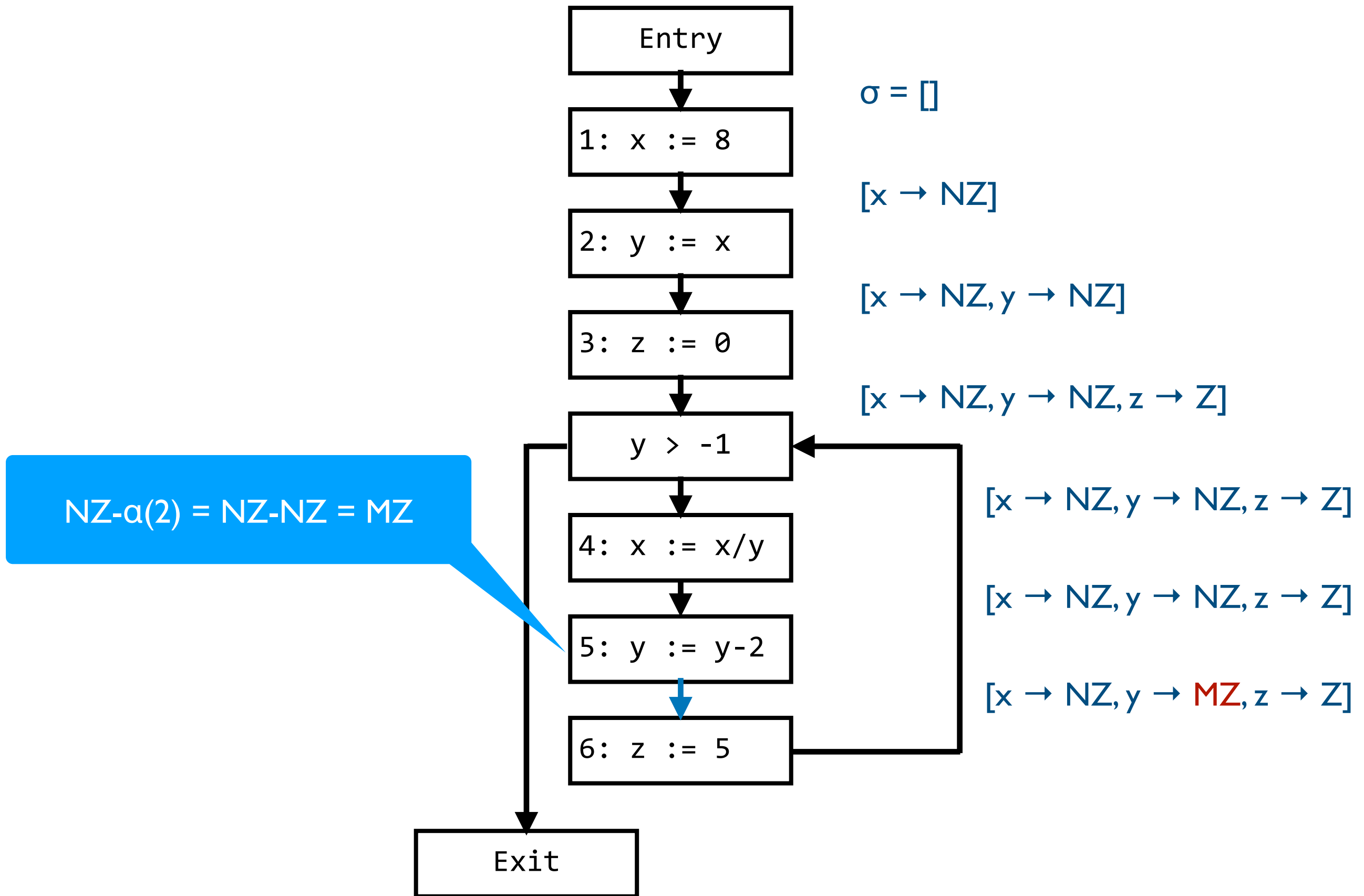


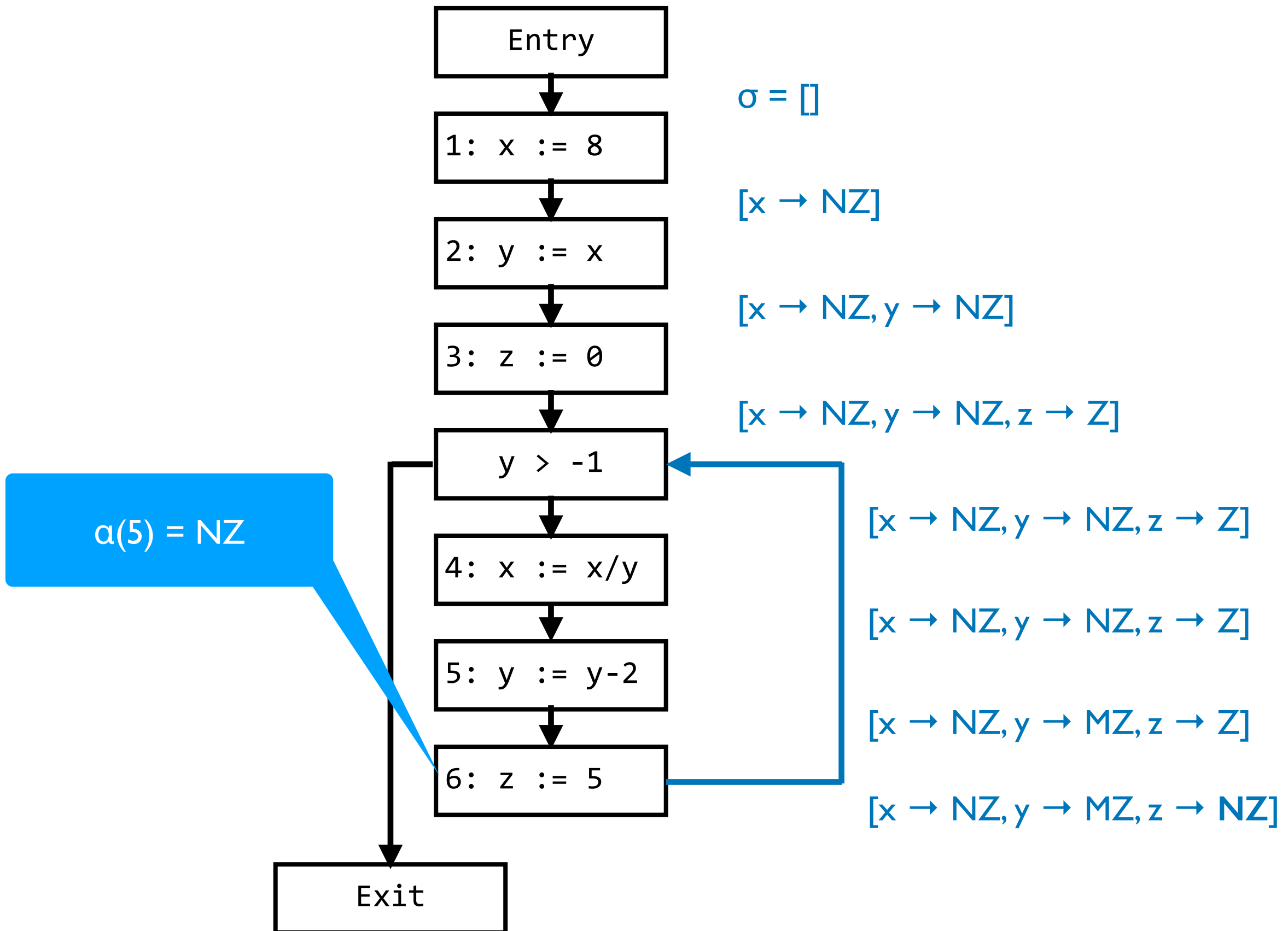




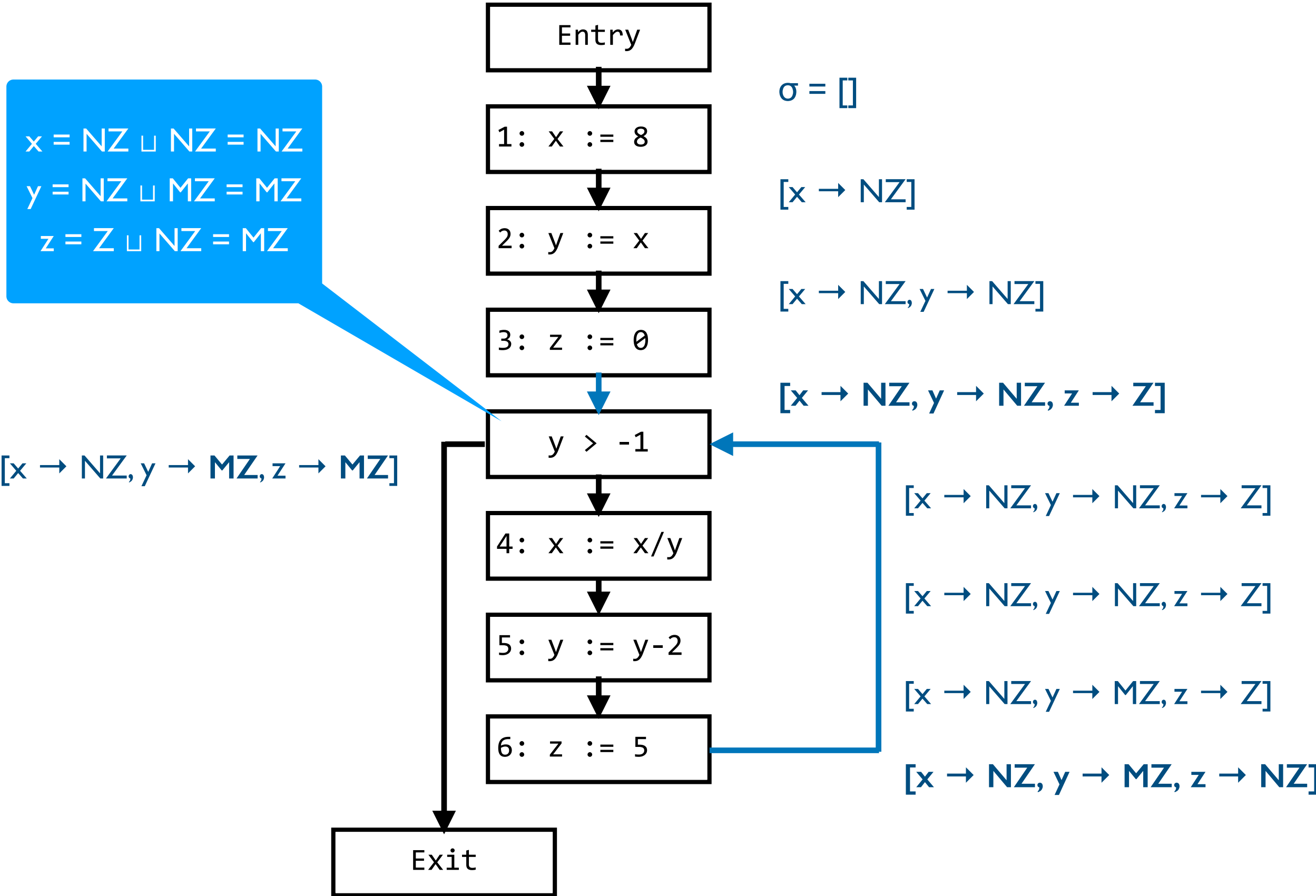


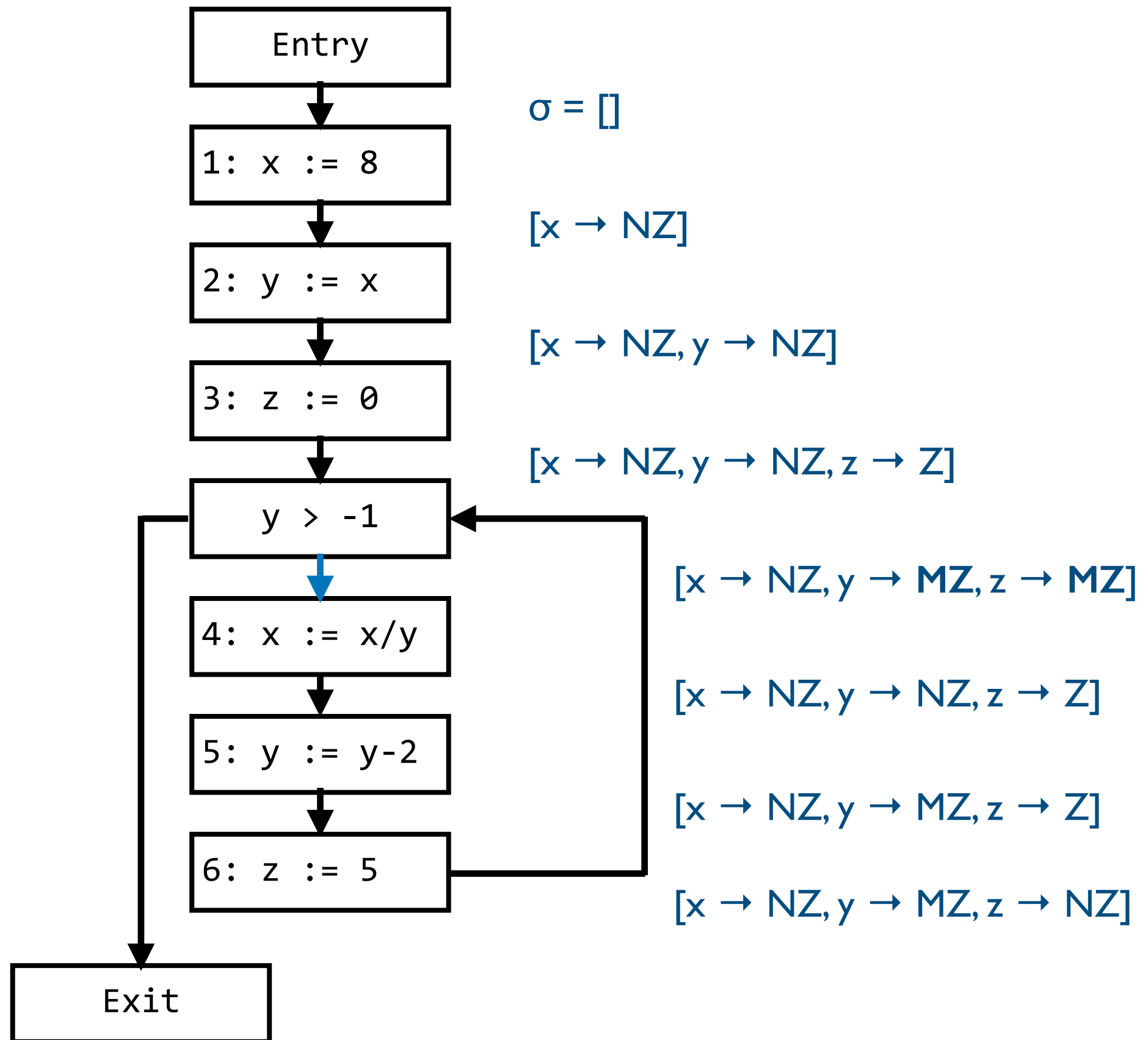


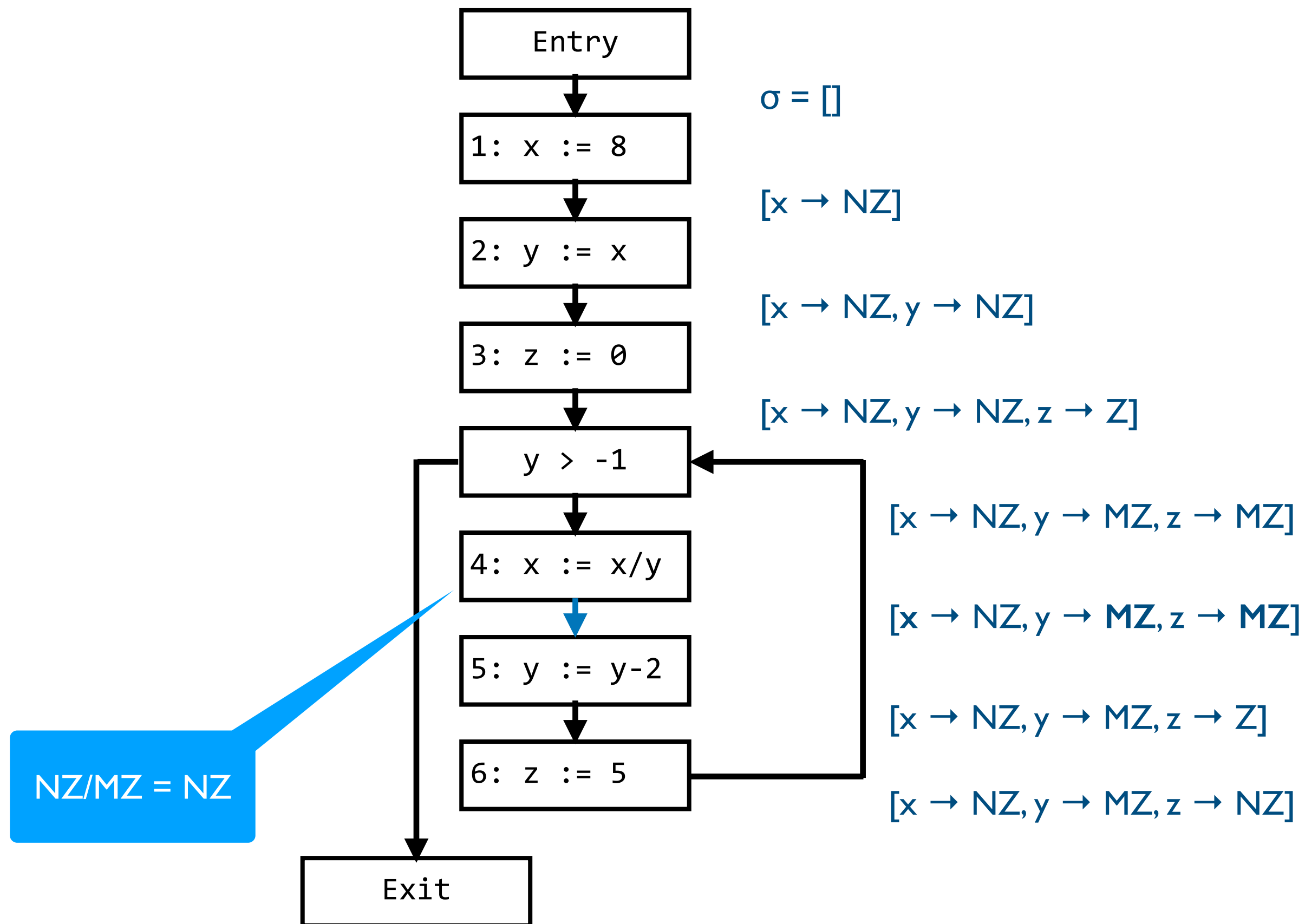


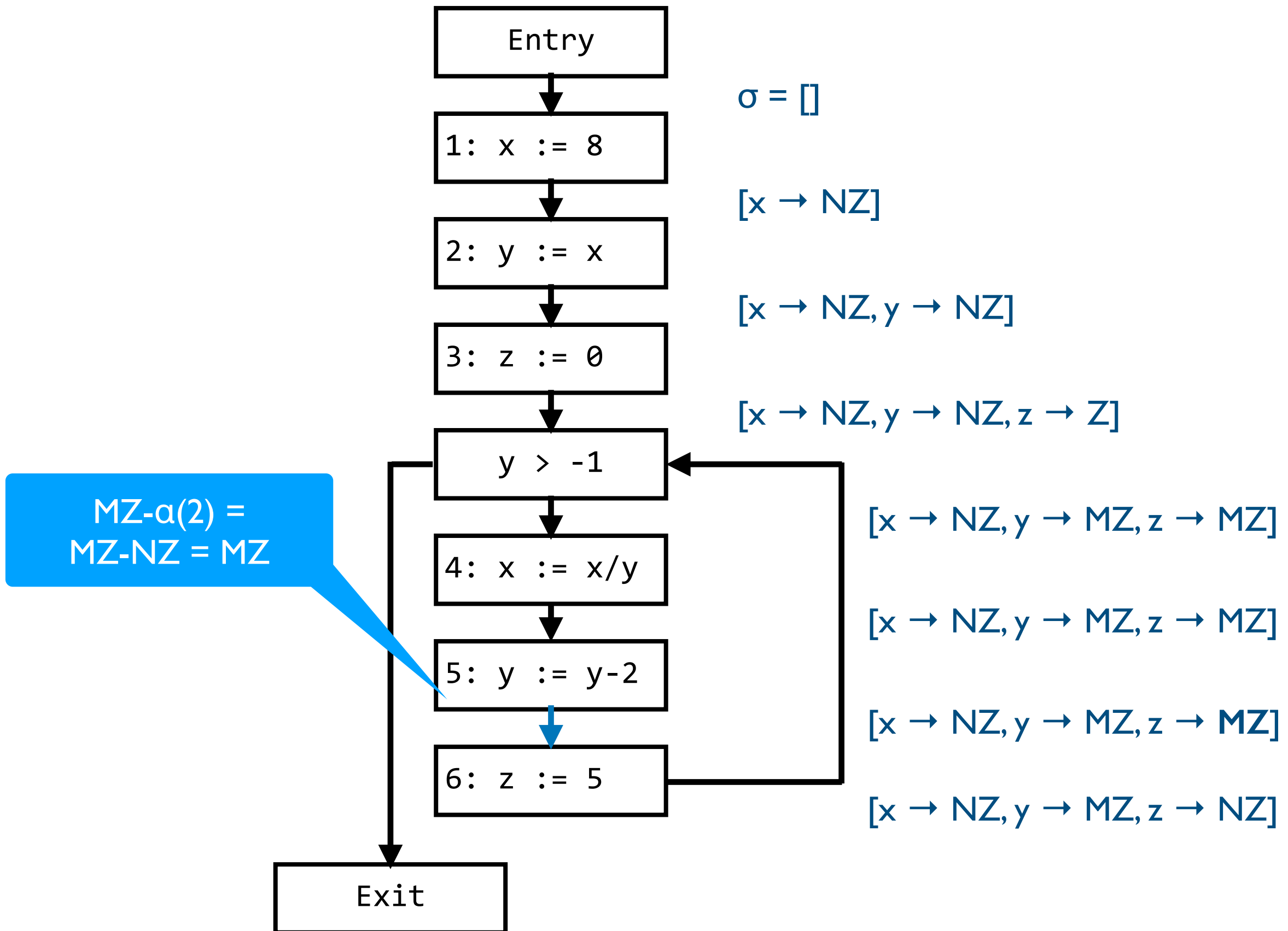


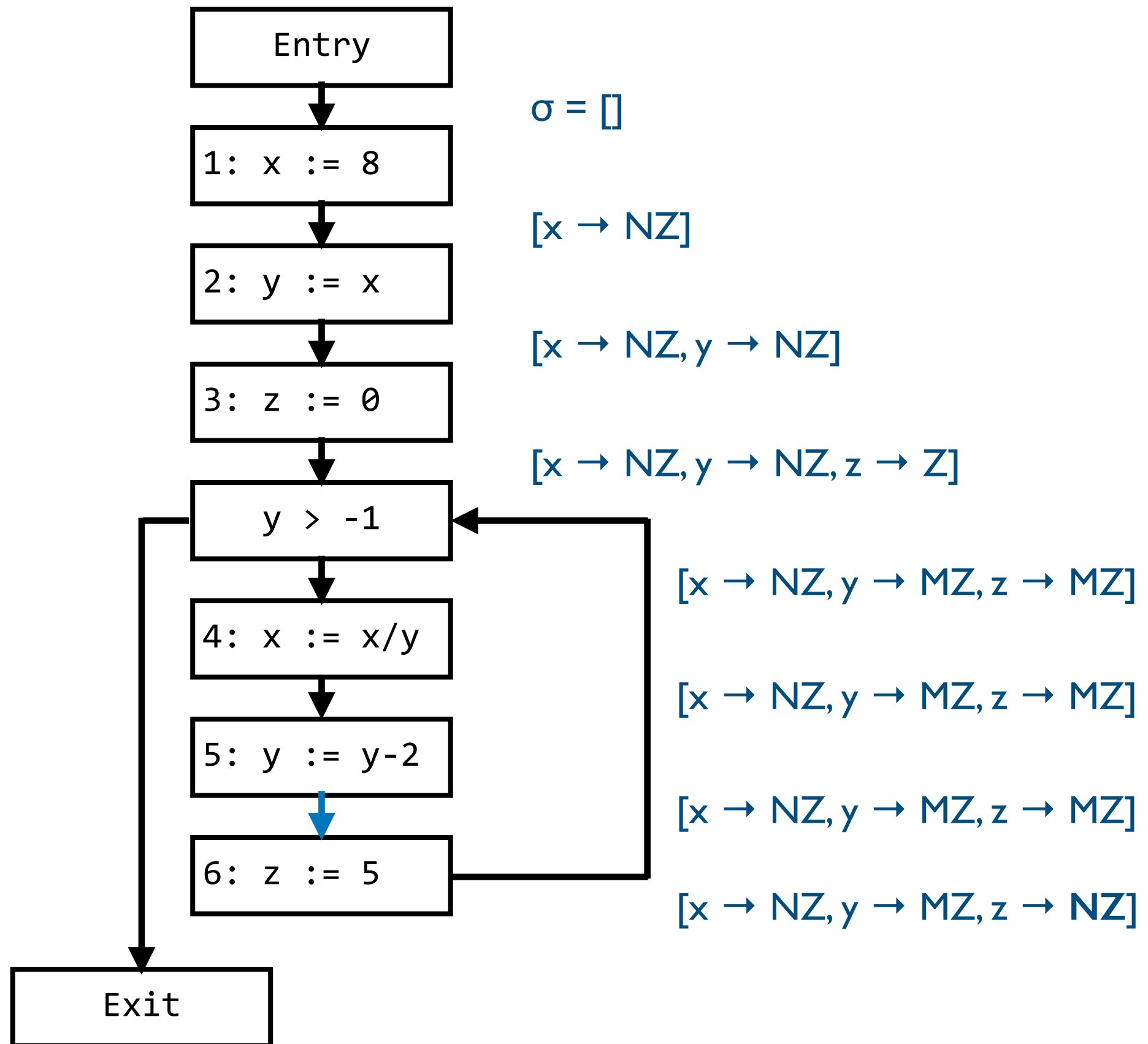
$x = \text{NZ} \sqcup \text{NZ} = \text{NZ}$
 $y = \text{NZ} \sqcup \text{MZ} = \text{MZ}$
 $z = \text{Z} \sqcup \text{NZ} = \text{MZ}$

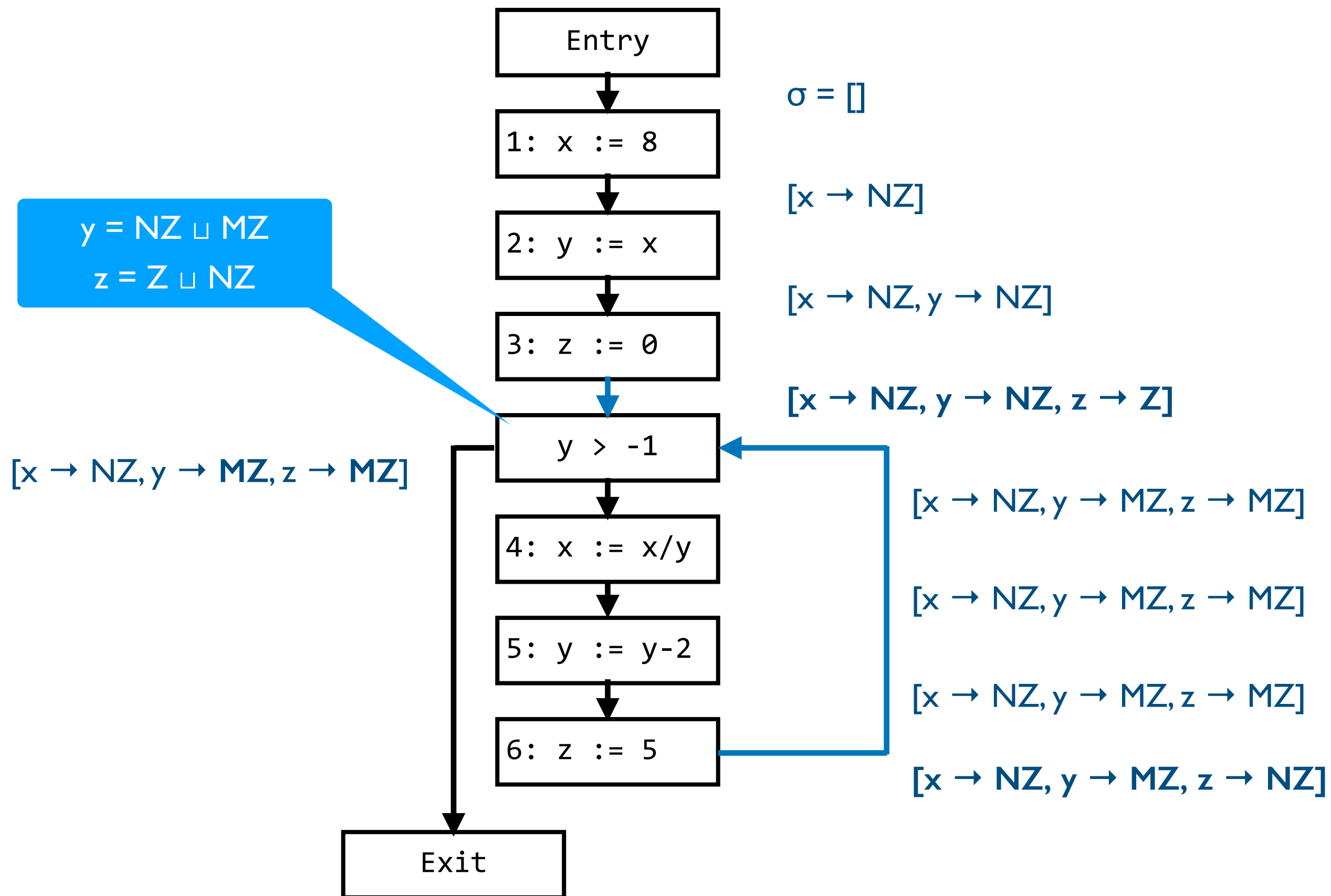












Example: Zero Analysis

Iteration 2

$\sigma = []$

$[x \rightarrow \text{NZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{NZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{NZ}, z \rightarrow \text{Z}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{NZ}]$

Iteration 3

$\sigma = []$

$[x \rightarrow \text{NZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{NZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{NZ}, z \rightarrow \text{Z}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$

$[x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{NZ}]$

No changes — fix point

Example: Zero Analysis

- Visit each division in the program
- Get the results of zero analysis for the divisor
- If the results are definitely zero, report an error
- If the results are possibly zero, report a warning

Example: Zero Analysis

```
x := 8;  
y := x;  
z := 0;  
while y > -1 do  
  x := x / y;  
  y := y - 2;  
  z := 5;
```

```
 $\sigma = []$   
x := 8;
```

```
 $\sigma = [x \rightarrow \text{NZ}]$   
y := x;
```

```
 $\sigma = [x \rightarrow \text{NZ}, y \rightarrow \text{NZ}]$   
z := 0;
```

```
 $\sigma = [x \rightarrow \text{NZ}, y \rightarrow \text{NZ}, z \rightarrow \text{Z}]$   
while y > -1 do
```

```
   $\sigma = [x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$ 
```

```
  x := x / y;
```

```
   $\sigma = [x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$ 
```

```
  y := y - 2;
```

```
   $\sigma = [x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{MZ}]$ 
```

```
  z := 5;
```

```
   $\sigma = [x \rightarrow \text{NZ}, y \rightarrow \text{MZ}, z \rightarrow \text{NZ}]$ 
```

Warning: This program
might produce a
division by zero

Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis
5. Lattice-Theoretic Framework
6. Example: Zero Analysis